

WENCHAO BAI

THEORETICAL TOOLS IN DB RESEARCH

Theoretical Tools in DB research

Lecture notes distilled from the SIGMOD 2026 full-corpus survey

Version 0.6 — Round-6 revision in progress.

Compiled 2026-09-15 09:37 CST (pdfL^AT_EX; sources under version control).

© 2026 Wenchao Bai. This work is licensed under the Creative Commons Attribution–NonCommercial 4.0 International Public License (CC BY-NC 4.0): you may share and adapt it for any non-commercial purpose with attribution — see <https://creativecommons.org/licenses/by-nc/4.0/> for details.

Typeset with the `tufte-book` class of the Tufte-L^AT_EX project (Apache 2.0), as bundled in Boaz Barak’s panbook (MIT); layout conventions follow Barak’s *Introduction to Theoretical Computer Science* lecture notes (text CC BY-NC-ND 4.0; conventions only, no content). Third-party license notices ship with the sources (`THIRD-PARTY-NOTICES.md`).

Contents

0	<i>Introduction: What DB Researchers Prove, and How</i>	7
0.0	<i>What these notes are.</i>	7
0.1	<i>The survey questions</i>	7
0.2	<i>Properties: first correct, then fast.</i>	8
0.3	<i>Tools: a trinity plus a riser.</i>	8
0.4	<i>Domains and dialects</i>	8
0.5	<i>The playbook: property $\times$ tool pairings.</i>	9
0.6	<i>Where theory sits.</i>	9
0.7	<i>Chapter map.</i>	10
0.8	<i>How these notes were made</i>	10
1	<i>Reductions</i>	13
1.0	<i>The tool: reductions</i>	13
1.1	<i>Gadget reductions: privacy under continual release.</i>	17
1.2	<i>Gap reductions: clustering with set outliers.</i>	19
1.3	<i>Both directions at once: volume-based answer diversity</i>	21
1.4	<i>Counting reductions: Shapley values for aggregate queries</i>	22
1.5	<i>Advanced usage and further reading</i>	24
1.6	<i>Appendix.</i>	27
2	<i>Concentration Inequalities.</i>	33
2.0	<i>The tool: concentration inequalities</i>	33

	2.1	<i>Trimmed statistics in a turnstile stream</i>	...
	2.2	<i>Maximum range sum by randomized grid shifting</i>	38
	2.3	<i>User-level private query processing with sampled tuples</i>	40
	2.4	<i>Advanced usage and further reading</i>	41
	2.5	<i>Appendix</i>	45
3		<i>Induction</i>	49
	3.0	<i>The tool: induction and invariants</i>	49
	3.1	<i>Resistance-distance labels on small-treewidth graphs</i>	52
	3.2	<i>Space-time plans for sum-product queries</i>	54
	3.3	<i>Near-duplicate text alignment by skyline sweeps</i>	56
	3.4	<i>Succinct representations of all width-1 join plans</i>	57
	3.5	<i>Advanced usage and further reading</i>	59
	3.6	<i>Appendix</i>	62
4		<i>Estimation Theory</i>	67
	4.1	<i>Estimators: the interface between sampling and guarantees</i>	67
	4.2	<i>The tool</i>	67
	4.3	<i>Exemplars from the SIGMOD'26 corpus</i>	69
5		<i>Adversarial Constructions</i>	73
	5.0	<i>The tool: adversarial constructions</i>	73
	5.1	<i>Defective clique enumeration: the output as adversary</i>	77
	5.2	<i>Proximity graphs: point sets that force edges</i>	79
	5.3	<i>Bi-core maintenance: an update pair that lies in wait</i>	81
	5.4	<i>Personalized PageRank: hard distributions with hidden splits</i>	83
	5.5	<i>Advanced usage and further reading</i>	85
	5.6	<i>Appendix</i>	88
6		<i>Differential Privacy Composition</i>	93
	6.1	<i>The tool</i>	93
	6.2	<i>Exemplars from SIGMOD'26</i>	94

7	<i>Exchange and Greedy Arguments</i>	97
7.1	<i>The Tool: Exchange Arguments and Their Guarantees</i>	97
7.2	<i>Five Views from the Corpus</i>	98
8	<i>Spectral and Matrix Tools</i>	103
8.1	<i>The tool: workhorse results</i>	103
8.2	<i>Three views from the corpus</i>	104
9	<i>Amortized Analysis and Potential Functions</i>	107
9.1	<i>The tool</i>	107
9.2	<i>Five views from the corpus</i>	108
10	<i>Coresets and Randomized Numerical Linear Algebra</i>	113
11	<i>Information-Theoretic Bounds</i>	115
12	<i>Communication Complexity</i>	117
13	<i>Online decision-making: competitive analysis and regret</i>	119
13.1	<i>The tools</i>	119
13.2	<i>Four exemplars from the corpus</i>	120

O

Introduction: What DB Researchers Prove, and How

0.0 What these notes are

These lecture notes are a *toolbox tour* of the theoretical machinery that database researchers actually use, extracted mechanically from the complete proceedings of **SIGMOD 2026**: 385 research papers (PACMMOD Vol. 4, rounds 3–4, plus Vol. 3 issues 4–6, rounds 1–2). Every paper was fetched (arXiv $\text{\LaTeX}$ sources when available, ACM camera-ready PDFs otherwise), machine-extracted into formal statements, proofs, and bound sentences (2,405 statements, 1,130 proofs), and 228 theory-relevant papers were tagged with a controlled vocabulary of properties, tools, domains, and roles. The aggregates behind every figure in this chapter come from that survey; every worked example in the later chapters is a real paper from the corpus, cited in that chapter’s reference list.

Two honesty markers used throughout:

- $\text{\texttt{PODS}}$ marks exemplars from the **PODS’26 issue** (PACMMOD Vol. 4 No. 2, DOI band 10.1145/3801890–3801919): 29 of the 228 tagged papers (12.7%). They are part of the proceedings bundle; they often provide particularly clean exhibits of a tool, but they carry the theory-first culture of a sibling conference.
- Formal statements are transcribed from machine extraction of the papers; where the extraction was noisy we paraphrase conservatively rather than decorate.

0.1 The survey questions

1. **Modeling** — how do papers formalize a systems problem (cost model, adversary, noise model, distance metric)?

2. **Properties** — what do they prove about the artifact: correctness, complexity bounds, approximation/competitive ratios, statistical guarantees?
3. **Tools** — which theoretical machinery delivers the proof?
4. **Role** — does theory motivate the work, drive the design, serve as the evaluation yardstick, or *is* it the contribution?

0.2 *Properties: first correct, then fast*

Figure 1 (left) ranks what the 228 tagged papers prove. The headline surprise of the full-corpus count: **correctness is #1** (98 papers, 43%) — ahead of upper bounds (75) and probabilistic guarantees (42). The typical SIGMOD theory section is not a big theorem; it is a small invariant certifying that a component preserves a property (monotonicity of timestamps, order-independence of index construction, merge-safety of compressed blocks). Hardness (36) plays a different role: the standard *license for heuristics* in graph search, repair, and mining papers. The tail is where culture lives: **tightness** (11 — matching upper and lower bounds, a $5.5\times$ jump over the arXiv-only view of the corpus) and freshly activated slots such as competitive ratio and regret bounds (2 papers each, entering through systems doors: when to adapt, when to stop).

0.3 *Tools: a trinity plus a riser*

Figure 1 (right) ranks the machinery. A *trinity* dominates: **reductions** (52 papers), **concentration inequalities** (39), and **induction** (36) — respectively the tools of the hardness culture, the statistical-certification culture, and the component-correctness culture. The big riser once the ACM-only mass arrived is **exchange-greedy** (4→17 papers): prove a selection problem is a submodular/metric instance, inherit the approximation factor. Chapter 0.7 (the index table) maps each tool to its chapter.

0.4 *Domains and dialects*

Figure 2 (left) shows where theory lives. Graph systems (56 papers) are now the dominant dialect — hardness → branch-and-bound and parameterized algorithms → index-correctness lemmas. Privacy (26) is a near-monoculture of composition accounting (69% of its papers), increasingly in Rényi/f-DP currency. Streaming and sketching splits into a pure-theory wing (matching lower bounds) and estimator systems;

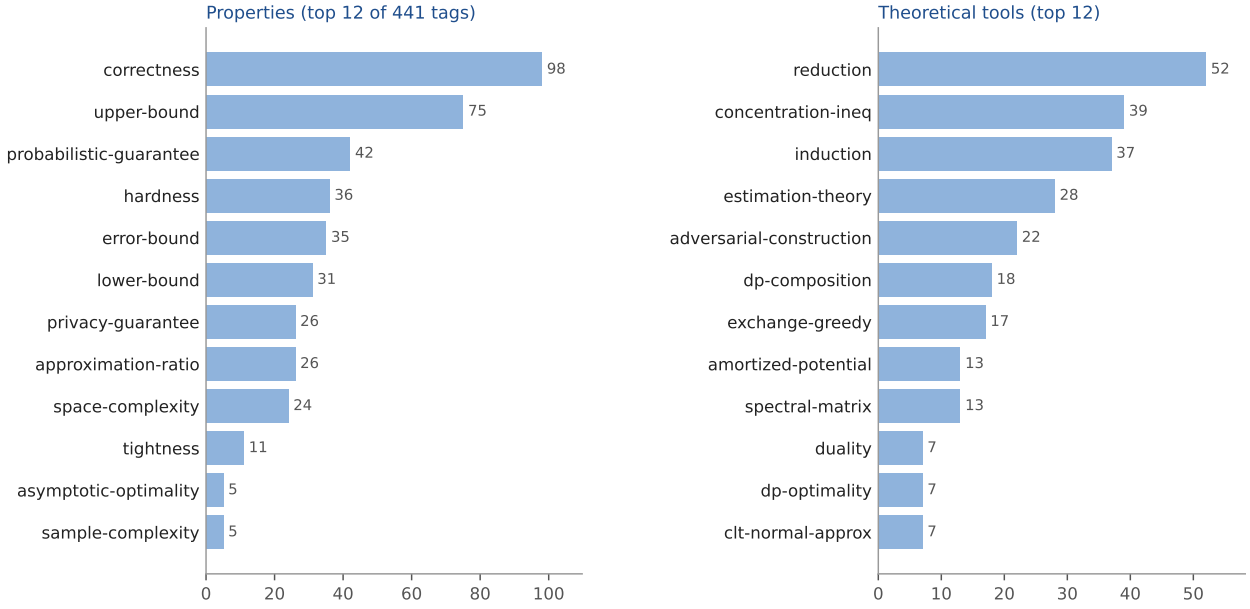


Figure 1: Left: most-studied properties (top 12 of 441 property tags on the 228 tagged papers). Right: most-used theoretical tools (top 12) — each later chapter is one tool.

vector search (25) splits between CLT-style certification and structural correctness of graphs and routers.

0.5 The playbook: *property* $\times$ *tool* pairings

Figure 2 (right) is the survey’s central cross-tab: which tool delivers which property. The two workhorses are **hardness** $\times$ **reduction** (32 papers) and **correctness** $\times$ **induction** (29). The pattern to internalize: DB theory compresses into about eight such pairings, and each chapter of these notes ends with the pairing(s) its tool owns.

0.6 Where theory sits

66% of tagged papers use theory as a *design driver*; in 25% the formal result *is* the paper (concentrated in the PODS issue and the pure streaming enclave); motivation-only and evaluation-yardstick roles cover the rest. The role split matters when reading exemplars: a design-driver paper proves what it builds; a theory-as-contribution paper builds nothing and proves everything.

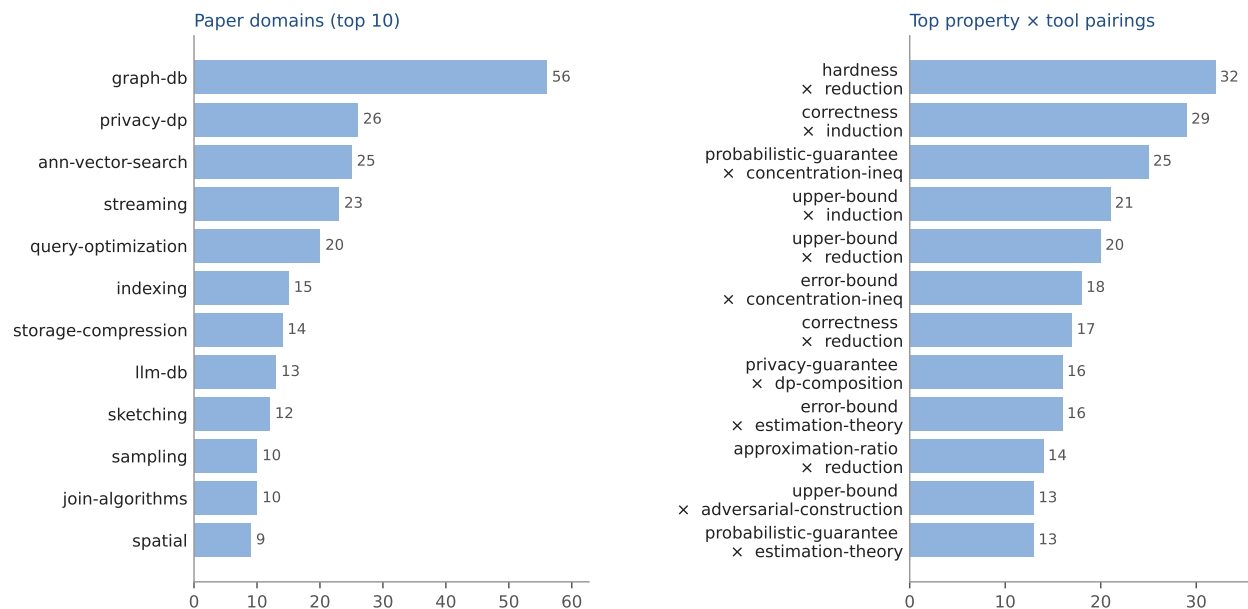


Figure 2: Left: paper domains (top 10). Right: top property $\times$ tool pairings (top 10) — the “playbook”.

0.7 Chapter map

Each of the following chapters is one tool: the formal core, then real SIGMOD 2026 exemplars with minimal context, then a “when to reach for it” box. Exemplars are transcribed from the papers’ own statements, with PODS-issue exemplars capped at one per chapter (two for communication complexity, whose corpus presence is small); a paper may recur in a second chapter when it is a clean exhibit of two tools — each chapter treats it under its own lens, and the ^{PODS} marker identifies PODS-issue exemplars in the text.

Tools not large enough for their own chapter in this corpus but worth knowing by name: adversarial-adjacent duality/LP arguments (7 papers), dynamic programming optimality (7), CLT/normal approximations (7), dyadic peeling (6), sketch mergeability theory (6), balls-bins-hashing (5) — several appear as guest stars inside the thirteen chapters.

0.8 How these notes were made

The survey was scripted end to end: the 385-paper list came from OpenAlex, sources were fetched (arXiv $\text{\LaTeX}$ when available, ACM camera-ready PDFs otherwise), statements, proofs, and bound sentences were machine-extracted (2,405 statements, 1,130 proofs), and the 228 theory-relevant papers were tagged against the controlled vocabulary above;

Table 1: Chapter map, ordered by how often the corpus uses each tool. “Papers” = tagged corpus papers using the tool; “Ex.” = exemplars featured in the chapter.

Ch.	Tool	Papers	Ex.
1	Reductions	50	6
2	Concentration inequalities	38	5
3	Induction	36	6
4	Estimation theory	27	6
5	Adversarial constructions	21	6
6	DP composition	18	6
7	Exchange-greedy arguments	16	6
8	Spectral / matrix tools	13	6
9	Amortized-potential analysis	11	6
10	Coresets & randomized NLA	8	6
11	Information-theoretic bounds	5	5
12	Communication complexity	4	4
13	Online decisions (competitive & regret)	4	4

every figure and count in this chapter is a direct aggregate of those tags.

These lecture notes are typeset with the `tufte-book` class of the Tufte- $\text{\LaTeX}$ project (Apache 2.0), as bundled in Boaz Barak’s panbook (MIT); the layout conventions — margin-column learning objectives, themed remark and recap blocks, end-of-proof markers, per-chapter references — follow Barak’s *Introduction to Theoretical Computer Science* notes (text licensed CC BY-NC-ND 4.0; we borrow the conventions only, no content). License notices for the distributed class files ship with the sources.

1

Reductions

Most “impossible” statements in database research are not proved from scratch: they are *imported*. A reduction exhibits a computable translation from a problem already believed hard — CLIQUE, Set Cover, inner-product estimation, hyperclique detection — into the problem at hand, so that any algorithm, approximation, private mechanism, or sampler for the target would also solve the source. The translation is a two-way conduit: instances travel forward through the gadget, and whatever the paper cares about — running time, approximation ratio, output count, memory, accuracy — travels backward, inheriting the source’s hardness. Run in the constructive direction, the same machinery turns new problems into old ones with known algorithms. Reductions are the most used theoretical tool in our SIGMOD’26 corpus: 50 of 228 tagged papers rely on them, 32 of those pair the reduction with an explicit hardness claim, and the heaviest concentration is graph analytics (14 papers).

This chapter builds the ladder of reduction types first — each rung named by the *currency* it preserves — and walks four corpus exemplars up it; Section 1.5 adds the advanced rungs (fine-grained hypotheses, reduction chains, algebraic transfer lemmas) as extension reading.

1.0 The tool: reductions

The problem class is uniform across applications. Somewhere there is a source problem H that a community has already failed to solve — CLIQUE, Set Cover, Disjointness, Hamiltonian path — together with evidence of its hardness: a proof (NP-completeness), a widely believed conjecture (the Unique Games Conjecture), or a quantitative hypothesis (SETH, combinatorial Boolean matrix multiplication, communication lower bounds). The paper’s target problem P is the one the reader actually cares about. What a reduction supplies is a pair of polynomial-time maps: a *gadget* f that builds an instance of P out of every instance of H , and a *decoder* g that reads a solution of quality q for $f(x)$ back into

LEARNING OBJECTIVES

- Recognize claims that need *imported* hardness, and pick the source problem.
- Match the rung to the currency: decision, ratio, count, time, space.
- Audit a gadget: instances forward, quality backward, parameters in every regime.
- Run reductions backwards to certify tractability — then glue both sides into a dichotomy.

a solution of quality $g(q)$ for x . Composing the two, any solver for P of quality q is a solver for H of quality $g(q)$; if H 's hardness statement rules that out, it rules out the solver for P . Figure 1.1 schematizes the two halves.

Hypotheses and currencies

Throughout, the *source* H carries a hardness statement at some resolution: qualitative (NP-hardness, #P-completeness), ratio-quantitative (no r -approximation under the Unique Games Conjecture), or fully quantitative (no $O(n^{2-\epsilon})$ algorithm under SETH; no $o(m)$ -communication protocol for Disjointness). The *currency* of a reduction is the quantity that travels backward through the decoder: yes/no membership, objective value (hence approximation ratio), output count, running time, memory, or estimation error. The target inherits exactly the source's resolution — no more: an NP-hardness source cannot yield an inapproximability ratio, and a conditional source yields a conditional bound.

1.0.1 The ladder

The classical rung preserves decision membership [6, 8, 10].

Definition 1.1 (Polynomial-time many-one reduction). Let L_1, L_2 be decision problems. We write $L_1 \leq_m^p L_2$ if there is a polynomial-time computable function f with $x \in L_1 \iff f(x) \in L_2$ for all x . ■

Proposition 1.2 — Hardness transfer. If L_1 is NP-hard and $L_1 \leq_m^p L_2$, then L_2 is NP-hard. ■

Proof. A polynomial-time algorithm for L_2 , composed with f , decides L_1 ; a polynomial composition of polynomials is polynomial. ■

This rung already reaches into databases, at the very origin of query-evaluation complexity [4]:

Example 1.3 (Conjunctive-query evaluation is NP-complete). Given a graph G and k , build in polynomial time the database D_G holding the edge relation of G , and let Q_k be the conjunctive query

$$Q_k = \exists v_1 \dots \exists v_k. \bigwedge_{1 \leq i < j \leq k} E(v_i, v_j). \quad \blacksquare$$

Then G has a k -clique iff $Q_k(D_G) \neq \emptyset$. Since CLIQUE is NP-complete, CQ evaluation is NP-hard (and it lies in NP).

Database problems, however, usually carry objectives — cost, error, latency — so the reduction must preserve *quality*, not just membership.

The standard rung is the approximation-preserving, or L -, reduction [6, 13]:

Definition 1.4 (L -reduction). An optimization problem A L -reduces to B if there are polynomial-time maps f (instances) and g (solutions) and constants $\alpha, \beta > 0$ such that, for every instance x of A , writing c_A, c_B for the objective values: (i) $\text{OPT}_B(f(x)) \leq \alpha \cdot \text{OPT}_A(x)$; and (ii) every solution y of $f(x)$ maps to a solution $g(y)$ of x with $|c_A(g(y)) - \text{OPT}_A(x)| \leq \beta \cdot |c_B(y) - \text{OPT}_B(f(x))|$. ■

Proposition 1.5 — Ratio transfer. If A L -reduces to B with constants α, β and B admits a polynomial-time r -approximation, then A admits an $\alpha\beta r$ -approximation. Hence inapproximability of A — e.g. Set Cover's $\ln n$ threshold — transfers to B . ■

Proof. Apply the B -solver to $f(x)$ and decode by g : chaining (i) and (ii), the excess of $c_A(g(y))$ over $\text{OPT}_A(x)$ is at most $\alpha\beta$ times the excess of $c_B(y)$ over $\text{OPT}_B(f(x))$, so the achieved cost is within a $1 + \alpha\beta(r - 1)$, hence an $\alpha\beta r$, factor of the optimum. ■

1

When polynomial versus exponential time is too coarse, the source carries a quantitative hypothesis, and the reduction must *audit parameters* [14]:

Proposition 1.6 — Fine-grained transfer. Suppose a polynomial-time gadget f translates instances of H with parameters π into instances of P , and a solver for P can be leveraged (with polynomial postprocessing) using $N(\pi)$ calls on produced instances of size $s(\pi)$. Then any algorithm for P running in time T on size- $s(\pi)$ inputs yields an algorithm for H of cost $\text{poly}(\pi) + N(\pi) \cdot T(s(\pi))$. In particular, if this beats the time lower bound assumed for H in every parameter regime, the translated bound holds for P under the same hypothesis. ■

Proof idea. Count the cost of the emulation; the caveat is the whole point — a gadget whose parameter blow-up is benign in one regime and wild in another proves nothing there (cf. Section 1.1 for the audit done right). ■

Two further rungs complete the ladder. For counting problems the currency is the output count, and the natural notion is oracle access:

Definition 1.7 (Counting (Turing) reduction). A counting problem $\#A$ reduces to $\#B$, written $\#A \leq_{\#}^T \#B$, if $\#A(x)$ is computable in polynomial time given oracle access to $\#B$ on instances constructible in polynomial time. ■

¹ A *gap* reduction is the same idea with the ratio built into the gadget: f maps yes-instances to solutions of cost $\leq c$ and no-instances to instances where every solution costs $> \gamma c$, so any $(\gamma - \varepsilon)$ -approximation for B would decide H . Gap form and L -form are interchangeable currency for ratios; the corpus exemplar of Section 1.2 uses the L -form against Set Cover.

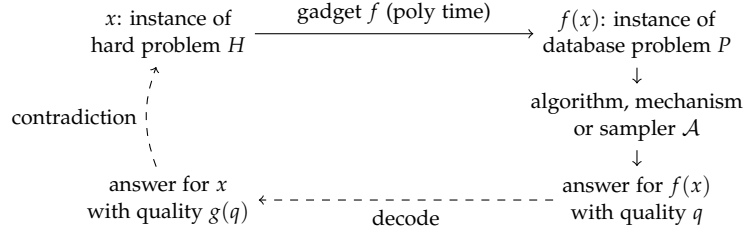


Figure 1.1: Every reduction has two halves: instances travel forward through the gadget; the currency — membership, cost ratio, count, time, space, or accuracy — travels backward, so quality achieved for P becomes quality for H , which the source’s hardness forbids.

Proposition 1.8 — #P-hardness transfer. If $\#A$ is #P-complete and $\#A \leq_{\#}^T \#B$, then $\#B$ is #P-hard. ■

Proof. Substitute the oracle: a polynomial-time algorithm computing $\#B$ turns the Turing reduction into a polynomial-time algorithm for the #P-complete $\#A$. ■

Finally, for streaming algorithms the classical bridge converts space into communication:

Proposition 1.9 — One pass, one message. If a one-pass streaming algorithm with s bits of memory computes a stream function F , then the two-party problem “Alice holds a stream prefix x , Bob holds the suffix y , compute $F(x \cdot y)$ ” has a deterministic protocol of communication s . A t -pass algorithm gives communication $O(s \cdot t)$. Consequently, any $\Omega(c)$ communication lower bound for the two-party problem forces $\Omega(c)$ (resp. $\Omega(c/t)$) space for one-pass (resp. t -pass) streams. ■

Proof. Alice simulates the algorithm on her prefix and sends Bob the final memory state — s bits, from which Bob resumes and finishes the pass. Multiple passes cross the channel once per pass per direction. ■

1.0.2 How to choose the rung

The selection procedure is mechanical once two questions are answered. *First, what must the claim rule out?* An algorithm (any quality) — rung of Definition 1.1; an approximation at some ratio — Definition 1.4 or its gap form; a fast algorithm at a refined running time — Proposition 1.6 under a quantitative hypothesis; a counter — Definition 1.7; a memory-frugal streaming algorithm — Proposition 1.9 against a communication bound. *Second, what does the source’s hardness actually say?* The target inherits the source’s resolution exactly: from Set Cover one may import a $\ln n$ inapproximability (under UGC) but not NP-hardness of approximation beyond it; from Disjointness, an $\Omega(m)$ space bound but

nothing about time; from SETH, quantitative time bounds conditioned on the hypothesis. Picking the source closest in *shape* to the target is what keeps the gadget small: covering structure suggests Set Cover, subgraph patterns suggest CLIQUE or hypercliques, private estimation suggests InnerProduct, enumeration suggests Hamiltonian path.

When to Reach for Reductions

Reach whenever a claim of the form “no system can do X ” or “doing X requires resource Y ” must be justified — or, constructively, whenever a new problem should inherit an existing algorithm. Then run the recipe:

1. Pick the canonical source problem closest in shape: Set Cover for covering and outlier budgets, CLIQUE/biclique for subgraph patterns, InnerProduct for private estimation, #Set Cover or the permanent for counting, Disjointness for stream space, Hamiltonian path for exact combinatorial targets.
2. Build the gadget f (instances forward) and the decoder g (quality backward); fix the currency and the matching rung of the ladder.
3. State explicitly which hypothesis the reader inherits: a proof (NP-, #P-), a conjecture (UGC), or a quantitative hypothesis (SETH, combinatorial BMM, a communication bound). Conditional claims must wear their conditions.
4. Audit the parameter blow-up of the gadget in *every* regime, and the ratio algebra ($\alpha\beta r$, Proposition 1.5) — a sloppy parameter map makes the bound vacuous somewhere.
5. Run the machinery backwards for the upper-bound side (reduce to LP feasibility, submodular maximization, dynamic programming); a dichotomy is two reductions glued at a common frontier.

1.1 Gadget reductions: privacy under continual release

The exemplar is a PODS’26 paper on differential privacy for streaming graphs [2]^{PODS}. When sensitive data arrives as a stream — friendship edges, transactions, sensor readings — a private system cannot wait for the stream to end: it must keep answering queries while the data grows, and every published answer leaks a little. The *continual release* model asks how accurate such a stream of answers can be while satisfying differential privacy throughout. Prior work had established

lower bounds only in restricted settings; this paper improves them to polynomial bounds for three graph statistics systems actually maintain — the maximum matching size, the degree histogram, and the k -core number — proving that any private incremental mechanism must incur additive error polynomial in the stream length T or the graph size n . It also gives the first lower bounds for Top- K norm estimation, extends the technique to monotone symmetric norms, and matches the lower bounds with binary-tree mechanisms whose error is polynomial with better-behaved constants — so the frontier it draws is essentially tight.

Problem. Work in the continual-release streaming model: an n -vertex graph evolves over T timesteps through a stream of edge insertions (*incremental streams*), and at every timestep the mechanism must output an estimate of a graph statistic g (*maintaining it*). Privacy is event-level differential privacy: any two streams differing in a single update event must yield indistinguishable output distributions. The mechanism has *additive error* α if, with probability at least $2/3$, at every timestep its estimate is within α of the true value $g(G)$ of the current graph.

2

Guarantee.

Theorem 1.10 — Continual-release lower bound, Theorem 1 of the paper. Let $\varepsilon \in [0, 1]$ and $\delta \in [0, 1/3]$. Any (ε, δ) -DP incremental mechanism in the exact setting for maintaining the size of the maximum matching, the degree histogram, or the core number of a vertex must have additive error

$$\alpha = \Omega(\min\{T^a, n^b\}),$$

for problem-specific constants $a, b \in [1/8, 1/2]$. ■

How the reduction is applied. The source problem is INNERPRODUCT: a secret dataset $x \in \{0, 1\}^d$ and queries $q^1, \dots, q^m \in \{0, 1\}^d$, with the mechanism answering every $\langle x, q^i \rangle$; a packing argument shows any (ε, δ) -DP mechanism for it has error $\tilde{\Omega}(\sqrt{d})$. The gadget is an *extendable bitwise-AND gadget*: a family of graphs H_x^i and Q_x^i , built from $v(d)$ vertices and $t(d)$ edges, in which the statistic's value jumps by exactly $\langle x, q^i \rangle$ between two queryable stream prefixes, and flipping one bit of x changes one stream event — event-level neighboring streams, so privacy of the graph mechanism implies privacy of the inner-product answers. The transfer is the paper's workhorse lemma:

Lemma 1.11 — Gadget transfer, Lemma 1 of the paper, notation cleaned. Let g be a graph problem that admits an extendable bitwise-AND gadget with $v(d)$ vertices and $t(d)$ edges per inner-product dimension. Any

² The “exact setting” in the paper’s statements means the mechanism maintains the exact statistic up to this additive error — no multiplicative slack is allowed. The exponents a, b are problem-specific: $1/4$ and $1/4$ for matching, $1/4$ and $1/2$ for the degree histogram, $1/8$ and $1/4$ for k -core.

(ε, δ) -DP event-level mechanism that maintains g in the exact setting, on n -node graphs over T timesteps, with additive error α , satisfies

$$\alpha = \tilde{\Omega}(\min\{t^-(T), v^-(n)\}), \quad f^-(x) = \sup\{d \mid f(d) \leq x\}. \quad \blacksquare$$

Proof sketch. Query the mechanism just before and just after inserting the edges that encode q^j ; the two estimates differ by $\langle x, q^j \rangle$ up to 2α , so the graph mechanism is an (ε, δ) -DP mechanism for INNERPRODUCT with error 2α ; the packing bound $\tilde{\Omega}(\sqrt{d})$ forces $\alpha \geq \tilde{\Omega}(\sqrt{d})$ with d set to the largest dimension the budget T, n admits. (Full proof in Section 1.6.) $\blacksquare$

The δ -tuning analogue is the *parameter audit*: d is chosen exactly as $d = \min\{t^-(T), v^-(n)\}$, the largest dimension whose gadget fits both the timestep budget T and the vertex budget n . Lemma 2 of the paper supplies the gadgets — $v(d) = t(d) = O(d^2)$ for matching, $v(d) = O(d)$ and $t(d) = O(d^2)$ for the degree histogram, $v(d) = O(d^2)$ and $t(d) = O(d^4)$ for k -core — and the audit turns them into exponents: matching gives $\alpha = \tilde{\Omega}(\min\{T^{1/4}, n^{1/4}\})$, the histogram $\alpha = \tilde{\Omega}(\min\{T^{1/4}, n^{1/2}\})$, and k -core $\alpha = \tilde{\Omega}(\min\{T^{1/8}, n^{1/4}\})$ — the constants $a, b \in [1/8, 1/2]$ of Theorem 1.10, with the polylogarithmic factors absorbed.

What it buys. The system designer learns that continually maintaining these statistics privately is polynomially expensive, *and where the escape hatches are*: the paper’s companion upper bounds (Theorem 2 of the paper) achieve error $O_{\varepsilon, \delta}(\min\{T^c, n^d\})$ with $c, d \in [1/6, 1/2]$ via binary-tree mechanisms and DP composition, while relaxing to the static setting — or allowing a $(1 + \zeta)$ multiplicative approximation — admits mechanisms with polylogarithmic error. The reduction did not just import hardness; it located the exact regime where hardness lives.

1.2 Gap reductions: clustering with set outliers

The exemplar is a SIGMOD’26 paper on robust clustering [12]. Clustering with outliers is a workhorse of robust analytics: alongside the k centers, the system may discard some data as unrepresentative. Prior models discard individual points; this paper discards whole *sets* from a candidate family — hyper-rectangles covering a spatial region (a maintenance window, a blackout area), or sets of tuples induced by a relational join, so that clustering can run directly over the join without materializing it. The paper develops near-linear-time $(2 + \varepsilon, 2, O(1))$ -approximation algorithms, coresets that shrink the instance to $O(\min\{n, k^2z, km\})$ points, and high-probability algorithms for the relational variants — and justifies the shape of these guarantees with a hardness result: under the Unique Games Conjecture, no polynomial-time algorithm can

keep all three quality dials at their optimum, and the outlier budget in particular cannot escape a logarithmic blow-up.

Problem. In *clustering with set outliers*, the input is a set P of n points, a family $\mathcal{H}$ of m subsets of P whose union covers P , and integer budgets $k, z > 0$. A solution picks at most k centers and at most z outlier sets from $\mathcal{H}$; all points covered by the chosen sets are discarded, and the goal is to minimize the clustering cost (here, the k -center objective) of the remaining points. An (α, β, γ) -approximation inflates the cost by at most α and the two budgets by at most β and γ . The relational variant takes P to be the answer set of an acyclic join query accessed through counting and sampling oracles.

3

³ The Unique Games Conjecture (UGC) asserts that a certain two-prover game is hard to distinguish with high probability; taken with it, Set Cover admits no polynomial-time $(1 - \varepsilon) \ln n$ -approximation for any $\varepsilon > 0$. It is the standard conditional source for covering-type inapproximability.

Guarantee.

Theorem 1.12 — Inapproximability under UGC, Theorem of the paper. If the Unique Games Conjecture is true, there exists no $(1, f - \varepsilon, \gamma)$ -approximation algorithm that runs in polynomial time for the problem, where $f = o(\log n)$, $\gamma = o(|\mathcal{H}|)$ is any sublinear value, and ε any positive real number. ■

How the reduction is applied. The source is Set Cover, on the ratio rung: the outlier-set budget is the coordinate that carries the Set-Cover ratio, because inflating the budget by a factor f is exactly allowing $f \cdot z^*$ sets to cover the elements. The transfer is clean enough to state in full:

Lemma 1.13 — Set-Cover transfer, Lemma of the paper. If Alg is a polynomial-time $(1, f - \varepsilon, \gamma)$ -approximation algorithm for the set-outlier clustering problem, with $\gamma = o(|\mathcal{H}|)$ sublinear, then there exists a polynomial-time $(f - \varepsilon)$ -approximation algorithm for Set Cover. ■

Proof sketch. The gadget is a line-point encoding: element x_i becomes the point p_i at coordinate i of the real line; each set y becomes the outlier set $\{p_i : x_i \in y\}$; padding points, each with its own singleton outlier set, complete the family. With no centers, the cost is zero exactly when the chosen outlier sets cover every element-point, so a clustering solution within the inflated budgets is a set cover of the corresponding size. (Full proof in Section 1.6.) ■

The UGC-conditional $(1 - \varepsilon) \ln n$ threshold for Set Cover then crosses over: no $f = o(\log n)$ outlier-budget factor, even with sublinear center-budget slack. The same paper runs the machinery constructively — the run-backwards half of the recipe: an LP feasibility relaxation with greedy rounding and a weight-charging argument delivers $(2 + \varepsilon, 2f, 2 +$

ϵ)-approximations, coresets shrink the instance, and the relational variant reuses the LP through join sampling and counting oracles, with high-probability guarantees.

What it buys. The lower bound tells the system *which budget must pay*: the paper’s algorithms inflate the outlier budget by a logarithmic factor, and Theorem 1.12 certifies that (conditioned on UGC) essentially nothing better is possible — so the practitioner can treat $\Theta(\log n)$ outlier slack as the price of admission rather than an artifact of the analysis, and the join-based variants inherit the same frontier.

1.3 Both directions at once: volume-based answer diversity

The exemplar is a SIGMOD’26 paper on diversifying query answers [1]. When a query returns millions of tuples, a system shows the user a few *diverse* answers rather than the top ones; choosing them means maximizing a diversity function over answer sets. The familiar diversity measures are distance-based — pairs of answers compared under Hamming or Euclidean distance — but these functions fail to be submodular, so the greedy heuristics systems actually run come without guarantees, and the best provable ratios depend on the metric. The paper proposes *volume-based* diversity: each answer maps to a measurable set (its “volume”), and the diversity of an answer set is the measure of its union — a direct generalization of the classic multi-attribute notion. The payoff is a single theorem pair: diversity is always monotone and submodular, so one greedy routine delivers the optimal $(1 - 1/e)$ approximation for *every* polynomially computable volume assignment — and a matching Max-Coverage reduction shows no better ratio is possible.

Problem. Fix a query Q over a database D with answer set $Q(D)$. A *volume assignment* attaches to every answer s a measurable set $\beta(s)$ in a measure space $(\mathcal{U}, \mu)$:

Definition 1.14 (Volume-based diversity, Definition of the paper). Given a volume assignment $\Omega = (\mathcal{U}, \mu, \beta)$, the diversity of a finite answer set S is

$$\delta_\Omega(S) = \mu\left(\bigcup_{s \in S} \beta(s)\right). \quad \blacksquare$$

DiverseSet asks for $S \subseteq Q(D)$ with $|S| = k$ maximizing $\delta_\Omega(S)$; *DiverseTopk* asks for ranked enumeration.

Guarantee.

Proposition 1.15 — Diversity is submodular, Proposition of the paper. Let Ω be any volume assignment over a universe $\mathcal{U}$ of possible solutions. Then δ_Ω is monotone and submodular: for $S_1 \subseteq S_2$ and any answer a ,

$$\delta_\Omega(S_1 \cup \{a\}) - \delta_\Omega(S_1) \geq \delta_\Omega(S_2 \cup \{a\}) - \delta_\Omega(S_2). \quad \blacksquare$$

Corollary 1.16 — One ratio for all volumes, results of the paper. For every volume assignment Ω computable in polynomial time and every $k \geq 1$, the greedy algorithm computes S with $\delta_\Omega(S) \geq (1 - \frac{1}{e}) \cdot \max_{|S'|=k} \delta_\Omega(S')$ in polynomial time. No polynomial-time algorithm achieves ratio $(1 - \frac{1}{e} + \epsilon)$ for all volume assignments unless $P = NP$. $\blacksquare$

How the reduction is applied. This exemplar runs the tool in *both directions*. Backwards, it reduces DiverseSet to monotone submodular maximization: Proposition 1.15 certifies the structure, and the classical greedy analysis of Nemhauser, Wolsey, and Fisher supplies the algorithm and the $(1 - 1/e)$ ratio — one inherited theorem for the whole family of measures. Forwards, it imports hardness from Max Coverage on the ratio rung: the reduction encodes a coverage instance into a volume assignment over the atom algebra of the sets, so that each tuple’s volume is exactly what it covers and $\delta_\Omega(S)$ is the covered mass — and since Max Coverage cannot be approximated beyond $(1 - 1/e)$ unless $P = NP$, neither can volume-based diversity. A second, purely combinatorial gadget handles the exact variant: for the acyclic path query $Q(x_1, \dots, x_n) : -E(x_1, x_2), \dots, E(x_{n-1}, x_n)$, answers are walks in the input graph and the diversity of a single answer counts the distinct vertices it visits, so a walk of diversity n exists exactly when the graph has a Hamiltonian path — NP-hardness already for $k = 1$. The workhorse is the measure-algebra proof of Proposition 1.15: the marginal gain of adding a is $\mu(\beta(a) \setminus \bigcup_{s \in S} \beta(s))$, which shrinks as S grows (full proof in Section 1.6).

4

⁴ For acyclic CQs the greedy step itself reduces further: finding the answer that maximizes marginal diversity is cast as a sum-product query over the tropical semiring, solvable in $O(|Q| \cdot |D|)$ time — so k greedy steps cost $O(k \cdot |Q| \cdot |D|)$, with ranked-enumeration preprocessing for DiverseTopk.

What it buys. The system keeps *one* diversification operator — greedy marginal-diversity — with a single certified ratio $(1 - 1/e)$ for every volume measure a user may define, instead of per-metric heuristics with per-metric guarantees; the matched lower bound certifies the ratio is optimal, and the tropical-semiring evaluation makes each step run in linear time over acyclic joins.

1.4 Counting reductions: Shapley values for aggregate queries

The exemplar is a SIGMOD’26 paper on explaining query answers [11]. When an aggregate answer surprises a user — why is this average so high? — the natural question is *which input facts drove it*. The Shapley

value, imported from cooperative game theory, is the canonical fair answer: it averages a fact's marginal contribution over all orders in which the endogenous facts could have arrived. Prior work pinned the complexity of Shapley values for Boolean conjunctive queries; this paper extends the analysis to *aggregate* queries — sum, max, min, and quantiles, with value functions applied at head positions — and draws the full frontier: computation is polynomial for hierarchical queries and #P-complete just past the frontier, with #P-completeness already for concrete small queries. Along the way it derives closed-form Shapley formulas for averages and quantiles, and uses the axioms of the Shapley value itself as an algorithmic tool on the tractable side.

Problem. A database D splits into an exogenous part D_{ex} (fixed) and an endogenous part D_{endo} (the facts that may be added or removed). For an aggregate query A the attribution question is:

Definition 1.17 (Shapley value of a fact, Definition of the paper). For $f \in D_{\text{endo}}$, let $\sigma_{<f}$ be the endogenous facts preceding f in a uniformly random permutation σ . The Shapley value of f is

$$\varphi(f, A) = \frac{1}{|D_{\text{endo}}|!} \sum_{\sigma \in \Sigma(D_{\text{endo}})} \left[A(D_{\text{ex}} \cup \sigma_{<f} \cup \{f\}) - A(D_{\text{ex}} \cup \sigma_{<f}) \right],$$

the expected amount by which inserting f changes the aggregate. ■

The aggregate queries are $A_\alpha = \alpha \circ Q$ for a self-join-free conjunctive query Q and $\alpha \in \{\text{sum}, \text{max}, \text{min}\}$ or a fixed quantile, composed with a *localized* value function applied to the values at head positions of Q .

5

Guarantee.

Theorem 1.18 — Tractability frontier, Theorems of the paper, notation cleaned. Let Q be a CQ without self-joins, $A_\alpha = \alpha \circ Q$ with $\alpha \in \{\text{sum}, \text{max}, \text{min}\}$ or a fixed quantile. If Q is hierarchical, then $\varphi(f, A_\alpha)$ is computable in polynomial time, given D and a fact f , for every localized value function. Otherwise, there is a localized value function such that computing $\varphi(f, A_\alpha)$, given D and a fact f , is #P-complete. ■

How the reduction is applied. The currency is a *count*, so the rung is Definition 1.7 — the number of set covers, a #P-complete quantity, must be recoverable from Shapley values. The source is #Set-Cover:

Lemma 1.19 — #Set-Cover transfer, Lemma of the paper, notation cleaned. There is a polynomial-time algorithm that, given a #Set-Cover instance $(X, \mathcal{Y})$ with $|X| = n$, $|\mathcal{Y}| = m$, constructs databases $D_{q,r}$, $q \in [n]$, $r \in [m]$, each

⁵ A query is *hierarchical* if whenever two variables share an atom, the variables of one are contained in the other's atom set — equivalently, the connected-variable sets form a laminar family. It is the classical frontier between easy and hard conjunctive-query evaluation, and this paper shows the same line governs explanation.

with a distinguished fact f , such that the numbers

$$Z_{i,j} = \#\{ \mathcal{Z} \subseteq \mathcal{Y} : |\mathcal{Z}| = j, \bigcup_{Z \in \mathcal{Z}} Z \text{ covers exactly } i \text{ elements of } X \}$$

— in particular the number of set covers, $\sum_j Z_{n,j}$ — are computable in polynomial time from the Shapley values $\varphi(f, A)$ over the $D_{q,r}$. Hence computing Shapley values is #P-hard. ■

Proof sketch. Each database is built so that a coalition’s aggregate encodes how much of X the corresponding subfamily covers: chosen sets become endogenous facts, sentinel facts weight each covered element, and the fact f flips the answer exactly when the subfamily covers everything — so $\varphi(f, A)$ is a known linear combination of the $Z_{i,j}$, and the family $D_{q,r}$ provides enough independent combinations to solve for all of them. Theorem 1.18’s hardness side follows, together with a sibling reduction from the permanent (counting exact covers) for average-type aggregates. ■

On the tractable side the reduction runs backwards, through the axioms of the Shapley value itself: linearity decomposes an aggregate into indicator queries, symmetry equalizes the values of interchangeable facts, and null players — facts that never flip the answer — contribute zero; for hierarchical Q these identities collapse the sum of Definition 1.17 into a polynomial-time dynamic program over the query’s variable tree.

What it buys. An explanation engine learns exactly when fact-ranking is cheap: within the hierarchical fragment, responsibility scores come at evaluation cost, and one step outside it they are #P-complete even for two-atom queries — so a system should either restrict explanations to the fragment or budget for approximation, and the classical query-tractability map doubles as the explanation-tractability map.

1.5 Advanced usage and further reading

Three refinements round out the toolbox: quantitative hypotheses when the target claim is a refined running time, chained reductions when one hop is not enough, and transfer lemmas when the reduction’s purpose is to build an algorithm. This section states the pattern, sketches two harder exemplars, and points to the literature.

1.5.1 Fine-grained and conditional reductions

When the claim is not “no polynomial algorithm” but “no $O(N^{2-\varepsilon})$ / quasilinear / constant-delay algorithm”, the source must be quantitative: SETH, the hyperclique hypothesis, combinatorial Boolean matrix

multiplication, or listing lower bounds. The machinery is Proposition 1.6 with the parameter audit as the proof of correctness. The corpus uses it for query-evaluation dichotomies: direct access to conjunctive query answers in aggregate-driven orders admits quasilinear preprocessing exactly when the query has no chordless path between order variables — the hard side encodes hyperclique detection under the Hyperclique hypothesis and SETH [3]^{PODS}. A second shape runs the reduction against a *randomized* solver: for uniform sampling from join-project queries, a sampler restarted $O(\text{OUT} \log \text{OUT})$ times would list all answers — the coupon-collector argument turned backwards — so the assumed listing lower bound rules out fast samplers, and combinatorial BMM instantiates the hypothesis for matrix queries [7]^{PODS}.

1.5.2 Two advanced exemplars, sketched

Both exemplars of this section push past a single many-one hop; each is sketched in one paragraph, and the reduction chain of the first is transcribed in the Appendix (Section 1.6).

Reduction chains: pseudodeterministic streaming [5]. A streaming algorithm is *pseudodeterministic* if on every input it outputs a *canonical* value with probability at least $2/3$ — the reproducibility a debugger or an audit needs — and *representation oblivious* if its output distribution depends only on the multiset of stream elements, not their order. The paper proves that any one-pass representation-oblivious pseudodeterministic (ROPD) algorithm (ϵ, δ) -approximating F_0 (the number of distinct elements) requires $\Omega(n)$ space — against the $O(\text{polylog } n)$ classics — and $\Omega(n/t(n))$ for $t(n)$ passes, matching a deterministic algorithm. The argument is a *chain* on the communication rung (Proposition 1.9): an ROPD F_0 estimator becomes an ROPD protocol for Promise-Union (do Alice’s and Bob’s sets union to more than $3n/4$ of the universe?), which becomes a pseudodeterministic protocol for Disjointness, contradicting the classical $\Omega(m)$ communication bound; obliviousness is what carries equivalence classes of streams through each hop, and pseudodeterminism is what makes the final contradiction deterministic. The constructive side checks that the classical sketches (BJKST, AMS, linear sketches) *are* representation oblivious — the restriction bites exactly where canonical outputs are impossible.

Transfer lemmas: one algorithm for many query problems [9]. Counting satisfying subsets, evaluating tuple-independent probabilistic databases, and maximizing a repaired query’s multiplicity under bag-set semantics look like three problems, but over hierarchical queries they are

one: the paper annotates facts in a *two-monoid* algebra of decomposable provenance trees and runs a single variable-elimination algorithm (eliminate private variables, merge duplicate atoms). Correctness is itself a reduction statement — a transfer lemma: any map φ from provenance trees to another two-monoid that commutes with the two operations on decomposable trees carries the algorithm’s output to the correct value for the target problem, so each application (probability, counting, repair) is a homomorphism instantiation rather than a new algorithm, and the whole runs with a linear number of monoid operations. The frontier is closed with a many-one reduction in the other direction: for every non-hierarchical query, the repair problem is NP-complete via a biclique gadget — a full dichotomy in the reachbox’s sense.

1.5.3 Further reading

- Garey and Johnson [6] — the NP-completeness handbook; the gadget-construction craft of Sections 1.1 and 1.2 is their subject in the small.
- Karp’s twenty-one problems [8] — the original demonstration that one reduction technique spans seemingly unrelated problems.
- Papadimitriou [10] — computational complexity including the counting classes behind Section 1.4.
- Vazirani [13] — L -reductions and the approximation-preserving technology of Section 1.2.
- Vassilevska Williams and Williams [14] — the fine-grained equivalences program underlying Section 1.5.1.
- Chandra and Merlin [4] — conjunctive queries and homomorphisms, where database reduction arguments begin (Example 1.3).

Chapter Recap

- **What the tool is.** A reduction is a two-way conduit: a polynomial gadget carries instances of a hard source problem into the target, and a decoder carries quality backward — so any solver for the target solves the source, importing the source’s hardness at exactly its resolution.
- **Which rung when.** Ruling out algorithms: many-one (Definition 1.1). Ruling out approximations: L -gap reductions against a covering source (Definition 1.4). Ruling out counters: #P Turing reductions (Definition 1.7). Refined time bounds: fine-grained transfer under SETH/BMM/hyperclique hypotheses (Proposition 1.6). Stream space: one-pass-to-communication

conversion (Proposition 1.9).

- **Why the audit matters.** The gadget’s parameter blow-up must be checked in every regime — the privacy gadget of Section 1.1 chose its dimension $d = \min\{t^-(T), v^-(n)\}$ exactly so that both budgets bind, turning gadget sizes directly into the exponents of Theorem 1.10.
- **What the corpus showed.** Continual-release privacy (Section 1.1): InnerProduct gadgets import error bounds into DP stream maintenance. Set-outlier clustering (Section 1.2): a line-point encoding transfers Set Cover’s UGC threshold to the outlier budget. Answer diversity (Section 1.3): one problem reduced to submodular maximization (algorithm) and from Max Coverage (hardness) — both directions at once. Shapley attribution (Section 1.4): #Set-Cover counting reductions draw a dichotomy at the hierarchical frontier.
- **Advanced usage.** Fine-grained dichotomies under SETH/hyperclique/BMM hypotheses and coupon-collector reversals against randomized solvers (Section 1.5.1); reduction chains for pseudodeterministic streaming and algebraic transfer lemmas that unify algorithms (Section 1.5.2). Deferred proofs live in the Appendix (Section 1.6).

1.6 Appendix

Proofs omitted from the main text are collected here.

Proof of Lemma 1.11 (gadget transfer, Section 1.1)

Statement (recalled). Any (ϵ, δ) -DP event-level mechanism maintaining exactly a graph problem g that admits an extendable bitwise-AND gadget with $v(d)$ vertices and $t(d)$ edges, on n -node graphs over T timesteps, has additive error $\alpha = \tilde{\Omega}(\min\{t^-(T), v^-(n)\})$, where $f^-(x) = \sup\{d \mid f(d) \leq x\}$. ■

Proof. Let $\mathcal{A}$ be such a mechanism with additive error α , and let $(x, q^1, \dots, q^m)$ be an INNERPRODUCT instance of dimension d , with d chosen as $d = \min\{t^-(T), v^-(n)\}$, so the gadget’s $v(d)$ vertices and $t(d)$ edges fit the budgets n and T . Emulate an (ϵ, δ) -DP mechanism for INNERPRODUCT: initialize the stream with the gadget’s H_{init} ; insert the edge e_i iff the data bit $x_i = 1$, giving the graph H_x^1 ; then, for each query q^j , (i) query $\mathcal{A}$, obtaining an estimate $\hat{e}^j$ of $g(H_x^j)$; (ii) insert the query edges S^j (those whose index corresponds to a query bit set to

one), giving Q_x^j , query $\mathcal{A}$ again, obtaining $\hat{q}^j$; (iii) output $\hat{q}^j - \hat{e}^j$ as the estimate of $\langle x, q^j \rangle$; (iv) insert the neutral edges T^j , giving H_x^{j+1} , ready for the next query. By the gadget's guarantee, the true values satisfy $\text{dec}_j(g(Q_x^j)) - \text{dec}_j(g(H_x^j)) = \langle x, q^j \rangle$, and the accuracy guarantee gives $|\hat{q}^j - g(Q_x^j)| \leq \alpha$ and $|\hat{e}^j - g(H_x^j)| \leq \alpha$ simultaneously with probability $2/3$, so every inner product is answered within 2α . Changing one bit of x changes one event of the stream (edge e_i inserted or not), so event-level DP of $\mathcal{A}$ makes the emulation (ε, δ) -DP — contradicting the packing lower bound $\tilde{\Omega}(\sqrt{d})$ for privately answering INNERPRODUCT unless $\alpha \geq \tilde{\Omega}(\sqrt{d}) = \tilde{\Omega}(\min\{t^-(T), v^-(n)\}^{1/2})$. ■

Proof of Lemma 1.13 (Set-Cover transfer, Section 1.2)

Statement (recalled). A polynomial-time $(1, f - \varepsilon, \gamma)$ -approximation for set-outlier clustering with sublinear γ yields a polynomial-time $(f - \varepsilon)$ -approximation for Set Cover. ■

Proof. Given a Set Cover instance $(X, \mathcal{Y})$, $X = \{x_1, \dots, x_{n'}\}$, build the clustering instance: for each element x_i , a point p_i at coordinate i of the real line; ℓ additional padding points $q_1, \dots, q_\ell$ at coordinates $2n' + 1, \dots, 2n' + \ell$; for each set $y \in \mathcal{Y}$, the outlier set $H_y = \{p_i : x_i \in y\}$; and singleton outlier sets $\{q_j\}$ for the padding points. Set the center budget to zero, so the objective is to cover all points by outlier sets, and the cost of any uncovered point exceeds any cover-based optimum (with the padding singletons forcing ℓ of the budget to be spent canonically, ℓ chosen polynomially bounded). *Yes to yes:* if $\mathcal{C} \subseteq \mathcal{Y}$ covers X with $|\mathcal{C}| \leq z$, then choosing the outlier sets $\{H_y\}_{y \in \mathcal{C}}$ plus the ℓ padding singletons covers every point within budget $z + \ell$, at cost zero. *Solution to cover:* conversely, any solution of cost zero (the cost factor 1 forces this) covers every element-point using some outlier sets; discarding the padding singletons, the corresponding sets y form a cover of X whose size is within the outlier-budget factor of the optimum cover size. Running the assumed $(1, f - \varepsilon, \gamma)$ -approximation on this instance (the sublinear center inflation γ is spent only on the padding construction) therefore returns a cover of size at most $(f - \varepsilon)$ times the optimum — a polynomial-time $(f - \varepsilon)$ -approximation to Set Cover, which the UGC-conditional $(1 - \varepsilon) \ln n$ threshold rules out for $f = o(\log n)$. ■

Proof of Proposition 1.15 (submodularity, Section 1.3)

Statement (recalled). For every volume assignment $\Omega = (\mathcal{U}, \mu, \beta)$, the diversity $\delta_\Omega(S) = \mu(\bigcup_{s \in S} \beta(s))$ is monotone and submodular. ■

Proof. *Monotonicity:* $S_1 \subseteq S_2$ implies $\bigcup_{s \in S_1} \beta(s) \subseteq \bigcup_{s \in S_2} \beta(s)$, and μ is monotone. *Submodularity:* let $S_1 \subseteq S_2$ and $a \notin S_2$ (if $a \in S_2$ both sides

are zero). For any S ,

$$\delta_\Omega(S \cup \{a\}) - \delta_\Omega(S) = \mu\left(\beta(a) \setminus \bigcup_{s \in S} \beta(s)\right),$$

by disjoint decomposition of the measure ($\mu(B \cup C) = \mu(B) + \mu(C \setminus B)$). Since $\bigcup_{s \in S_1} \beta(s) \subseteq \bigcup_{s \in S_2} \beta(s)$, we have $\beta(a) \setminus \bigcup_{s \in S_2} \beta(s) \subseteq \beta(a) \setminus \bigcup_{s \in S_1} \beta(s)$, and monotonicity of μ gives $\mu(\beta(a) \setminus \bigcup_{s \in S_2} \beta(s)) \leq \mu(\beta(a) \setminus \bigcup_{s \in S_1} \beta(s))$, which is the stated inequality. (This is the paper's proof; the Corollary 1.16 guarantee then follows from the classical greedy analysis of monotone submodular maximization.) ■

From estimator to protocol (Section 1.5.2)

Statement (recalled). If a one-pass ROPD streaming algorithm (ε, δ) -approximates F_0 over universe $[n]$ using s bits, then there is a pseudo-deterministic protocol of communication $O(s)$ for a Promise-Union $_n$ problem whose communication complexity is $\Omega(n)$; hence $s = \Omega(n)$ (Theorem 3.1 of the paper), and t -pass algorithms similarly need $\Omega(n/t)$ space. ■

Proof. Estimator to protocol. Alice holds $A \subseteq [n]$, Bob holds $B \subseteq [n]$. Alice converts A to its canonical stream $D(A)$ (the elements in lexicographic order), runs the estimator on it, and sends Bob the final memory state — s bits, exactly Proposition 1.9. Bob continues the execution with $D(B)$, so the algorithm processes the stream of $A \cup B$, and outputs 1 iff the returned estimate is at least $n/2$. On the promise (unions of size $\geq 3n/4$ versus $\leq n/4$), any (ε, δ) -approximation of $F_0 = |A \cup B|$ with $\varepsilon, \delta \leq 1/3$ decides correctly with probability $\geq 2/3$. *Obliviousness carries the class:* $D(A) \cdot D(B)$ is a canonical representative of the R_0 -equivalence class of streams over $A \cup B$, so representation obliviousness makes the output distribution depend only on the union; pseudodeterminism supplies a canonical value g , and Bob's output equals the fixed predicate $h(A, B) = \mathbf{1}[g(D(A) \cdot D(B)) \geq n/2]$ with probability $2/3$ — the protocol itself is pseudodeterministic. *The final hop:* disjointness embeds into the promise gap by padding both players' inputs so that disjoint inputs land above the high threshold and intersecting ones below it, yielding a pseudodeterministic protocol for Disj $_m$ with communication $o(m)$ — contradicting the classical $\Omega(m)$ bound. The multi-pass bound ships the state back and forth once per pass: communication $O(s \cdot t)$, so $s = \Omega(n/t)$. ■

References

- [1] Marcelo Arenas, Timo Camillo Merkl, Reinhard Pichler, and Cristian Riveros. Query answering under volume-based diversity

- functions. *Proceedings of the ACM on Management of Data*, 3, 2025. DOI: 10.1145/3767717. SIGMOD 2026 (rounds 1-2).
- [2] Bardiya Aryanfard, Monika Henzinger, David Saulpic, and A. R. Sricharan. Improved lower bounds for privacy under continual release. *Proceedings of the ACM on Management of Data*, 4, 2026. DOI: 10.1145/3801903. SIGMOD 2026.
 - [3] Nofar Carmeli and Nikolaos Tziavelis. Fine-grained dichotomies for conjunctive queries with minimum or maximum. *Proceedings of the ACM on Management of Data*, 4, 2026. DOI: 10.1145/3801899. SIGMOD 2026.
 - [4] Ashok K. Chandra and Philip M. Merlin. Optimal implementation of conjunctive queries in relational data bases. In *Proceedings of the 9th ACM Symposium on Theory of Computing (STOC)*, pages 77–90, 1977.
 - [5] Michael Chen, Sourav Chakraborty, A. Pavan, and N. V. Vinodchandran. Representation obliviousness and pseudodeterminism in streaming algorithms. *Proceedings of the ACM on Management of Data*, 3, 2025. DOI: 10.1145/3767718. SIGMOD 2026 (rounds 1-2).
 - [6] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
 - [7] Xiao Hu and Jinchao Huang. Towards output-optimal uniform sampling and approximate counting for join-project queries. *Proceedings of the ACM on Management of Data*, 4, 2026. DOI: 10.1145/3801916. SIGMOD 2026.
 - [8] Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972.
 - [9] Mahmoud Abo Khamis, Jesse Comer, Phokion G. Kolaitis, Sudeepa Roy, and Val Tannen. A unifying algorithm for hierarchical queries. *Proceedings of the ACM on Management of Data*, 3, 2025. DOI: 10.1145/3767710. SIGMOD 2026 (rounds 1-2).
 - [10] Christos H. Papadimitriou. *Computational Complexity*. Addison-Wesley, 1994.
 - [11] Christoph Standke and Benny Kimelfeld. Tractability frontiers of the shapley value for aggregate conjunctive queries. *Proceedings of the ACM on Management of Data*, 3, 2025. DOI: 10.1145/3767720. SIGMOD 2026 (rounds 1-2).

- [12] Vaishali Surianarayanan, Neeraj Kumar, and Stavros Sintos. Clustering with set outliers and applications in relational clustering. *Proceedings of the ACM on Management of Data*, 3, 2025. DOI: 10.1145/3767712. SIGMOD 2026 (rounds 1-2).
- [13] Vijay V. Vazirani. *Approximation Algorithms*. Springer, 2001.
- [14] Virginia Vassilevska Williams and Ryan Williams. Subcubic equivalences between path, matrix, and triangle problems. In *Proceedings of the 51st Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 645–654, 2010.

2

Concentration Inequalities

Concentration inequalities bound the probability that a random quantity strays far from its expectation. For a systems paper they upgrade a randomized component that is correct *in expectation* — a sampler, sketch, random walk, or noise mechanism — into one correct *with high probability (whp)*: a quantified contract such as “ $(1 \pm \varepsilon)$ -accurate with probability $1 - 1/\text{poly}(n)$ ” or “running time $O(T)$ with probability $1 - 1/N$.” The recipe is constant: name the bad event, bound it by a tail inequality, union-bound over all copies of the component. In the SIGMOD’26 corpus this is among the most-used theoretical tools (38 tagged papers), directly supplying the probabilistic-guarantee property in 25 of them.

This chapter presents the basic ladder and the standard exemplars first; Section 2.4 adds the advanced rungs (martingales, two harder exemplars) as extension reading.

2.0 The tool: concentration inequalities

The problem class is uniform across applications: we can compute (or bound) the expectation of some random quantity X — the number of sampled points falling in a region, the error of a private estimate, the load of a hash bucket — but an algorithm must succeed on the *actual* draw, not the average. What concentration delivers is a bound of the form

$$\Pr[|X - \mathbb{E}X| \geq t] \leq \text{some decreasing function of } t,$$

plus the craft of making that function decrease fast enough. How fast is “fast enough” is the organizing question of this chapter: a bound of δ per bad event survives a union bound over $1/\delta$ events, so a component replicated $\text{poly}(n)$ times needs per-event failure $1/\text{poly}(n)$, which in turn needs tails that decay *exponentially* in the deviation.

LEARNING OBJECTIVES

- Recognize when a component needs a *whp* contract, and formalize the bad event.
- Prove the ladder Markov $\rightarrow$ Chebyshev $\rightarrow$ Chernoff/Hoeffding, and match each rung to the dependence structure.
- Tune per-event failure to $\delta/(\text{events})$; union-bound over all copies.
- Dissect a paper’s argument: random object, independence, rung, δ budget.

Notation

“whp” (with high probability) always means probability at least $1 - 1/\text{poly}(n)$, or $1 - 1/N$ for a system of N components; the polynomial in the failure probability is set by the union bound over the whole system.

2.0.1 The basic ladder

Every bound in the family is Markov’s inequality in disguise [2, 4, 13].

Theorem 2.1 — Markov. Let $X \geq 0$ be a random variable with $\mathbb{E}X < \infty$. Then for every $a > 0$,

$$\Pr[X \geq a] \leq \frac{\mathbb{E}X}{a}. \quad \blacksquare$$

Proof. $\mathbb{E}X \geq \mathbb{E}[X \cdot \mathbf{1}\{X \geq a\}] \geq a \Pr[X \geq a]$; divide by a . $\blacksquare$

One moment, one side, polynomial tail. Applying it to $(X - \mu)^2$ instead of X gives the two-sided, second-moment rung:

Corollary 2.2 — Chebyshev. Let $\mu = \mathbb{E}X$ and $\text{Var}(X) < \infty$. Then for every $t > 0$,

$$\Pr[|X - \mu| \geq t] \leq \frac{\text{Var}(X)}{t^2}. \quad \blacksquare$$

Proof. Markov (Theorem 2.1) applied to the nonnegative variable $(X - \mu)^2$ with $a = t^2$. $\blacksquare$

Polynomial tails suffice for constant success probability — boosted to high probability by repetition and medians — but they cannot survive a union bound over $\text{poly}(n)$ events. Exponential tails come from applying Markov to $e^{\lambda X}$:

Lemma 2.3 — Chernoff method. For any random variable X , any $a \in \mathbb{R}$ and any $\lambda > 0$,

$$\Pr[X \geq a] = \Pr[e^{\lambda X} \geq e^{\lambda a}] \leq e^{-\lambda a} \mathbb{E}[e^{\lambda X}],$$

and one optimizes over $\lambda > 0$ (the *moment generating function* controls the tail). $\blacksquare$

Proof. Markov (Theorem 2.1) applied to the nonnegative variable $e^{\lambda X}$ at level $a' = e^{\lambda a}$; the inequality is monotone in λ ’s sign, so only $\lambda > 0$ is useful. $\blacksquare$

Evaluating the moment generating function of a sum of independent bounded variables yields the two workhorses of randomized systems. The multiplicative form is the one to memorize for sampling and sketching [2, 3, 13]:

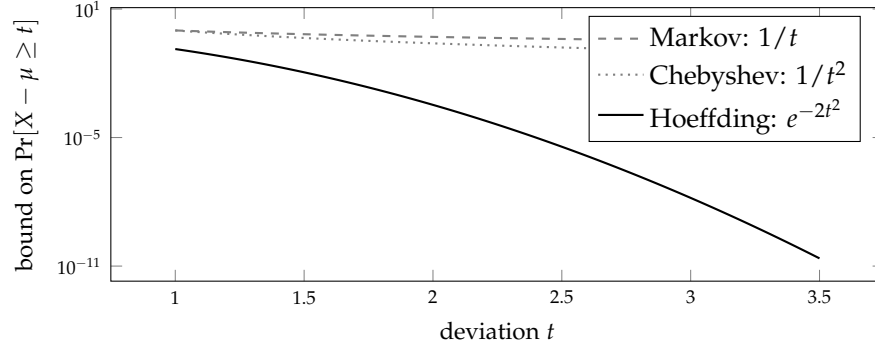


Figure 2.1: Tail decay for a unit-scale random quantity: moment bounds fall polynomially, independence buys the exponential curve — the gap that makes $\text{poly}(n)$ -way union bounds affordable.

Theorem 2.4 — Chernoff, multiplicative form. Let $X_1, \dots, X_m$ be independent random variables with $X_i \in [0, 1]$, $X = \sum_{i=1}^m X_i$, $\mu = \mathbb{E}X$. Then for every $\varepsilon \in (0, 1)$,

$$\Pr[X \geq (1 + \varepsilon)\mu] \leq e^{-\varepsilon^2\mu/3}, \quad \Pr[X \leq (1 - \varepsilon)\mu] \leq e^{-\varepsilon^2\mu/2}. \quad \blacksquare$$

Proof idea. Chernoff method (Lemma 2.3) on X , then the elementary bound $(1 + \varepsilon)\ln(1 + \varepsilon) - \varepsilon \geq \varepsilon^2/3$ (and its lower-tail twin) on the optimized exponent; the full derivation is in the Appendix (Section 2.5). $\blacksquare$

The additive form is interchangeable whenever the summands are bounded but not naturally counts [6, 11, 13]:

Theorem 2.5 — Hoeffding, additive form. Let $X_1, \dots, X_m$ be independent with $X_i \in [a_i, b_i]$ a.s. and $X = \sum_i X_i$. Then for every $t > 0$,

$$\begin{aligned} \Pr[X - \mathbb{E}X \geq t] &\leq \exp\left(-\frac{2t^2}{\sum_i (b_i - a_i)^2}\right), \\ \Pr[X - \mathbb{E}X \leq -t] &\leq \exp\left(-\frac{2t^2}{\sum_i (b_i - a_i)^2}\right). \end{aligned} \quad \blacksquare$$

Proof idea. Chernoff method plus *Hoeffding's lemma*, $\mathbb{E}[e^{\lambda(X_i - \mathbb{E}X_i)}] \leq e^{\lambda^2(b_i - a_i)^2/8}$; optimize λ (proof in Section 2.5). $\blacksquare$

1

When independence fails but each random choice shifts the running statistic by a bounded amount, the ladder has two more rungs — Azuma–Hoeffding and McDiarmid — built on martingales; they are stated, with usage guidance, in Section 2.4.

2.0.2 How to choose a rung

The selection procedure is mechanical once two questions are answered. *First, what depends on what?* If the random object is a sum of independent

¹ The entry fee for both forms is *independence plus bounded summands*. The tunable currency is the exponent $-\varepsilon^2\mu$ or $2t^2 / \sum_i (b_i - a_i)^2$ — which must be pushed past $\log(\#\text{events}/\delta)$ for the failure probability to amortize. When the summands are unbounded but light-tailed (Gaussian, Laplace, sub-exponential, sub-gamma), the same Chernoff-method machinery applies with a moment-generating function that exists near 0; corpus papers use sub-gamma tails for approximate counters (Morris-type) and Laplace tails for privacy noise, and we will see the latter in Section 2.4.2.

bounded indicators — samples retained, points in a region, hashed keys — use multiplicative Chernoff when the bound should be relative to the expectation μ (sampling rates, subsampling), and additive Hoeffding when it should be relative to the range parameters $b_i - a_i$ (weighted sums, real-valued measurements). *Second, what moments are available?* If only the mean (or an exponent-optimized mean of a transform) is computable, Markov is the floor — and, applied to $e^{\lambda X}$, it is secretly the ceiling too.

When to Reach for Concentration

Reach whenever a randomized component must be certified — sampling/sketching estimators, random walks, sampled data structures, hashing, privacy noise — and the deliverable is a probability-qualified contract: accuracy $(1 \pm \varepsilon)$ whp, or time/space bounds holding with probability $1 - 1/N$. Then run the recipe:

1. Name the random object X and the bad event $\{|X - \mathbb{E}X| \geq t\}$.
2. Check the dependence structure: independent bounded summands $\rightarrow$ Chernoff/Hoeffding; adaptive exposure or capped per-variable influence $\rightarrow$ the martingale rungs of Section 2.4; only moments available $\rightarrow$ Markov/Chebyshev — constant probability, boostable by repetition with medians (and Markov on $e^{\lambda X}$ is the engine of every exponential-tail argument anyway).
3. Tune parameters so each bad event fails with probability at most $\delta/(\text{events})$ — i.e. push the exponent past $\log(\text{events}/\delta)$; only exponential tails amortize over $\text{poly}(n)$ copies, polynomial ones do not.
4. Union-bound over all copies of the component.

2.1 Trimmed statistics in a turnstile stream

The exemplar is a PODS'26 paper on robust streaming statistics [8]^{PODS}. Frequency moments, $F_p = \sum_i |x_i|^p$, are the workhorses of database analytics: query optimization, data mining, and network monitoring all rely on them, and the classical streaming results estimate them over the *whole* frequency vector. Many analytics tasks, however, want order-restricted quantities instead: statistics built from only the heaviest (or the central) part of the data, with the extremes treated as outliers — the streaming analogue of the trimmed mean from robust statistics. The paper studies such *trimmed statistics* — notably the F_p moment of the top- k coordinates, a Ky-Fan-type norm behind impact measures like the h -index — in the turnstile streaming model, and shows that for every

$p \in [0, 2]$ they can be estimated in a single pass using $\text{poly}(\log n / \varepsilon)$ bits of space.

Problem. Work in the *strict turnstile* streaming model: an n -dimensional frequency vector $\mathbf{x} \in \mathbb{Z}^n$ starts at zero and receives a long stream of updates of the form “add Δ to coordinate i ” (with Δ of either sign); the algorithm sees updates one at a time and must answer queries about the final $\mathbf{x}$ using space far less than n . Let $a_1, \dots, a_k$ be the k coordinates of $\mathbf{x}$ of largest absolute value. The *trimmed statistic* of order (p, k) is $\sum_{i=1}^k |a_i|^p$ for $0 \leq p \leq 2$: the F_p mass contributed by the top k coordinates — the part of the data that survives trimming away the tail. Such statistics are the streaming analogue of trimmed estimators in robust statistics.

Guarantee.

Theorem 2.6 — Top- k F_p estimation, Theorem 4 of the paper. Let $\mathbf{x} \in \mathbb{Z}^n$, $p \in [0, 2]$, $k \geq 0$, $\varepsilon \in (0, 1)$. Suppose $\|\mathbf{x}_{-k}\|_2^2 \geq (\varepsilon / \log n)^c \cdot k$ for a constant c , where $\mathbf{x}_{-k}$ excludes the top- k coordinates. Then there exists a *linear sketch* using $\text{poly}(\log n / \varepsilon)$ bits of space that estimates $\sum_{i=1}^k |a_i|^p$ to a $(1 \pm \varepsilon)$ factor with high constant probability. ■

2

How concentration is applied. The algorithm partitions coordinates by magnitude into *geometric level sets* whose boundaries are shifted by a random offset ζ uniform in $[1/2, 1]$, then *dyadically subsamples* each level: each coordinate of level S_j is kept independently with probability $r = \min\{1, C \log n / (\varepsilon^2 s_j)\}$ where $s_j = |S_j|$ and C is a constant. Survivors are found with a CountSketch³ fed with $\Theta(\log n)$ buckets per subsampling rate. The concentration step is the following lemma; its random objects are the independent retention indicators X_i of the coordinates of S_j , each Bernoulli(r), so the survivor count has mean $s_j r$.

Lemma 2.7 — Level-size estimation, Lemma 4.4 of the paper, notation cleaned. Let S_j be a level set with $s_j = |S_j|$, and subsample each coordinate independently with probability $r = \min\{1, C \log n / (\varepsilon^2 s_j)\}$. If z coordinates of S_j survive and $\tilde{s}_j = z/r$, then with probability $1 - 1/\text{poly}(n)$,

$$(1 - \varepsilon) s_j \leq \tilde{s}_j \leq (1 + \varepsilon) s_j. \quad \blacksquare$$

Proof sketch. Write $u_1, \dots, u_{s_j}$ for the coordinates of S_j , and let X_i be the indicator that u_i is sampled, so $\mathbb{E} \sum_i X_i = s_j r$. Multiplicative Chernoff gives $\Pr[|\sum_i X_i - s_j r| \geq \varepsilon s_j r] \leq 2 \exp(-\varepsilon^2 s_j r / 3) \leq 2 \exp(-C \log n / 3) \leq 1 / \text{poly}(n)$; divide by r . (Full proof in Section 2.5.) ■

² A *linear sketch* stores $A\mathbf{x}$ for a random matrix A ; it is the standard route to mergeable, one-pass, deletion-tolerant stream summaries. The condition on $\|\mathbf{x}_{-k}\|_2^2$ holds automatically once $k \geq \text{polylog } n$, so the theorem covers the practically interesting regime.

³ CountSketch (Charikar–Chen–Farach-Colton) recovers, from hashed counters, all coordinates whose value exceeds a θ -fraction of the residual ℓ_2 norm (a *heavy hitter* structure); the paper needs only this black box.

The δ tuning is explicit: r is chosen exactly so that the Chernoff exponent $\epsilon^2 s_j r$ equals $C \log n$, i.e. per-level failure $1/\text{poly}(n)$; there are only $O(\log_{1+\epsilon} m)$ levels (magnitude scales), so a union bound over levels preserves the polynomial failure probability. A companion lemma (Lemma 4.5 of the paper) uses the same Chernoff bound on the binomial survivor count to show that fewer than $O(\log n / \epsilon^2)$ coordinates survive per level whp — which is what makes the heavy-hitter structure, and hence the space bound, $\text{poly}(\log n / \epsilon)$.

What it buys. The system gets a one-pass, sublinear-space summary of trimmed statistics — robust aggregate mass that ignores the tail — with a certificate strong enough to union-bound into larger algorithms; the randomness of the offset ζ (rather than an adversarial grid) is what makes the level-set decomposition itself immune to worst-case input placement.

2.2 Maximum range sum by randomized grid shifting

The exemplar is a SIGMOD’26 paper on spatial query processing [5]. The *maximum range sum* (MaxRS) problem sits behind every “find the best location” query: given weighted points — customers, incidents, sensors — and a fixed-size geometric region Q , place Q anywhere in space so that it captures as much total weight as possible; that placement is where to site a store, a base station, or an emergency facility. The problem is well studied by both the database and the computational-geometry communities, and the paper revisits it in three practically motivated variations: *dynamic* MaxRS, where points are inserted and deleted and a good placement must be maintained under updates; *batched* MaxRS, where many query regions must be answered at once; and *colored* variants that group the points into classes. Its headline results are a randomized $(\frac{1}{2} - \epsilon)$ -approximation for dynamic MaxRS with polylogarithmic update time — the first algorithm of this kind — together with conditional lower bounds showing that the batched problem cannot be solved substantially faster than the trivial approach.

Problem. In the *maximum range sum* (MaxRS) problem, the input is a set P of n points in $\mathbb{R}^d$, each carrying a weight, and a query shape Q — here a d -ball of given radius. The goal is to place Q (translate it anywhere) so as to maximize the total weight of the points of P covered.

Exact MaxRS is expensive: for Q a 2-ball the colored version carries a conditional $\Omega(n^2)$ lower bound, and assuming that $(\min, +)$ -convolution on length- n sequences requires $\Omega(n^2)$ time, the batched variant on n points and m query radii requires $\Omega(mn)$ time — almost matching the trivial $O(mn \log n)$ upper bound.

Guarantee.

Theorem 2.8 — Static MaxRS, one representative case of the paper. For static MaxRS with Q a d -ball and any $0 < \varepsilon < 1/2$, there is a randomized $(\frac{1}{2} - \varepsilon)$ -approximation algorithm running in time $O(\varepsilon^{-2d-2} n \log n)$; the approximation factor holds with high probability, i.e. $1 - n^{-\Omega(1)}$. The dynamic version achieves the same factor with amortized update time $O(\varepsilon^{-2d-2} \log n)$. ■

How concentration is applied. The algorithm randomly shifts a grid over the point set and, per cell, takes a random sample of the cell's points; the candidate balls it examines are drawn from a finite set of $k \leq n$ possibilities determined by the shift. Two facts combine. First, a *geometric covering lemma*: if C is a d -ball of radius ρ and B a unit d -ball whose center lies at distance ρ^2 from the center of C , then the $(d-1)$ -dimensional measure of the cap $B \cap \partial C$ is at least a $(\frac{1}{2} - \Theta(\varepsilon))$ fraction of the measure of ∂C — i.e., a random point of C 's boundary lands in any fixed candidate ball with probability at least $\frac{1}{2} - \Theta(\varepsilon)$. Second, the concentration step:

Lemma 2.9 — Sampling concentration, Lemma of the paper, notation cleaned. Let $S = \{p_1, \dots, p_t\}$ be such that each point of S lies inside the candidate ball B independently with probability at least $\frac{1}{2} - \Theta(\varepsilon)$, and let $Y_B = \sum_{i=1}^t \mathbf{1}\{p_i \in B\}$. Then with high probability, simultaneously for all $k \leq n$ candidate balls,

$$Y_B \geq (1 - \varepsilon) \left(\frac{1}{2} - \Theta(\varepsilon) \right) t. \quad \blacksquare$$

Proof sketch. The $X_i = \mathbf{1}\{p_i \in B\}$ are independent indicators with $\mathbb{E}[Y_B] \geq (\frac{1}{2} - \Theta(\varepsilon))t$; multiplicative Chernoff gives $\Pr[Y_B < (1 - \varepsilon)\mathbb{E}Y_B] \leq e^{-\varepsilon^2 t/2}$, and a union bound over the $k \leq n$ balls keeps this whp provided $t = \Theta(\varepsilon^{-2} \log n)$. (Full proof in Section 2.5.) ■

The random objects are the per-point sampling/shift indicators; the structure is full independence across points (the random shift acts independently per cell); the rung is multiplicative Chernoff; and the δ tuning sets $t = \Theta(\varepsilon^{-2} \log n)$ so each of the n candidate balls fails with probability $n^{-\Omega(1)}$ — the same “push the exponent past $\log(\text{events})$ ” accounting as in Section 2.1, with the exponent coming from sample size rather than subsample rate.

What it buys. The spatial engine answers best-placement queries in near-linear time with a certified $(1/2 - \varepsilon)$ fraction of the optimum whp — and supports updates in polylog amortized time — where

⁴ *Conditional lower bound:* a reduction from $(\min, +)$ -convolution — widely conjectured to need quadratic time — transfers that hardness to MaxRS. This is the paper's evidence that *approximation* is necessary, and approximation is exactly where concentration enters.

exact computation faces (conditional) quadratic barriers. The same Chernoff-plus-union-bound skeleton, re-instantiated with trapezoidal maps, gives the paper’s $(1 - \epsilon)$ -approximation for the colored variant.

2.3 User-level private query processing with sampled tuples

The exemplar is a SIGMOD’26 paper on private query processing [12]. Answering select-join-aggregate (SJA) queries under differential privacy is a basic building block of privacy-preserving analytics: the system must return an accurate aggregate while guaranteeing that no adversary can confidently infer whether any one *user’s* data was present or absent. The state of the art achieves strong, instance-optimal accuracy at this *user level*, but pays for it by solving expensive optimization programs over the whole database — prohibitive at scale. Sampling is the classic escape hatch, buying a tunable trade-off between accuracy and computational cost, yet combining it with private processing is subtle: it is unclear *what* to sample for the best accuracy under a given compute budget, and the existing mathematical tool chains were not designed with sampling in mind. The paper builds a scalable user-level-DP SJA engine that privately truncates each heavy user’s contribution and Poisson-samples tuples to accelerate the truncation search, with an error analysis that cleanly separates sampling error from privacy noise.

Problem. Consider a select-project-join-aggregate workload over a relational instance whose rows belong to *users*, each user contributing many tuples. The privacy contract is differential privacy at the granularity of users:

Definition 2.10 (ϵ -differential privacy, user level). A randomized mechanism M satisfies ϵ -differential privacy if for all *neighboring* inputs D, D' and all measurable sets S ,

$$\Pr[M(D) \in S] \leq e^\epsilon \Pr[M(D') \in S].$$

For *user-level* DP, “neighboring” means *all* tuples of one user added or removed — much stricter than tuple-level DP.

Unbounded per-user contributions then blow up sensitivity, so the system [12] truncates each user’s contribution to a threshold τ chosen privately by a search (sparse vector technique), and accelerates the search by Poisson-sampling tuples at rate q before privatizing.

⁵ Poisson sampling also *amplifies* privacy: a mechanism that is ϵ -DP on the sample is (ϵ') -DP on the full data with $\epsilon' = \ln(1 + q(e^\epsilon - 1)) < \epsilon$; the paper’s Sample-and-Explore comparison baselines use this. The proof is a binomial tail computation — again concentration.

Guarantee.

Theorem 2.11 — Error of the sampled truncation, Algorithm 2 of the paper, compressed. The mechanism satisfies $(\varepsilon, 0)$ -DP. Let n be the number of records, m the number of users, and q the sample rate. Write $\tau^*(T) = \max_u \sum_{t \in T} \mathbf{1}[u \prec t]$ for the true maximum per-user contribution, and $\varepsilon_i^* \geq \varepsilon/L$ for the noise-calibration budget over the L search iterations. With probability $1 - \beta$, the released answer $\hat{f}(T)$ obeys

$$|\hat{f}(T) - f(T)| = \tilde{O}\left(\sqrt{\frac{n}{q}} + \frac{1}{q} + \frac{1}{\varepsilon_i^*} \left(\tau^*(T) + \sqrt{\frac{\tau^*(T)}{q}}\right)\right),$$

where $\tilde{O}(\cdot)$ hides constants and polylogarithmic factors in m and $1/\beta$. ■

How concentration is applied. The random objects are the independent sampling indicators $\mathbf{1}\{t \text{ sampled}\} \sim \text{Bernoulli}(q)$ over the n tuples. The sampled statistic $f(S) = \sum_{t \in T} w_t \mathbf{1}[t \text{ sampled}]$ satisfies $\mathbb{E}[f(S)] = q f(T)$ and $\text{Var}[f(S)] = q(1-q) \sum_t w_t^2 \leq qn$: a sum of independent bounded variables with an explicit variance proxy — the territory of the Bernstein/Chernoff rung, whose bound gives $|f(S) - qf(T)| \lesssim \sqrt{qn \log(3L/\beta)}$ with probability $1 - \beta/(3L)$; dividing by q yields the $\sqrt{n/q}$ term. The δ -budgeting is again structural: the truncation-threshold search runs L iterations, each is allocated failure probability $\beta/(3L)$, and a union bound over iterations upgrades the per-iteration bound to the global $1 - \beta$ contract — with the same per-event log factor $\log(3L/\beta)$ appearing inside the noise scale. The $\tau^*(T)/\varepsilon_i^*$ terms come from the Laplace-style calibration of the final release against the (privately learned) per-user cap.

What it buys. The query engine can offer user-level — not merely tuple-level — privacy at a modest accuracy price: the sampling error $\sqrt{n/q}$ decouples from the privacy noise, so tuning q trades compute for accuracy without touching the privacy guarantee, and the explicit $\tau^*(T)$ dependence quantifies exactly how much heavy user-contributors cost the answer.

2.4 Advanced usage and further reading

Everything so far needs independence. Two extensions belong in every practitioner’s backpack: the martingale rungs that replace independence with bounded per-step influence, and the light-tailed regime where summands are unbounded but have a moment generating function. This section states the rungs, sketches two harder exemplars, and points to the literature.

⁶ Recall: a martingale difference sequence (Y_i) with respect to a filtration $(\mathcal{F}_i)$ has $\mathbb{E}[Y_i \mid \mathcal{F}_{i-1}] = 0$; the partial sums $M_j = \sum_{i \leq j} Y_i$ then satisfy $\mathbb{E}[M_j \mid \mathcal{F}_{j-1}] = M_{j-1}$.

2.4.1 Martingale rungs: Azuma–Hoeffding and McDiarmid

Expose the randomness sequentially and book the shifts as a *martingale*⁶:

Theorem 2.12 — Azuma–Hoeffding. Let $(Y_i)_{i=1}^m$ be a martingale difference sequence with $|Y_i| \leq c_i$ a.s. Then for every $t > 0$,

$$\Pr \left[\sum_{i=1}^m Y_i \geq t \right] \leq \exp \left(- \frac{t^2}{2 \sum_{i=1}^m c_i^2} \right). \quad \blacksquare$$

Proof idea. Chernoff method on $\sum_i Y_i$, Hoeffding’s lemma applied to each Y_i *conditionally on the past* (boundedness is all that is needed; independence is replaced by iterated conditioning), then optimize λ (Section 2.5). $\blacksquare$

The function-of-independent-variables form is the rung most used for adaptive algorithms and data structures, where the analyzed quantity is not a sum but a function whose per-variable influence is capped [2, 4, 10]:

Theorem 2.13 — McDiarmid, bounded differences. Let $X_1, \dots, X_m$ be independent random variables and $f: \prod_i \mathcal{X}_i \rightarrow \mathbb{R}$ a measurable function satisfying the bounded differences condition: for each i there is $c_i \geq 0$ with

$$|f(x_1, \dots, x_i, \dots, x_m) - f(x_1, \dots, x'_i, \dots, x_m)| \leq c_i$$

for all inputs and all x'_i . Then for every $t > 0$,

$$\Pr [f(X_1, \dots, X_m) - \mathbb{E}f \geq t] \leq \exp \left(- \frac{2t^2}{\sum_{i=1}^m c_i^2} \right). \quad \blacksquare$$

Proof idea. Apply Azuma–Hoeffding (Theorem 2.12) to the *Doob martingale* $Y_i = \mathbb{E}[f \mid X_1, \dots, X_i] - \mathbb{E}[f \mid X_1, \dots, X_{i-1}]$, whose increments are bounded by c_i exactly because changing X_i moves f by at most c_i (Section 2.5). $\blacksquare$

When these rungs earn their keep. If the algorithm is *adaptive* — later random choices conditioned on earlier estimates, sampling without replacement, stopping times — the summands are dependent but each step’s effect is bounded, and Azuma–Hoeffding applies. If the quantity is a *function* of many independent inputs (a data structure’s quality, a clustering cost, a query plan’s runtime) with per-input influence at most c_i , McDiarmid converts that directly. Hashing with limited independence is the classic gap: the indicators are not fully independent, but exposure by hash key gives bounded increments.

2.4.2 Two advanced exemplars, sketched

Both exemplars of this section are privacy papers that push past the sum-of-independent-bounded-summands regime; each is sketched in

one paragraph, and the round-by-round argument of the first is transcribed in the Appendix (Section 2.5).

Private graph pattern counting under LDP [7]. Counting subgraph patterns — triangles, paths, stars — is a cornerstone of network analysis (community detection, fraud detection), and releasing such counts leaks information about individual edges. Under *local* differential privacy there is no trusted curator: each node of the graph randomizes its own messages before anything is aggregated, and existing LDP mechanisms were ad hoc, handling only specific patterns such as triangles and stars. The paper gives the first general mechanism for counting arbitrary *acyclic* patterns under LDP. It runs k rounds of message passing, adding fresh Laplace noise — *sub-exponential* tails, so the Chernoff method still applies — at every node in every round. The δ -accounting is the recipe of Section 2.0.2 in its purest form: failure budget β'/N per node per round, union bound over the kN node-rounds, then an *induction on rounds* that grows the noise scale geometrically but controllably. The resulting estimate is unbiased with error $\tilde{O}(\sqrt{N d(G)^{k-1}})$ — the square root of the maximal possible count. For simple paths, the tail is only polynomial in $1/\beta$ (a variance/Markov argument), illustrating the price of dropping exponential tails.

Product noise as a concentration bound [9]. Adding calibrated noise to a released statistic is the most fundamental route to differential privacy: the system outputs $f(x) + \mathbf{n}$, and the guarantee rests on the tail of the *privacy-loss random variable* $L_M = \ln \frac{dP}{dQ}$ — (ϵ, δ) -DP holds exactly when $\Pr[L_M \geq \epsilon] \leq \delta$, a tail bound in disguise. For Gaussian noise L_M is itself a univariate Gaussian, and this is where standard analyses stop. The paper instead exploits the spherical symmetry of typical noise distributions to generate $\mathbf{n}$ from a *product* of two independent components — a radius controlling the noise magnitude and a direction controlling its angle relative to the sensitivity vector — which re-expresses L_M as a product measure and lets the tail be optimized directly. The paper never writes the word “Chernoff,” but its argument — factorize L_M into independent terms, bound $\mathbb{E}[U^q]$ for a transform U , and *minimize the resulting δ over the exponent q* — is exactly the Chernoff method (Lemma 2.3) on a likelihood ratio. The payoff: expected noise smaller by a $\Theta(\log(1/\delta))$ factor than the Gaussian mechanism at equal privacy.

2.4.3 Further reading

- Boucheron, Lugosi, and Massart [2] — the comprehensive non-asymptotic treatment; the martingale and supremum versions live here.

- Dubhashi and Panconesi [4] — the randomized-algorithms viewpoint, with the exposure/Doob-martingale technique foregrounded.
- Mitzenmacher and Upfal [11] — textbook treatment with CS applications and all the Chernoff variants used in systems papers.
- The originals: Chernoff (1952) [3], Hoeffding (1963) [6], Azuma (1967) [1], and McDiarmid’s bounded-differences survey [10].
- For a two-page refresher with proofs of the additive forms, the Stanford CS229 notes [13].

Chapter Recap

- **What the tool is.** Concentration inequalities convert expectations into with-high-probability contracts: $\Pr[|X - \mathbb{E}X| \geq t]$ bounded by a tail function, tuned so that per-event failure $\delta/(\text{events})$ survives a union bound over all copies of the randomized component.
- **Which rung when.** Independent bounded indicators/sums: multiplicative Chernoff (relative error) or additive Hoeffding (range-based). Adaptive or dependent exposure with bounded per-step effect: Azuma–Hoeffding. Function of many independent inputs with capped per-input influence: McDiarmid. Only moments available: Markov and Chebyshev — constant-probability results, boostable by medians. Light-tailed unbounded noise (Laplace, Gaussian, sub-gamma): the Chernoff method still applies via the moment generating function.
- **Why exponential tails.** Only $e^{-\Omega(\text{exponent})}$ tails make $\text{poly}(n)$ -way union bounds affordable; polynomial tails ($1/t$, $1/t^2$) cannot amortize — visible in the corpus as the gap between $\tilde{O}(\cdot)$ guarantees (Laplace/Chernoff paths) and $1/\beta$ -type errors (variance-only paths).
- **What the corpus showed.** Trimmed-statistic sketching (Section 2.1): Chernoff on subsampling survivors, union over $O(\log_{1+\epsilon} m)$ levels. Spatial MaxRS (Section 2.2): Chernoff on sampled point-in-ball indicators, union over $\leq n$ candidate balls. User-level private queries (Section 2.3): Bernstein/Chernoff on Bernoulli-sampled sums with $\beta/(3L)$ budgets over search iterations.
- **Advanced usage.** The martingale rungs (Section 2.4.1) replace independence with bounded per-step influence; the privacy exemplars of Section 2.4.2 run the Chernoff method on Laplace

noise and on a likelihood ratio. Long proofs of everything deferred live in the Appendix (Section 2.5).

2.5 Appendix

Proofs omitted from the main text are collected here.

Proof of Theorem 2.4 (Chernoff, multiplicative form)

Statement (recalled). Let $X_1, \dots, X_m$ be independent with $X_i \in [0, 1]$, $X = \sum_i X_i$, $\mu = \mathbb{E}X$. Then $\Pr[X \geq (1 + \varepsilon)\mu] \leq e^{-\varepsilon^2\mu/3}$ and $\Pr[X \leq (1 - \varepsilon)\mu] \leq e^{-\varepsilon^2\mu/2}$. ■

Proof. By the Chernoff method (Lemma 2.3), for $\lambda > 0$, $\Pr[X \geq (1 + \varepsilon)\mu] \leq e^{-\lambda(1+\varepsilon)\mu} \prod_i \mathbb{E}[e^{\lambda X_i}]$. For $X_i \in [0, 1]$ independence gives $\mathbb{E}[e^{\lambda X_i}] = 1 + p_i(e^\lambda - 1) \leq \exp(p_i(e^\lambda - 1))$, so the product is at most $e^{\mu(e^\lambda - 1)}$ and, choosing $\lambda = \ln(1 + \varepsilon)$,

$$\Pr[X \geq (1 + \varepsilon)\mu] \leq \left(\frac{e^\varepsilon}{(1 + \varepsilon)^{1+\varepsilon}} \right)^\mu \leq e^{-\varepsilon^2\mu/3},$$

using $(1 + \varepsilon)\ln(1 + \varepsilon) - \varepsilon \geq \varepsilon^2/3$ for $\varepsilon \in (0, 1)$. The lower tail is symmetric, with exponent $\varepsilon^2/2$. ■

Proof of Theorem 2.5 (Hoeffding, additive form)

Statement (recalled). Let $X_1, \dots, X_m$ be independent with $X_i \in [a_i, b_i]$ a.s. and $X = \sum_i X_i$. Then for every $t > 0$, $\Pr[X - \mathbb{E}X \geq t] \leq \exp(-2t^2 / \sum_i (b_i - a_i)^2)$, and the same for the lower tail. ■

Proof. Hoeffding's lemma: if $Y \in [a, b]$ a.s. and $\mathbb{E}Y = 0$, then $\mathbb{E}[e^{\lambda Y}] \leq e^{\lambda^2(b-a)^2/8}$. (Proof: $e^{\lambda y}$ is convex, so $e^{\lambda Y} \leq \frac{b-Y}{b-a}e^{\lambda a} + \frac{Y-a}{b-a}e^{\lambda b}$ pointwise; take expectations and minimize over a shifted λ .) Applying the lemma to each centered summand and multiplying by independence, $\mathbb{E}[e^{\lambda(X - \mathbb{E}X)}] \leq e^{\lambda^2 \sum_i (b_i - a_i)^2/8}$; the Chernoff method then gives $\Pr[X - \mathbb{E}X \geq t] \leq e^{-\lambda t + \lambda^2 \sum_i (b_i - a_i)^2/8}$, and optimizing at $\lambda = 4t / \sum_i (b_i - a_i)^2$ yields the stated $e^{-2t^2 / \sum_i (b_i - a_i)^2}$. ■

Proof of Theorems 2.12 and 2.13 (martingale rungs)

Statement (recalled). (i) If $(Y_i)_{i=1}^m$ is a martingale difference sequence with $|Y_i| \leq c_i$ a.s., then $\Pr[\sum_i Y_i \geq t] \leq \exp(-t^2/2 \sum_i c_i^2)$. (ii) If f has bounded differences (c_i) over independent $X_1, \dots, X_m$, then $\Pr[f - \mathbb{E}f \geq t] \leq \exp(-2t^2 / \sum_i c_i^2)$. ■

Proof. Let $M_j = \sum_{i \leq j} Y_i$ with $\mathbb{E}[Y_i \mid \mathcal{F}_{i-1}] = 0$ and $|Y_i| \leq c_i$. Iterated conditioning gives $\mathbb{E}[e^{\lambda M_m}] = \mathbb{E}[e^{\lambda M_{m-1}} \mathbb{E}[e^{\lambda Y_m} \mid \mathcal{F}_{m-1}]]$, and *conditionally on $\mathcal{F}_{m-1}$* the variable Y_m is centered and bounded in an interval of length $2c_m$, so Hoeffding's lemma applies with $(b-a) = 2c_m$. Unrolling, $\mathbb{E}[e^{\lambda M_m}] \leq e^{\lambda^2 \sum c_i^2 / 2}$, and optimizing λ gives the stated tail. For McDiarmid, set $Y_i = \mathbb{E}[f \mid X_1, \dots, X_i] - \mathbb{E}[f \mid X_1, \dots, X_{i-1}]$: this is a martingale difference sequence with $|Y_i| \leq c_i$ precisely because resampling only X_i moves f by at most c_i (take conditional expectations over the two values of X_i), and $\sum_i Y_i = f - \mathbb{E}f$. Azuma–Hoeffding applied to (Y_i) is McDiarmid's inequality. ■

Proof of Lemma 2.7 (level-size estimation, Section 2.1)

Statement (recalled). Subsample each coordinate of a level set S_j , $|S_j| = s_j$, independently with probability $r = \min\{1, C \log n / (\varepsilon^2 s_j)\}$. If z coordinates survive, then $\tilde{s}_j = z/r$ satisfies $(1 - \varepsilon)s_j \leq \tilde{s}_j \leq (1 + \varepsilon)s_j$ with probability $1 - 1/\text{poly}(n)$. ■

Proof. Denote the coordinates of S_j by u_i , $i \in [s_j]$, and let X_i be the indicator that u_i is sampled into substream P , so the X_i are independent with $\mathbb{E}[X_i] = r$ and $\mathbb{E} \sum_i X_i = s_j r$. Multiplicative Chernoff (Theorem 2.4) gives

$$\begin{aligned} \Pr\left[\left|\sum_i X_i - s_j r\right| \geq \varepsilon s_j r\right] &\leq 2 \exp(-\varepsilon^2 s_j r / 3) \leq 2 \exp(-C \log n / 3) \\ &\leq 1/\text{poly}(n), \end{aligned}$$

where the second inequality uses $s_j r = C \log n / \varepsilon^2$ (when the minimum in the definition of r is attained at this branch; otherwise $r = 1$ and there is nothing to estimate). Dividing by r converts the event into $(1 - \varepsilon)s_j \leq \tilde{s}_j \leq (1 + \varepsilon)s_j$. ■

Proof of Lemma 2.9 (sampling concentration, Section 2.2)

Statement (recalled). If each point of $S = \{p_1, \dots, p_t\}$ lies in the candidate ball B independently with probability at least $\frac{1}{2} - \Theta(\varepsilon)$, then simultaneously for all $k \leq n$ candidate balls, $Y_B = \sum_i \mathbf{1}\{p_i \in B\} \geq (1 - \varepsilon)(\frac{1}{2} - \Theta(\varepsilon))t$ whp. ■

Proof. Fix a candidate ball B and let $X_i = \mathbf{1}\{p_i \in B\}$; the X_i are independent, and by the covering property $\mathbb{E}[Y_B] = \sum_i \mathbb{E}X_i \geq (\frac{1}{2} - \Theta(\varepsilon))t$. Multiplicative Chernoff gives

$$\Pr[Y_B < (1 - \varepsilon)\mathbb{E}Y_B] \leq e^{-\varepsilon^2 \mathbb{E}Y_B / 2} \leq e^{-\Theta(\varepsilon^2 t)}.$$

With $t = \Theta(\varepsilon^{-2} \log n)$ this is $n^{-\Omega(1)}$ per ball, and a union bound over the $k \leq n$ candidate balls gives the simultaneous statement $Y_B \geq (1 - \varepsilon)(\frac{1}{2} - \Theta(\varepsilon))t$ for all of them, with high probability. ■

Round-by-round noise control (Section 2.4.2)

Statement (recalled). The k -round walk mechanism of [7] is unbiased for the walk count W_k and, with probability $1 - \beta$, errs by at most $k\gamma\sqrt{N}(d(G) + \gamma)^{k-1}$, where $\gamma = 2k\sqrt{8\log(2kN/\beta)}/\varepsilon$. ■

Proof. Write $X_i^{(\ell)}$ for node i 's round- ℓ value. The mechanism gives, for each node i and round $\ell \in \{1, \dots, k\}$,

$$X_i^{(\ell)} = \sum_{j \in N(i)} X_j^{(\ell-1)} + \text{Lap}(2k\|X^{(\ell-1)}\|_\infty/\varepsilon),$$

so $\sum_{j \in N(i)} X_j^{(\ell-1)} \leq d(G)\|X^{(\ell-1)}\|_\infty$ always. Allocate failure β'/N per node per round: the Laplace tail $\Pr[\text{Lap}(b) > u] \leq \frac{1}{2}e^{-u/b}$ gives, with probability $1 - \beta'/N$ per node-round, $2k\text{Lap}(\|X^{(\ell-1)}\|_\infty/\varepsilon) \leq \gamma\|X^{(\ell-1)}\|_\infty$ for the stated γ (with $\beta' \approx \beta/k$). Union-bounding over the $\leq kN$ node-rounds, all these events hold simultaneously with probability $\geq 1 - (k-1)\beta'$, and on that event an induction gives

$$\|X^{(\ell)}\|_\infty \leq (d(G) + \gamma)\|X^{(\ell-1)}\|_\infty \implies \|X^{(\ell)}\|_\infty \leq (d(G) + \gamma)^\ell$$

for all $\ell \leq k-1$. The released count aggregates the N per-node values, whose centered Laplace contributions are independent sub-exponential variables each of scale at most $O(\gamma(d(G) + \gamma)^{k-1})$; their sum concentrates around zero at scale $\sqrt{N} \cdot \tilde{O}(\gamma(d(G) + \gamma)^{k-1})$ by the same Chernoff-method argument applied to light-tailed variables, which yields the stated $k\gamma\sqrt{N}(d(G) + \gamma)^{k-1}$ error radius with probability $1 - \beta$. ■

References

- [1] Kazuoki Azuma. Weighted sums of certain dependent random variables. *Tohoku Mathematical Journal, Second Series*, 19(3):357–367, 1967.
- [2] Stéphane Boucheron, Gábor Lugosi, and Pascal Massart. *Concentration Inequalities: A Nonasymptotic Theory of Independence*. Oxford University Press, 2013.
- [3] Herman Chernoff. A measure of asymptotic efficiency for tests of a hypothesis based on the sum of observations. *The Annals of Mathematical Statistics*, 23(4):493–507, 1952.
- [4] Devdatt Dubhashi and Alessandro Panconesi. *Concentration of Measure for the Analysis of Randomized Algorithms*. Cambridge University Press, 2009.

- [5] Rachana Gusain, Saladi Rahul, and A. Subramanian. A bouquet of results on maximum range sum: General techniques and hardness reductions. *Proceedings of the ACM on Management of Data*, 3, 2025. DOI: 10.1145/3767709. SIGMOD 2026 (rounds 1-2).
- [6] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58 (301):13–30, 1963.
- [7] Yihua Hu, Kuncan Wang, and Wei Dong. Acyclic graph pattern counting under local differential privacy. *Proceedings of the ACM on Management of Data*, 4, 2026. DOI: 10.1145/3802006. SIGMOD 2026.
- [8] Honghao Lin, Hoai-An Nguyen, and David P. Woodruff. On sketching trimmed statistics. *Proceedings of the ACM on Management of Data*, 4, 2026. DOI: 10.1145/3801907. SIGMOD 2026.
- [9] Shuainan Liu, Tianxi Ji, Zhongshuo Fang, Lu Wei, and Pan Li. Privacy loss of noise perturbation via concentration analysis of a product measure. *Proceedings of the ACM on Management of Data*, 4, 2026. DOI: 10.1145/3786680. SIGMOD 2026.
- [10] Colin McDiarmid. On the method of bounded differences. In J. Siemons, editor, *Surveys in Combinatorics 1989*, volume 141 of *London Mathematical Society Lecture Note Series*, pages 148–188. Cambridge University Press, 1989.
- [11] Michael Mitzenmacher and Eli Upfal. *Probability and Computing: Randomized Algorithms and Probabilistic Analysis*. Cambridge University Press, 2005.
- [12] Yuan Qiu, Xiaokui Xiao, and Yin Yang. Dp-s4s: Accurate and scalable select-join-aggregate query processing with user-level differential privacy. *Proceedings of the ACM on Management of Data*, 4, 2026. DOI: 10.1145/3802042. SIGMOD 2026.
- [13] Stanford CS229 course staff. Cs229 supplemental lecture notes: Hoeffding’s inequality. <https://cs229.stanford.edu/extra-notes/hoeffding.pdf>, accessed 2026. Stanford University.

3

Induction

Induction turns a claim about an unbounded process — millions of cache fills, node eliminations, rewrite steps, recursive calls — into two finite checks: the claim holds at the start, and one more step cannot break it. For a systems paper this is the instrument of *component* correctness. No grand named theorem is involved; the corpus pattern is thousands of small, unnamed invariants — half-page lemmas asserting that one more elimination step, recursive call, or rewrite keeps a data structure consistent — with each cache, rule, or decomposition step verified locally and the global guarantee assembled from the local ones. Induction is also the standard scaffolding for complexity proofs: the same step-by-step argument that certifies what a component computes carries the per-step cost accounting that becomes a running-time bound.

In the SIGMOD’26 corpus, 36 tagged papers lean on induction — the third most-used tool, behind reductions and concentration inequalities — spanning graph analytics, join processing, entity resolution, and query optimization. The pairing *correctness* $\times$ *induction*, with 29 papers, is the single most frequent property-tool co-occurrence of the survey, and induction also powers 21 papers tagged *upper-bound*: the invariant proves the algorithm right, the accompanying decreasing measure proves it fast. This chapter develops the three substrates and the craft first; Section 3.5 adds strengthened and nested invariants and two harder exemplars.

3.0 The tool: induction and invariants

The problem class is uniform across applications. A component evolves by discrete local steps — an algorithm executes operations, an elimination pipeline removes one node, a rewrite system fires one rule, a recursive evaluator makes one call — and the claim we need is global: *every* answer produced is correct, *every* intermediate structure stays within its budget, *no* reachable configuration drifts. The number of configurations is astronomically larger than the code, so the claim cannot

LEARNING OBJECTIVES

- Recognize the three substrates: execution steps, recursive structure, measures.
- State an invariant, close base and step cases, audit every operation.
- Strengthen an invariant that will not close; pair it with a measure for costs.
- Dissect a paper’s proof: induction variable, invariant, step case, payoff.

Figure 3.1: An invariant travels down every run: I holds at s_0 , each operation preserves I , so I holds at every reachable state — and I implies the specification.

$$\begin{array}{ccccccc} s_0 & \xrightarrow{\text{op}_1} & s_1 & \xrightarrow{\text{op}_2} & \cdots & \xrightarrow{\text{op}_t} & s_t \\ I(s_0) & & I(s_1) & & \cdots & & I(s_t) \end{array}$$

be checked state by state. What induction delivers is a predicate I on single configurations — the *invariant* — with two provable properties: it holds initially, and each single step preserves it. Those two finite checks certify the unbounded behavior. Every inductive argument in the corpus is an instance of this shape, differing only in *what the steps are*: states of a run, pieces of a recursively built object, or values of a decreasing counter.

Notation

I always denotes the predicate being propagated (the invariant); Γ the set of operations; $m: \mathcal{C} \rightarrow \mathbb{N}$ a measure. *Leaf-peeling* is the default move on tree-shaped objects: remove a leaf, invoke the hypothesis on the smaller object, extend by one step.

3.0.1 The three substrates

Substrate 1: execution steps. The component is imperative: it induces a sequence of states $s_0, s_1, \dots, s_t$, one per operation. This is the inductive-assertion method of Floyd [4] and Hoare [5], the loop-invariant discipline of algorithms texts [2], and Manna–Pnueli *safety* reasoning — nothing bad ever becomes true [9].

Theorem 3.1 — Invariant preservation. Let s_0 be an initial state and Γ a set of operations on states. Suppose the predicate I satisfies (i) $I(s_0)$, and (ii) $I(s) \Rightarrow I(\text{op}(s))$ for every $\text{op} \in \Gamma$. Then $I(s)$ holds for every state s reachable from s_0 by a finite sequence of operations; consequently, any specification P implied by I ($I \Rightarrow P$) holds at every reachable state. ■

Proof. Induction on the length t of the run $s_0 \rightarrow s_1 \rightarrow \cdots \rightarrow s_t$. The base case $t = 0$ is (i). For the step, $s_t = \text{op}(s_{t-1})$ for some operation, and (ii) propagates $I(s_{t-1})$ to $I(s_t)$. ■

Substrate 2: structural recursion. The object is built recursively — a tree decomposition, an elimination forest, a recursive pseudo-tree, a relational expression — and the property is proved on the whole from the property on the parts. The database workhorse is leaf-peeling: remove a leaf bag or leaf vertex, apply the hypothesis to the smaller object, then extend by one step.

Proposition 3.2 — Structural induction. Let $\mathcal{C}$ be a class of objects generated from atoms by formation rules. If a property P holds for every atom,

and each formation rule maps ingredients satisfying P to a result satisfying P , then every object of $\mathcal{C}$ satisfies P . In particular, on trees it suffices to prove P for leaves and to prove that a node whose children's subtrees satisfy P satisfies P itself.

Proof. Induction on the number of formation steps used to build the object. Atoms use zero steps. For a composite object, each immediate ingredient is built with fewer steps, so the hypothesis gives P on the ingredients, and the preservation assumption lifts P to the composite. The leaf-peeling form is the same argument read from the root downwards. ■

Substrate 3: a decreasing measure. Many arguments are phrased not over runs or recursions but over a quantity that strictly decreases — nodes left to eliminate, bags left to peel, rewrite steps remaining. Structural induction is routinely converted into this form by choosing a measure [10], and the classical least-counterexample principle closes both:

Lemma 3.3 — Strong induction. Let $m: \mathcal{C} \rightarrow \mathbb{N}$ be a measure. Suppose that for every object X , if $P(X')$ holds for all X' with $m(X') < m(X)$, then $P(X)$. Then $P(X)$ holds for every object X . ■

Proof. If not, the set of counterexamples has an m -minimal element X^* (the natural numbers are well-ordered). Every X' with $m(X') < m(X^*)$ satisfies P , so the hypothesis yields $P(X^*)$ — a contradiction. ■

1

3.0.2 How to choose a substrate

The selection is mechanical once one question is answered: *what does one step look like?* If the component is an explicit state machine — a sweep with a stack, an incrementally maintained index — take execution steps and a state invariant (Substrate 1). If the object of study is recursively defined and the claim is about the finished object — decompositions, pseudo-trees, expressions — take structural induction, leaf-peeling by default (Substrate 2). If the argument is really about a process with no convenient state — an elimination order, a rewrite sequence, a peeling procedure — phrase it as strong induction on the natural measure (Substrate 3). All three are the same lemma in different clothes; pick the clothing that makes the step case a purely local check.

¹ The measure earns a second salary: since each step decreases m by at least one, the component terminates in at most $m(s_0)$ steps, and attaching a per-step cost to the induction turns the correctness proof into a running-time proof. This is how the same argument underwrites both the *correctness* pairing (29 papers) and the *upper-bound* pairing (21 papers) of this chapter's corpus.

When to Reach for Induction

Reach whenever a component has state evolving by local steps and the claim is “nothing drifts” — caches, elimination pipelines, rewrite rules, recursive evaluators — or whenever a guarantee must hold for *all* inputs of unbounded size. Then run the recipe:

1. Name the substrate and fix the induction variable precisely: the step index i of a run, the subtree rooted at a node A , the number of bags/nodes/rules left.
2. Write the invariant I as a property of *one* state or *one* subtree, strong enough to imply the specification at final states (or at the whole object) and concrete enough to check in one step. Audit the step case over *every* operation — and the trivial base cases (empty input, single bag, zero eliminations): one missed case silently breaks the entire argument.
3. If the step case will not close, *strengthen* I : carry more information (the exact block shape, the exact cache contents, both directions of an equivalence). Strengthening is the normal iteration of the method, not a failure — see Proposition 3.13.
4. Pair I with a strictly decreasing measure to obtain termination and per-step cost accounting — the same induction then yields the time/space bound, not just correctness.

3.1 Resistance-distance labels on small-treewidth graphs

The exemplar is a SIGMOD’26 paper on exact graph analytics over small-treewidth graphs [8]. Resistance distance — the effective resistance between two nodes when the graph is viewed as an electrical network — is a workhorse similarity measure in graph databases: link prediction, recommendation, and graph-feature extraction all use it because it averages over *all* paths between two nodes, not just the shortest one. Computing it exactly requires the pseudo-inverse of the graph Laplacian, a matrix inversion that does not scale to large graphs, which is why practitioners settle for approximations. The paper observes that on graphs of small treewidth — road networks, hierarchies, many social and biological networks — Gaussian elimination ordered along a tree decomposition factors the computation *once*, so that afterwards every pair query is answered from two small labels alone. Its headline results: an index built in $O(n \cdot d_{\max} \cdot h_G^2)$ time, labels of total size $O(n \cdot h_G)$, and *exact* single-pair queries in $O(h_G)$, where h_G is the height of the tree decomposition and $d_{\max}$ the maximum degree. The correctness of the

elimination pipeline is where induction enters.

Problem. Let $G = (V, E)$ be a graph with n nodes, Laplacian $L = D - A$, and $r(s, t) = (\mathbf{e}_s - \mathbf{e}_t)^\top L^+ (\mathbf{e}_s - \mathbf{e}_t)$ the resistance distance, where L^+ is the Moore–Penrose pseudo-inverse. The input includes a tree decomposition of G of height h_G (the maximum number of bags on a root-to-leaf path). The task is to preprocess G into one label $\ell(v)$ per node so that any query $r(s, t)$ is answered exactly from $\ell(s)$ and $\ell(t)$ alone, with preprocessing, label size, and query time parameterized by n , $d_{\max}$, and h_G .

Guarantee.

Theorem 3.4 — Labelling scheme, complexity lemmas of the paper. Given G and a tree decomposition of height h_G , the index construction computes labels of total size $O(n \cdot h_G)$ in time $O(n \cdot d_{\max} \cdot h_G^2)$, and any single-pair query $r(s, t)$ is answered *exactly* in $O(h_G)$ time from the two labels; a single-source query over all t costs $O(n \cdot h_G)$. ■

2

How induction is applied. The algorithm eliminates the nodes one at a time, in an order given by a traversal of the tree decomposition, maintaining a shrinking *state matrix* S initialized to the inverse of the Laplacian restricted to the current node set; eliminating node v applies the rank-one update $S \leftarrow S[:, v] S[:, v]^\top / S[v, v]$. The induction variable is the number of eliminated nodes, and the induction hypothesis is stated as an explicit invariant of the state:

Lemma 3.5 — Elimination-state invariant, Lemma of the paper, notation cleaned. Let $U_1 \subseteq U_2 \subseteq V$. After the elimination of the nodes of $U_2 \setminus U_1$, the state matrix satisfies

$$S_{|U_2 \setminus U_1|} = \begin{pmatrix} (L_{|U_1|})^{-1} & 0 \\ 0 & 0 \end{pmatrix},$$

i.e. all rows and columns of eliminated nodes are zero, and the block on the surviving nodes is the inverse of the Laplacian restricted to U_1 . ■

Proof sketch. Induction on the number k of eliminated nodes. Base case $k = 0$: the state is $(L_{|U_2|})^{-1}$ by initialization. Step: partition the state around the next eliminated node v ; the rank-one update zeroes row and column v and replaces the surviving block by a Schur complement, which the block-inverse formula identifies with the inverse of the Laplacian restricted to the remaining nodes. (Full proof in Section 3.6.) ■

² Why exactness is plausible here: on a tree decomposition, eliminating a node interacts only with its bag, so the inverse Laplacian factors into per-path pieces — the random-walk interpretation (resistance is a commute time) then reconstructs $r(s, t)$ from the pieces on the two root paths, via cut properties: a vertex cut of s, t zeroes the cross terms.

The labels store the elimination history: for each node, the columns of the state at the moments its neighbors were eliminated (a subtree-contiguity property of the traversal keeps each column segment addressable), and the query walks the two root paths recombining those pieces through the resistance formula. Note the craft: the invariant is an *exact block shape*, not “approximately correct” — strong enough to pin every entry of the state, which is what makes the reconstruction exact. The running-time bound rides the same induction: each of the n elimination steps touches at most $d_{\max}$ already-processed neighbors on segments of length $O(h_G)$, giving the $O(n \cdot d_{\max} \cdot h_G^2)$ total.

What it buys. The engine answers exact resistance-distance queries in time proportional to the decomposition height rather than the graph size, after near-linear preprocessing on low-treewidth data — no approximation error, no sampling. The pattern — eliminate along a decomposition, store the elimination history as labels, reconstruct from labels by an invariant — transfers to any computation that factors through Gaussian elimination on small-treewidth structure.

3.2 *Space–time plans for sum–product queries*

The exemplar is a SIGMOD’26 paper on the space–time complexity of conjunctive-query evaluation [3]. A *sum–product query* is a conjunctive query evaluated over a semiring: ordinary joins (Boolean semiring), output counts (natural numbers), tuple probabilities, or optima (tropical semiring). Worst-case optimal join algorithms evaluate such queries in optimal time but materialize as they go, paying full space for intermediate results; systems instead want caching plans that trade time for space. The paper asks which points of the space–time plane a query can occupy: it defines a ladder of plan classes — pseudo-trees, pseudo-trees with caching, tree-decomposition plans, and recursive pseudo-trees (RPTs) — ordered by a domination preorder on their (s, t) exponent pairs, and identifies what each class achieves. Headline results: every tree-decomposition plan is dominated by an RPT plan (proved by the second induction exhibited below), the dominations are mostly strict, witnessed by explicit path queries, and a Triple- k -Clique conjecture yields conditional space–time lower bounds over the tropical semiring.

Problem. Fix a semiring $\mathbb{K}$ and a sum–product query $Q(\mathbf{X}) = \prod_i R_i(\mathbf{X}_i)$; the database annotates tuples with values of $\mathbb{K}$, an assignment multiplies its annotations, and Q sums over satisfying assignments. A *pseudo-tree* is a directed tree over the variables of Q in which every atom’s variables sit on a single root-to-leaf branch; a plan is a pseudo-tree, possibly with caches at some vertices and applied recursively to

subqueries, and its exponents s and t (defined recursively in the paper) guarantee evaluation in space $O(|D|^s)$ and time $O(|D|^t)$. Plan P_1 *dominates* P_2 if $t(P_1) \leq t(P_2)$ and $s(P_1) \leq s(P_2)$; a class dominates another if for every query and every plan of the latter there is a dominating plan of the former.

Guarantee.

Theorem 3.6 — RPT evaluation, Theorem of the paper, compressed. Every recursive pseudo-tree plan $(\mathcal{P}, \mathcal{C}, \rho)$ of Q with exponents $s = s(\mathcal{P}, \mathcal{C}, \rho)$ and $t = t(\mathcal{P}, \mathcal{C}, \rho)$ correctly evaluates Q over any semiring, in space $O(|D|^s)$ and time $O(|D|^t)$. ■

3

How induction is applied. Two inductions run side by side. The *correctness* half is execution-step induction over the run of the recursive algorithm: the induction variable is the sequence of function calls $\text{solve}(A, \mathbf{x})$, and the invariant says each call returns exactly the value of the subquery rooted at A , with the cache holding exactly that value:

Lemma 3.7 — Cache invariant, Lemma of the paper, paraphrased. During any run of the algorithm on an RPT $(\mathcal{P}, \mathcal{C}, \rho)$, every call $\text{solve}(A, \mathbf{x})$ returns exactly the value of the subquery of Q rooted at A , conditioned on the part of the partial assignment $\mathbf{x}$ over the variables of A ; after the subsequent fillCache call, the cache M_A stores exactly this value. Hence the top-level call $\text{solve}(r(\mathcal{P}), ())$ returns $Q(\mathbf{D})$. ■

Proof sketch. Structural induction on the RPT, i.e. on the number of variables in the subtree rooted at A . Base case (single variable): the call accumulates $\bigoplus_a a$ directly, which is the subquery value. Step: the call iterates over candidate values a , multiplies the — by hypothesis correct — returns of the children’s calls, and accumulates; the cache variant additionally uses that two partial assignments agreeing on A ’s interface define the same subquery value. (Full proof in Section 3.6.) ■

The *domination* half (every tree-decomposition plan is dominated by an RPT plan) is leaf-peeling structural induction on the number of bags of the tree decomposition T : with a single bag the plan is already a pseudo-tree plan (base case); otherwise let v be a leaf of T with parent p , eliminate the variables occurring in v ’s bag but not in p ’s into a single new atom over their joint neighbors, apply the hypothesis to the smaller decomposition, and attach the resulting RPT at the parent — each peeling step preserves plan value and exponents, so the constructed RPT dominates.

³ The benchmark being traded against: a worst-case optimal join evaluates a full query in time $O(|D|^{\rho^*})$ with ρ^* the fractional edge cover — but materializes intermediates, i.e. space exponent ρ^* as well. The paper’s sublinear-space plans give up time exponents to buy space back.

What it buys. The optimizer gets a certified menu of space–time trade-offs — sublinear-space evaluation of full queries by caching at the right granularity — rather than a single all-or-nothing algorithm. The cache invariant is the implementer’s contract: it says precisely what each cache entry means, so the recursion can be memoized (and its cost accounted) safely; and the strict separations tell designers when adding a plan class genuinely buys new exponent pairs.

3.3 Near-duplicate text alignment by skyline sweeps

The exemplar is a SIGMOD’26 paper on all-pair near-duplicate text alignment [11]. Finding all pairs of near-duplicate substrings in long texts is a bread-and-butter entity-resolution task: plagiarism detection, version tracking, deduplication of logs and records. Locality-sensitive hashing (LSH) is the standard sieve — hash positions so that similar stretches collide, then verify candidates — but the candidate generation itself is the bottleneck: the maximal hash-consistent stretches, which the paper calls *tight intervals*, live in a space of quadratically many intervals, and missing even one means a silently missed near-duplicate pair. The paper characterizes tight intervals through a *dominance skyline* of positions and gives a monotone-stack sweep that generates exactly the tight intervals of a text of length n in $O(n)$ time per hash function, with $O(\log n)$ expected stack — and an $O(mn)$ expected-time alignment pipeline with a completeness guarantee: no near-duplicate pair is missed.

Problem. Let T be a text of length n and h a hash function on positions (drawn from a random family); write $h(T[q])$ for the hash of the token at position q . Position q *dominates* p (for $p < q$) iff $h(T[q]) < h(T[p])$. An interval $[l, r]$ is *left-tight* iff $l = 1$ or $h(T[l - 1])$ is smaller than every hash on $[l, r]$, and *right-tight* iff $r = n$ or $h(T[r + 1])$ is smaller than every hash on $[l, r]$; for a composite $g = (h_1, \dots, h_m)$, $[l, r]$ is *tight* iff it is left-tight under some h_i and right-tight under some h_j . The task: enumerate, exactly once each, all tight intervals of T under h (and, for m functions, under g).

Guarantee.

Theorem 3.8 — Tight-interval generation, Theorems 1–2 of the paper. The monotone-stack sweep (Algorithm 1 of the paper) generates exactly the tight intervals of T under h — every generated interval is tight, and every tight interval is generated — in $O(n)$ time, producing $O(n)$ intervals, with the stack holding $O(\log n)$ positions in expectation. With m independent hash functions the full pipeline runs in $O(mn)$ expected time. ■

How induction is applied. The induction variable is the scan position r ; the state is the *ordered skyline* $S(r)$ — the positions $p \leq r$ not dominated by any later position, in increasing order, whose hash values increase along the skyline and whose last element is r . The invariant: at the end of each loop iteration, S is exactly the ordered skyline of the current position. The workhorse is the one-step update:

Lemma 3.9 — Skyline update, Lemma 6 of the paper. Let $S(r) = (p_1, p_2, \dots, p_s)$ be the ordered skyline at position r and $i = \min\{j \in [1, s] : h(T[p_j]) < h(T[r+1])\}$. Then the ordered skyline at position $r+1$ is $S(r+1) = (p_1, \dots, p_i, r+1)$. ■

Proof sketch. By the monotonicity of hashes along the skyline, every p_j with $j > i$ has $h(T[p_j]) > h(T[r+1])$, so the new position $r+1$ dominates exactly those — they leave the skyline. Positions are dominated only by later ones, so the prefix $(p_1, \dots, p_i)$ keeps its pairwise order, and $r+1$ (the latest position, with the largest hash on the new skyline) is appended last. ■

The base case is $S(0) = (0)$. The stack operations — pop while the top’s hash exceeds $h(T[r+1])$, then push $r+1$ — implement exactly the update of Lemma 3.9, so the invariant holds after every iteration. Exactness then follows by combining the invariant with two characterization lemmas of the paper: an interval $[l, r]$ is left-tight iff $l-1 \in S(r) \setminus \{r\}$, and right-tight iff $h(T[r+1]) < h(T[p])$ where p is the smallest skyline position $\geq l$ (whose hash is the minimum on $[l, r]$). The sweep emits an interval exactly at a pop, which is exactly when both characterizations fire — soundness and completeness at once. (Full proof in Section 3.6.)

What it buys. The alignment stage enumerates all LSH candidates in linear time per hash function, replacing quadratic scans, with $O(\log n)$ expected working memory — and the exactness theorem doubles as a *recall certificate* for the entity-resolution pipeline: no near-duplicate pair can be missed. The pattern — a dominance order on positions, the skyline as state, a monotone stack as its compressed representation — is the same skeleton as classical monotone-stack algorithms, here made inductive and analyzable.

3.4 Succinct representations of all width-1 join plans

The exemplar is a SIGMOD’26 paper on compressing the optimizer’s search space [6]. Join-order selection is NP-hard in general, so cost-based optimizers enumerate and cost candidate *join trees*; for star queries the space is enormous — n^{n-1} join trees for n relations — and enumerating them one by one does not scale. The paper focuses

on the plans that matter most, *width-1* plans whose intermediate results are bounded by $O(N)$ in the maximum base cardinality N , and asks for a succinct representation from which all of them can be enumerated without duplicates. Its headline results: width-1 plans exist exactly for acyclic queries; a *meta-decomposition* — a labeled tree of size $O(|E(H)|)$ computed in $O(|E(H)|^3)$ — represents all join trees of an acyclic hypergraph; and an enumeration procedure outputs exactly the join trees, so the optimizer searches a linear-size object instead of an exponential family.

Problem. Let Q be a conjunctive query with hypergraph H (edges = atoms). A *join tree* of H is a tree over the hyperedges such that the hyperedges containing any vertex form a connected subtree; H is *acyclic* iff it has a join tree. For a (physical) query plan P , its *width* $w(P)$ is the least exponent such that every intermediate result of P is $O(N^{w(P)})$. For a hyperedge e , its *outer part* is $o(e, H) = e \cap (\bigcup(E(H) \setminus \{e\}))$ — the portion of e shared with other edges — and an *ear* is a hyperedge the classical ear-removal characterization of acyclicity may peel off. The task: characterize the width-1 plans and represent all join trees succinctly.

Guarantee.

Theorem 3.10 — Width-1 plans and their enumeration, Theorems of the paper, compressed. (i) A conjunctive query Q has a width-1 query plan if and only if Q is acyclic. (ii) For acyclic H , the meta-decomposition M produced by the algorithm has $|V(M)|, |E(M)| = O(|E(H)|)$, is computed in $O(|E(H)|^3)$ time, and the enumeration procedure outputs exactly the join trees of H — every output is a join tree, and every join tree is output. ■

⁴ The n^{n-1} count for star queries is Cayley's formula — the paper pairs it with Prüfer sequences to count join trees of specific shapes, sharpening just how exponential the naive search space is.

4

How induction is applied. The elimination structure is the ear-removal order: the meta-decomposition tree records it (a node p is a descendant of q iff e_p is removed before e_q), and the correctness and completeness of the enumeration are proved by structural induction over that tree — the induction hypothesis at a node is that the subtree's physical nodes induce a connected subtree of every join tree, so local choices extend to full join trees and every join tree arises. The step case that lets the algorithm remove ears in *batches* is:

Proposition 3.11 — Batch ear removal, Proposition of the paper. Let H be an acyclic hypergraph and $S = \{e_1, \dots, e_n\} \subseteq E(H)$, $n \geq 2$, a set of distinct ears with a common outer part $o(e_1, H) = \dots = o(e_n, H) =: o$. Then

the hypergraph H' with $E(H') = (E(H) \setminus S) \cup \{o\}$ is again acyclic. ■

The enabling fact — which pins the shared structure of same-outer-part ears — is a two-line set-algebra lemma:

Lemma 3.12 — Ears with equal outer parts, Lemma of the paper. If two ears e_1, e_2 of an acyclic hypergraph H satisfy $o(e_1, H) = o(e_2, H)$, then $o(e_1, H) = o(e_2, H) = e_1 \cap e_2$. ■

Proof sketch. The common value lies in both ears (each outer part is contained in its own edge), and conversely $e_1 \cap e_2$ lies in the outer part of e_2 because e_1 is one of “the other edges” for e_2 — a double inclusion. (Full proof in Section 3.6.) ■

With the batch proposition, the peeling induction proceeds on the number of ears remaining: remove all same-outer-part ears at once, insert the single new ear o , witness a join tree of H' from one of H (the shared outer part glues the removed ears’ positions into one subtree), and recurse on the strictly smaller hypergraph — the measure $|E(H)|$ decreases at every step.

What it buys. The optimizer keeps a linear-size certificate that stands for an exponential family of plans: enumerating all width-1 — i.e. linear-cost — join plans becomes a traversal of the meta-decomposition, with the exactness of the enumeration (no invalid plan, no duplicates, nothing missed) inherited from the peeling induction. On top of it, the paper evaluates relation-dominated queries in $O(|db|)$ time by restricting plan search to the width-1 space the theorem certifies.

3.5 Advanced usage and further reading

The substrate lemmas of Section 3.0 are complete — nothing deeper is ever added. What distinguishes advanced usage is the *craft* on top: deliberate strengthening of the hypothesis, and nesting one induction inside another. This section states the strengthening principle, sketches two harder exemplars, and points to the literature.

3.5.1 The advanced rungs: strengthening and nesting

Proposition 3.13 — Strengthening. To certify a specification P it suffices to exhibit *any* inductive predicate I with $I \Rightarrow P$; the specification P itself need not be inductive. Formally: if $I(s_0)$, I is preserved by Γ , and $I(s) \Rightarrow P(s)$ for all states s , then P holds at every reachable state. ■

Proof. Theorem 3.1 applied to I gives I at every reachable state; $I \Rightarrow P$ transfers it to P . ■

The point is methodological. Target specifications are almost never inductive — “the output equals the subquery answer” says nothing about a half-finished run — so the prover iterates: add bookkeeping to the predicate until the step case closes. Every exemplar of this chapter did it: the exact block shape (Lemma 3.5), the exact cache contents (Lemma 3.7), the ordered skyline with its monotone hashes (Lemma 3.9), the ear-removal order with shared outer parts (Proposition 3.11). *Nesting* is the second rung: two induction variables, an outer structural induction with an inner execution-step induction per piece — the sum-product exemplar of Section 3.2 nests its cache-invariant run argument inside the leaf-peeling domination proof. Beyond the naturals, termination arguments move to well-founded measures on multisets; those live in the term-rewriting literature rather than in this corpus.

3.5.2 Two advanced exemplars, sketched

Both exemplars of this section push induction into proof machinery that is itself the contribution; each is sketched in one paragraph.

Fixed-parameter width checking [7]^{PODS}. This PODS’26 paper settles the parameterized complexity of two central width measures: it is fixed-parameter tractable to check $\text{ghw}(H) \leq k$ and $\text{fhw}(H) \leq k$ — generalised and fractional hypertree width, the decomposition measures behind beyond-acyclic query evaluation — parameterised by k , the rank of H , and its maximum predicate arity $\Delta(H)$ (Corollary 8 of the paper; the first FPT algorithms for these problems). The engine transplants the MSO-transduction treewidth algorithm of Bojańczyk and Pilipczuk from tree decompositions to *elimination forests*: a transduction rewrites a suboptimal forest into an optimal one, and its correctness is an induction over the forest whose step case is the paper’s Reduced-Hypergraph Dealternation Lemma — alternation depth (the *split* and *irregularity* parameters) stays bounded, with conflict-graph coloring and neutral-swap/pigeonhole arguments supplying the bound. The invariant carried by the induction: the output is always an elimination forest of the same hypergraph, of width no larger than the input’s.

Codd’s theorem over semirings, and its limit [1]. Codd’s theorem — relational algebra equals domain-independent relational calculus — is the foundation of declarative query languages, and provenance semirings (annotating tuples with probabilities, counts, confidences) are the standard way to give queries provenance semantics. The paper transports Codd’s theorem to $\mathbb{K}$ -relations over semirings with *monus* (truncated subtraction) and *support*: RA-expressibility, domain-independent cal-

culus definability, and calculus definability coincide exactly over zero-sum-free semirings (Theorem of the paper), and the equivalence fails without zero-sum-freeness. Both directions rest on structural induction on expressions and formulas: every algebra operator translates to a calculus connective and back, and the hypothesis “the translation defines the same $\mathbb{K}$ -relation” is preserved by each operation. The same technique then proves an *inexpressibility*: over the bag semiring $\mathbb{N}$, division is not definable in basic relational algebra. Witness databases make division produce a tuple of multiplicity 2^n , while an induction on the construction of any expression E bounds its support by $n^{l(E)}$ and its multiplicities by $p_E(n) \cdot 2^{l(E)}$ — growth that 2^n outruns. Induction here counts what expressions *can* do: the mirror image of its usual certifying use.

3.5.3 Further reading

- Floyd [4] and Hoare [5] — the origins of the invariant-assertion method: verification as base case plus preservation.
- Manna and Pnueli [9] — safety properties (“nothing bad ever happens”) as the invariant-based fragment of temporal logic.
- Cormen, Leiserson, Rivest, and Stein [2] — the loop-invariant discipline for algorithms, with worked analyses.
- Rosen [10] — a refresher on strong induction, structural induction, and the well-ordering principle they rest on.

Chapter Recap

- **What the tool is.** Induction converts claims about unbounded behavior into two finite checks — an invariant holds initially and survives one step — over whichever substrate the component offers: execution steps, recursive structure, or a decreasing measure (all three interderivable, Lemma 3.3).
- **Which substrate when.** Explicit state machine (sweep, incremental index): state invariant. Recursively built object (decomposition, pseudo-tree, expression): structural induction, leaf-peeling by default. Process without explicit state (elimination order, rewrite sequence): strong induction on the natural measure.
- **Why the craft matters.** Specifications are rarely inductive; strengthen the predicate — exact block shape, exact cache contents, monotone skyline, removal order — until the step case closes (Proposition 3.13), audit every operation, and pair the invariant with a decreasing measure to get termination and cost

bounds.

- **What the corpus showed.** Resistance-distance labels (Section 3.1): induction on eliminated nodes, block-shape invariant. Sum-product plans (Section 3.2): cache invariant over the run, leaf-peeling over decompositions. LSH alignment (Section 3.3): skyline invariant over the scan, monotone-stack update. Succinct plan representations (Section 3.4): ear-removal induction with batch peeling.
- **Advanced usage.** Strengthening and nesting (Section 3.5.1) are the rungs beyond the basics; the advanced exemplars of Section 3.5.2 run induction inside MSO transductions and use it to count what expressions can do. Deferred proofs are collected in the Appendix (Section 3.6).

3.6 Appendix

Proofs omitted from the main text are collected here.

Proof of Lemma 3.5 (elimination invariant)

Statement (recalled). After eliminating the nodes of $U_2 \setminus U_1$ from the state initialized to $(L_{|U_2|})^{-1}$, the state matrix is $\begin{pmatrix} (L_{|U_1|})^{-1} & 0 \\ 0 & 0 \end{pmatrix}$ — zero rows and columns on eliminated nodes, the inverse restricted Laplacian on the survivors. ■

Proof. Induction on the number k of eliminated nodes. *Base case* $k = 0$: the state is $S_0 = (L_{|U_2|})^{-1}$ by initialization, and no rows or columns are zeroed. *Induction hypothesis*: after k eliminations, all rows and columns of the eliminated nodes are zero, and the block on the remaining node set U is the inverse of the Laplacian restricted to U . *Step*: eliminate $v = v_{k+1}$ and partition the state as $S_k = \begin{pmatrix} A & b \\ b^\top & c \end{pmatrix}$ with $c = S_k[v, v]$. The rank-one update gives

$$S_{k+1} = S_k - \frac{S_k[:, v] S_k[:, v]^\top}{c} = \begin{pmatrix} A - b b^\top / c & 0 \\ 0 & 0 \end{pmatrix},$$

so row and column v are zeroed. The surviving block $A - b b^\top / c$ is the Schur complement of c in S_k , and by the block-inverse identification — if $M = \begin{pmatrix} A' & B \\ C & D \end{pmatrix}$ has inverse $\begin{pmatrix} E & F \\ G & H \end{pmatrix}$, then $E^{-1} = A' - B D^{-1} C$ — the inverse of that block is the restriction of $(S_k)^{-1}$ to the remaining nodes, which by the induction hypothesis is the Laplacian restricted to $U \setminus \{v\}$. After all $|U_2 \setminus U_1|$ eliminations the block on U_1 is $(L_{|U_1|})^{-1}$. ■

Proof of Lemma 3.7 (RPT cache invariant)

Statement (recalled). Every call $\text{solve}(A, \mathbf{x})$ returns exactly the value of the subquery of Q rooted at A conditioned on the part of $\mathbf{x}$ over A 's variables, and after fillCache the cache M_A stores exactly this value; hence the top-level call returns Q . ■

Proof. Write, as in the paper, $Q[Y \mid \mathbf{y}]$ for the subquery of Q restricted to the variables Y and conditioned on the partial assignment $\mathbf{y}$, and let $\chi(A)$ be the variables of the subtree rooted at A . Induction on $|\chi(A)|$. *Base case* $|\chi(A)| = 1$: the call allocates an output relation and accumulates over the candidate values at A ; by the paper's subquery identity this accumulation is exactly $Q[\chi(A) \mid \mathbf{x}]$. *Step:* for $|\chi(A)| > 1$, the call iterates over candidates a and multiplies the returns of the children's calls,

$$\text{OUT} = \bigoplus_a \left(a \otimes \bigotimes_{B \in \text{child}(A)} \text{solve}(B, (\mathbf{x}, a)) \right),$$

which, by the induction hypothesis applied to the children (whose subtrees are strictly smaller) and the same subquery identity, equals $Q[\chi(A) \mid \mathbf{x}]$. For the cache clause, let $\mathbf{y}$ be the first argument reaching A whose restriction to A 's interface agrees with $\mathbf{x}$'s: the call stores $Q[\chi(A) \mid \mathbf{y}]$ in M_A , and every later call with the same interface returns it — correct because two assignments agreeing on the interface define equal subquery values, $Q[\chi(A) \mid \mathbf{y}] = Q[\chi(A) \mid \mathbf{x}]$. The top-level call has an empty assignment and the full tree, so it returns Q . ■

Proof of Theorem 3.8 (tight-interval exactness)

Statement (recalled). The sweep generates exactly the tight intervals of T : every generated interval is tight, every tight interval is generated; a text of length n is processed in $O(n)$ time with $O(n)$ intervals emitted and an $O(\log n)$ expected stack. ■

Proof. First, by induction on r : after the iteration processing r , the stack S is the ordered skyline $S(r)$. Base case $r = 0$: $S = (0)$. Step: the while-loop pops exactly the positions whose hashes exceed $h(T[r+1])$, and the push appends $r+1$, so by Lemma 3.9 the stack is $S(r+1)$. Second, soundness: the sweep emits $[l, r]$ when popping p , where $l-1$ is the new stack top, so $l-1 \in S(r) \setminus \{r\}$ and the paper's Lemma 1 gives left-tightness; the popped p is the smallest skyline position $\geq l$, its hash is the minimum on $[l, r]$, and the pop condition $h(T[r+1]) < h(T[p])$ gives right-tightness by the paper's Lemma 3. Third, completeness: fix a tight $[l, r]$. By Lemma 1, $l-1 \in S(r)$ at the start of iteration r ; right-tightness gives $h(T[r+1]) < h(T[p])$ for p the smallest skyline position $\geq l$, so the pop of p — which the paper's Lemma 4 shows

cannot be avoided — exposes $l - 1$ as the top, and $[l, r]$ is emitted. For the complexity: each position is pushed once and popped at most once, so the loop body executes $O(n)$ times and emits one interval per pop; the stack bound is the paper’s Lemma 7, $\mathbb{E} |S(r)| = O(\log r)$ over the random hash family. ■

Proof of Lemma 3.12 (equal outer parts)

Statement (recalled). If ears e_1, e_2 of an acyclic hypergraph H satisfy $o(e_1, H) = o(e_2, H)$, then the common value is $e_1 \cap e_2$. ■

Proof. Recall $o(e, H) = e \cap (\bigcup (E(H) \setminus \{e\}))$, so $o(e, H) \subseteq e$ for every edge e . Let $o = o(e_1, H) = o(e_2, H)$. Since $o = o(e_1, H) \subseteq e_1$ and $o = o(e_2, H) \subseteq e_2$, we get $o \subseteq e_1 \cap e_2$. Conversely, e_1 is one of the other edges for e_2 , so

$$e_1 \cap e_2 \subseteq e_2 \cap (\bigcup (E(H) \setminus \{e_2\})) = o(e_2, H) = o.$$

The two inclusions give $o = e_1 \cap e_2$. ■

References

- [1] Guillermo Badía, Phokion G. Kolaitis, and Carles Noguera. Codd’s theorem for databases over semirings. *Proceedings of the ACM on Management of Data*, 3, 2025. DOI: 10.1145/3767713. SIGMOD 2026 (rounds 1-2).
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.
- [3] Kyle Deeds, Timo Camillo Merkl, Reinhard Pichler, and Dan Suciu. The space-time complexity of sum-product queries. *Proceedings of the ACM on Management of Data*, 3, 2025. DOI: 10.1145/3767719. SIGMOD 2026 (rounds 1-2).
- [4] Robert W. Floyd. Assigning meanings to programs. *Proceedings of Symposia in Applied Mathematics*, 19:19–32, 1967.
- [5] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.
- [6] Zhekai Jiang, Qichen Wang, and Christoph T. Koch. Succinct structure representations for efficient query optimization. *Proceedings of the ACM on Management of Data*, 4, 2026. DOI: 10.1145/3802117. SIGMOD 2026.

- [7] Matthias Lanzinger, Igor Razgon, and Daniel Unterberger. Fpt parameterisations of fractional and generalised hypertree width. *Proceedings of the ACM on Management of Data*, 4, 2026. DOI: 10.1145/3801900. SIGMOD 2026.
- [8] Meihao Liao, Yueyang Pan, Rong-Hua Li, and Guoren Wang. Efficient exact resistance distance computation on small-treewidth graphs: A labelling approach. *Proceedings of the ACM on Management of Data*, 3, 2025. DOI: 10.1145/3769788. SIGMOD 2026 (rounds 1-2).
- [9] Zohar Manna and Amir Pnueli. *Temporal Verification of Reactive Systems: Safety*. Springer, 1995.
- [10] Kenneth H. Rosen. *Discrete Mathematics and Its Applications*. McGraw-Hill, 8 edition, 2019.
- [11] Y. Zhang, Zhencan Peng, Miao Qiao, Wei Zhang, Feifei Li, and Dong Deng. Lshalign: All-pair near-duplicate text alignment via locality-sensitive hashing. *Proceedings of the ACM on Management of Data*, 4, 2026. DOI: 10.1145/3802083. SIGMOD 2026.

4

Estimation Theory

4.1 Estimators: the interface between sampling and guarantees

Database systems rarely compute quantities exactly: sketches compress frequency vectors, samplers select a few rows or a few random spanning trees, and privacy mechanisms deliberately add noise. Estimation theory is what lets a paper say something rigorous about the resulting number. Whenever a data structure or mechanism reports a value $\hat{\theta}$ together with a claim of the form “within ε of the truth with probability at least $1 - \delta$,” an *estimator* has been constructed and its error has been bounded. The three classical questions transfer verbatim: is $\hat{\theta}$ *unbiased*, how large are its *variance* or mean-squared error, and can *any* estimator in the same situation do better, i.e., what does a *minimax* or information-theoretic lower bound permit [? ?]? In the SIGMOD’26 corpus, estimation theory is used by 27 of the 228 tagged papers and is the most frequent companion of the *error-bound* (16 papers) and *probabilistic-guarantee* (13 papers) tags: it is the tool that turns a data reduction into a guarantee.

The workflow typically runs backwards from the target guarantee: build a sketch or sampler; read an estimator off it; bound the estimator’s variance; convert variance into a confidence statement with a concentration inequality; pay a union bound over the keys, nodes, or rounds that must be simultaneously correct; and, in the strongest papers, prove a matching lower bound showing the error or the space is optimal.

4.2 The tool

Bias–variance decomposition. For an estimator $\hat{\theta}$ of a parameter θ ,

$$\text{MSE}_{\theta}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \theta)^2] = \underbrace{(\mathbb{E}[\hat{\theta}] - \theta)^2}_{\text{squared bias}} + \underbrace{\text{Var}(\hat{\theta})}_{\text{variance}}, \quad (4.1)$$

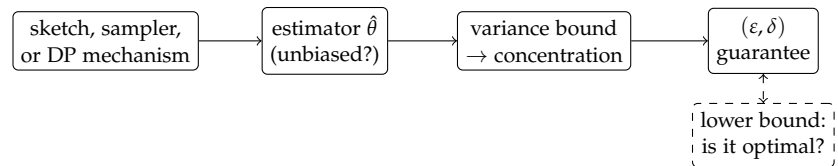
and every design decision in a sketch or mechanism is a move between the two terms. Unbiasedness ($\mathbb{E}[\hat{\theta}] = \theta$) is prized because variances *add* over independent contributions — per-key counters, per-round messages, per-node estimates — so one variance computation feeds Chebyshev or Chernoff bounds and yields the (ε, δ) statement directly [?]. Its fragility matters equally: unbiasedness is destroyed by nonlinear post-processing, $\mathbb{E}[g(X)] \neq g(\mathbb{E}[X])$, so estimating x^p from an unbiased estimate of x requires a Taylor-series or geometric-mean correction rather than plugging in.

Unbiasedness versus minimax. A single unbiased estimator says nothing about optimality. The minimax risk

$$\inf_{\hat{\theta}} \sup_{\theta \in \Theta} \mathbb{E}[(\hat{\theta} - \theta)^2]$$

benchmarks *all* estimators at once. For unbiased estimators the Cramér–Rao inequality $\text{Var}_{\theta}(\hat{\theta}) \geq 1/I(\theta)$, with $I(\theta) = \mathbb{E}[(\partial_{\theta} \log p_{\theta}(X))^2]$ the Fisher information, gives a local benchmark; two-point (Le Cam) and Fano arguments give distribution-free ones [?]. In DB papers the “information” is usually combinatorial rather than Fisherian: the sensitivity of a query under neighboring inputs, the norm of the stream, or the entropy of any encoding of the sketch. The last turns estimation error into a space lower bound: if the estimator must be (E, δ) -accurate, the sketch state must be incompressible below the entropy of what it has to distinguish [?].

The estimator-to-bound pipeline. Corpus papers instantiate the same five-step pipeline: (i) construct $\hat{\theta}$ from a sketch, a sample, or a randomized mechanism, and check its expectation; (ii) bound $\text{Var}(\hat{\theta})$ or a sub-Gaussian proxy in terms of structural parameters (stream norms, degrees, sensitivities); (iii) apply a concentration inequality to pass from variance to a per-quantity (ε, δ) statement; (iv) union-bound over all coordinates that must hold simultaneously, paying a $\log(\#\text{quantities})$ factor; (v) match with a lower bound (variance, Le Cam/Fano, or entropy) to certify optimality [? ? ?].



The estimator pipeline: every exemplar below walks left to right; the strongest also close the loop.

4.3 Exemplars from the SIGMOD'26 corpus

4.3.1 Perfect ℓ_p sampling from exponential variables (streaming)

One-pass streaming algorithms must output a single index i distributed with probability proportional to $|f_j|^p$, the ideal ℓ_p sample; the classical difficulty is that the stream must be tracked under this unusual distribution. The paper treats the output index itself as the estimator and bounds its accuracy per coordinate.

Theorem 4.1 — Theorem 1 of [?]: ℓ_p sampler. For any constant $p \in (0, 2]$, there is a one-pass streaming algorithm that runs in space $\tilde{O}(\log^{c(p)} n \cdot \log(1/\delta))$ and outputs an index i such that, with probability at least $1 - \delta$, for every $j \in [n]$,

$$\Pr[i = j] = \frac{|f_j|^p}{\|f\|_p^p} \pm \frac{1}{\text{poly}(n)}, \quad c(p) = \begin{cases} 1 & 0 < p < 2, \\ 2 & p = 2. \end{cases}$$

Corollary 4.2 — Corollary 3.3 of [?]. Let $f \in \mathbb{R}^n$ be a frequency vector and let $(e_1, \dots, e_n)$ be i.i.d. exponential random variables with rate 1. Define $z_i = f_i / e_i^{1/p}$ and let $(D^{(1)}, \dots, D^{(n)})$ be the anti-rank vector of the vector $(z_1^{-p}, \dots, z_n^{-p})$. Then

$$\Pr[D^{(1)} = i] = \frac{|f_i|^p}{\|f\|_p^p}.$$

What is estimated is a whole distribution rather than a scalar, and the error bound is an ℓ_∞ bound on the sampled probabilities. The estimator trick is the exponential race: by scaling, $e_i / f_i^p = z_i^{-p}$ is exponential with rate $|f_i|^p$, so the minimum is attained at i with exactly ℓ_p probability; the streaming algorithm then only has to track the rescaled vector z (via BPTree, CountSketch, and AMS sketches), repairing the nonlinear x^p step with Taylor-series and geometric-mean estimators, with Chebyshev and union bounds supplying the $\pm 1/\text{poly}(n)$ accuracy. [?] ^{PODS}

4.3.2 Unbiased Laplace-noised walk counting (local differential privacy on graphs)

Counting k -line walks in a graph where every node privatizes its own view (edge-local DP) forces fresh Laplace noise at each of k synchronous rounds, so the naive error compounds with k ; the question is how much accuracy a k -round message-passing mechanism can retain.

Theorem 4.3 — Theorem 4.2 of [?]. For any graph G , walk length k , and parameters ε and β , the k -round mechanism $\mathcal{M}_{k\text{-wk}}$ satisfies $\mathbb{E}[\mathcal{M}_{k\text{-wk}}(G, \varepsilon)] = W_k$, and with probability at least $1 - \beta$,

$$|\mathcal{M}_{k\text{-wk}}(G, \varepsilon) - W_k| \leq k\gamma\sqrt{N} (d(G) + \gamma)^{k-1} = \tilde{O}(\sqrt{N d(G)^{k-1}}),$$

where $\gamma = 2k\sqrt{8\log(2kN/\beta)}/\varepsilon = \tilde{O}(1)$, with N the number of nodes and $d(G)$ a degree parameter of G .

What is estimated is W_k , the number of k -line walks; the bound is unbiasedness plus a high-probability additive error of $\tilde{O}(\sqrt{Nd(G)^{k-1}})$, improving on the typical $O(N^{k/2})$ of naive Laplace counting. The estimator trick is sensitivity control by induction: each round adds per-node Laplace noise scaled by the running maximum $\|X^{(\ell-1)}\|_\infty$, a Laplace concentration bound (failure budget β'/N per node per round) yields $\|X^{(\ell)}\|_\infty \leq (d(G) + \gamma)^\ell$, and union bounds close the loop; a complementary variance lower bound of $\Omega(N^{k+1})$ for the baseline estimator, obtained by summing variances over all oriented k -line walks, shows the multi-round design is genuinely needed. [?]]

4.3.3 Variance recursion and an entropy lower bound for expandable sketches

A frequency-estimation (FE) sketch for streams of unknown, unbounded length must expand as the stream grows; the paper asks both how accurate such a sketch can be and how much memory the expansion forces, matching the two sides.

Lemma 4.4 — Variance of the sketch [?]. Like CountSketch, the sketch provides unbiased estimates. Moreover, letting $V(F)$ denote its estimation variance when using a size function $W(\cdot)$ and one array on a stream with squared ℓ_2 -norm $F = \sum_y (f(y))^2$, we have at the end of epoch i that

$$V(F_{i+1}) \leq \frac{1}{W_i} \cdot \sum_{j=0}^i 2^{i-j} \cdot (F_{j+1} - F_j),$$

where F_j denotes the squared ℓ_2 -norm of the stream at the start of epoch j .

Theorem 4.5 — Space lower bound [?]. Consider an FE sketch with only overestimation errors that processes growing streams with an error bound of $E(N)$ and a confidence of $1 - \delta$. Such a sketch must use at least

$$\frac{N}{E(N)} \cdot \left[\Omega\left(\log \frac{1}{\delta} + \log \log \frac{N}{E(N)}\right) - \Theta(1) \right]$$

bits of memory at some point in time while processing a stream.

What is estimated are per-key frequencies, unbiasedly as in CountSketch; the error bound is a variance recursion over geometrically spaced epochs, each past expansion contributing with weight 2^{i-j} , and Markov/Chebyshev convert it into the stated confidence. The complementary trick is the lower bound: an entropy-incompressibility argument (by reduction to filters) shows any sketch achieving error

$E(N)$ at confidence $1 - \delta$ must pay the displayed bits, and the constructed sketch lands within a small logarithmic factor of it — the estimation-theoretic ideal of a matched pair. [?]]

4.3.4 Concentration-chosen sample sizes for influence minimization

Blocking nodes to curb time-critical misinformation spread requires maximizing a benefit function $D(\cdot)$ that is $\#P$ -hard to evaluate and non-submodular, so greedy must run on sampled surrogates; the question is how many samples suffice for the greedy choice to provably transfer to the true objective.

Theorem 4.6 — Theorem 6.7 of [?]]. Given $0 \leq \varepsilon, \delta \leq 1$, CBFM returns B^* satisfying

$$\Pr [D(B^*) \geq \tau(1 - \frac{1}{e} - \varepsilon) \cdot D(B_o)] \geq 1 - 3\delta,$$

where B_o denotes an optimal solution.

What is estimated is $D(B)$ itself, by an unbiased estimator $\hat{D}(B)$ built from reverse-score sample sets (GRS/TRS) whose coverage terms are unbiased; the error bound is an end-to-end $(1 - 1/e - \varepsilon)$ -style approximation holding with probability $1 - 3\delta$, where τ reflects that greedy optimizes D through the submodular surrogates sandwiching it ($D_L \leq D \leq D_U$). The estimator trick is to derive the sample threshold analytically: concentration bounds fix the maximal sample size $\theta_{\max}$, and three union-bound events account for the 3δ . [?]]

4.3.5 How many random trees? (biharmonic distance)

Pairwise biharmonic distance requires quantities from the square of the Laplacian pseudoinverse, which no near-linear graph algorithm computes exactly; the paper reformulates the distance and estimates each term by averaging over randomly sampled spanning trees.

Theorem 4.7 — Theorem 6.3 of [?]]. If $\omega_t = d_v^2 \Delta^2 \log(2n/\delta)/(2\varepsilon^2)$, FastTree samples ω_t trees, and the corresponding estimators satisfy that for each node u ,

$$|x_a(u) - \tilde{x}_a(u)| \leq \varepsilon \quad \text{and} \quad |x_b(u) - \tilde{x}_b(u)| \leq \varepsilon.$$

What is estimated are the two terms x_a, x_b of the reformulated distance (a pivoted identity rewrites $\beta(s, t)$ via $\hat{L}_v^{-1}$ and a rank-structured correction), each by the average of ω_t unbiased tree-sampling estimators grounded in electrical-network/Kirchhoff identities; the error bound is per-node ε -accuracy from Chernoff–Hoeffding plus a union bound over all n nodes. The trick is visible in the sample count itself: it is exactly

the canonical Hoeffding size (range scale $d_v \Delta$, with d_v the pivot degree and Δ a graph parameter) squared, times $\log(2n/\delta)$, over $2\epsilon^2$ — the matrix inverse is replaced by an expectation over absorbing random walks. [?]

When to reach for estimation theory

- You report a number derived from a sketch, a sample, or a randomized mechanism, and must attach error bars or an (ϵ, δ) confidence to it.
- Unbiasedness is the workhorse: it makes variances add across keys, rounds, or nodes, so one variance bound plus Chebyshev/Chernoff delivers the guarantee.
- Nonlinearity breaks unbiasedness ($\mathbb{E}[g(X)] \neq g(\mathbb{E}[X])$); fix it with Taylor-series or geometric-mean corrections instead of ignoring the bias.
- To size a structure, invert the concentration bound: sample counts of the form $\omega \approx \text{range}^2 \log(n/\delta)/(2\epsilon^2)$ should fall out analytically, not be tuned.
- To certify optimality, pair the upper bound with a variance, Cramér–Rao, Le Cam/Fano, or entropy-incompressibility lower bound; matching within logarithmic factors is the gold standard.
- Remember the two hidden costs: union bounds charge $\log(\#\text{quantities})$, and minimax bounds rate the estimator class, not your implementation.

5

Adversarial Constructions

An upper bound is a promise; an impossibility result is the proof that no better promise exists. The second kind of theorem is not derived from the first — it is *built*. To show that no algorithm, language, or index in a model can do better, the author constructs the world in which every allowable procedure fails: a pair of instances that look identical, a distribution of inputs hiding a secret, a stream of updates engineered so that staying correct is provably expensive, a witness object so large that merely reporting it exceeds the budget. That engineered world is the *adversary*, and building it is a craft with a small number of recurring patterns. In the SIGMOD’26 corpus the tool is used by 21 tagged papers, and it owns the survey’s signature pairing: in 13 of them the same article proves an upper bound *and* the adversarial lower bound that matches it. The pairing is no accident — the upper bound tells you what the adversary must force; the adversary certifies that the upper bound is the end of the line.

This chapter presents the basic patterns and the standard exemplars first; Section 5.5 adds the advanced regimes (adversaries against computational indistinguishability and against structural properties) as extension reading.

5.0 The tool: adversarial constructions

The problem class is uniform across applications. There is a *model* $\mathcal{M}$ — an observation interface (queries into a graph, a stream seen once, a sketch, the syntax of an algebra), a class of permitted procedures, and a cost measure (queries, time, space, edges, update work) — and a task f with a required accuracy. The deliverable is a theorem of the form “no procedure in $\mathcal{M}$ achieves f within resource R ,” and its proof always exhibits an explicit counter-world: the adversary. What varies from paper to paper is only *what all procedures are forced to share* — an identical observation, an identical data-structure state, a bounded growth ceiling, or an output that is simply too large — and that invariant is the

LEARNING OBJECTIVES

- Recognize the four targets of an impossibility result: cost, expressivity, output size, update scope.
- Build the three plants: an instance pair, a hard distribution, a witness object.
- Match the pattern to the target; import adversaries via reductions when direct planting fails.
- Dissect a lower-bound section: model, family, indistinguishability, contradiction.

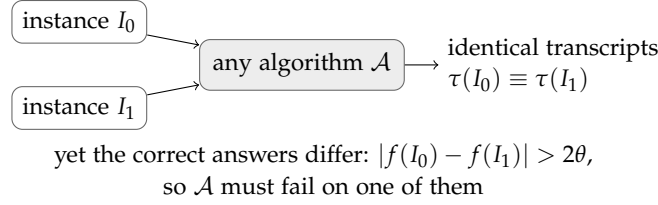


Figure 5.1: The two-instance pattern: the adversary plants instances that every procedure in the model must observe identically, while the demanded answers differ by more than the tolerance. Dynamic settings (Section 5.3) use the same picture with I_0, I_1 two futures of an update stream and τ the data structure’s state.

organizing question of this chapter.

Notation

“Adversary” always means a construction fixed *before* the algorithm’s coins: an instance pair, an input distribution, or an update sequence. A lower bound of $\Omega(g(n))$ is a statement about every procedure in the model; a *conditional* lower bound holds under a stated conjecture. Φ denotes an aspect ratio (largest over smallest nonzero distance), λ a doubling dimension, δ_G the degeneracy of a graph G .

5.0.1 The basic patterns

Every adversarial argument in the corpus is one of five patterns, stated here in their bare forms. The first is the atom from which the others are assembled.

Definition 5.1 (Observation model and transcripts). An *observation model* assigns to each instance I and each (possibly randomized) algorithm $\mathcal{A}$ the *transcript* $\tau_{\mathcal{A}}(I)$: the complete record of what $\mathcal{A}$ observes on I , including its own coins. Instances I_0, I_1 are *indistinguishable*, written $\tau(I_0) \equiv \tau(I_1)$, if for every procedure permitted by the model the distributions of $\tau_{\mathcal{A}}(I_0)$ and $\tau_{\mathcal{A}}(I_1)$ coincide.

Proposition 5.2 — Two-instance lower bound. Let f be a task solved with tolerance θ (the answer must fall within θ of $f(I)$). If there exist instances I_0, I_1 with $\tau(I_0) \equiv \tau(I_1)$ but $|f(I_0) - f(I_1)| > 2\theta$, then no algorithm in the model solves f . ■

Proof. On the shared observation, the algorithm’s output distribution is the same for both instances. Whatever value v it emits with positive probability, v cannot simultaneously lie within θ of $f(I_0)$ and within θ of $f(I_1)$; hence on one of the two instances the algorithm errs with positive probability. ■

1

Randomized algorithms do not escape the pattern — they route it through a distribution. *Yao’s minimax principle* [13] says that derandomizing the *input*, not the algorithm, loses nothing:

Theorem 5.3 — Yao’s minimax principle. For any cost function $\text{cost}(\mathcal{A}, x) \geq 0$,

$$\min_{\text{randomized } \mathcal{R}} \max_x \mathbb{E}[\text{cost}(\mathcal{R}, x)] \geq \max_{\text{distributions } \mathcal{D}} \min_{\text{deterministic } \mathcal{A}} \mathbb{E}_{x \sim \mathcal{D}}[\text{cost}(\mathcal{A}, x)].$$

Proof. Fix any $\mathcal{D}$ and any randomized $\mathcal{R}$. Averaging over the coins of $\mathcal{R}$ first, $\mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}[\text{cost}(\mathcal{R}, x)] = \mathbb{E}_{\text{coins}} \mathbb{E}_{x \sim \mathcal{D}}[\text{cost}(\mathcal{A}_{\text{coins}}, x)] \geq \min_{\mathcal{A}} \mathbb{E}_{x \sim \mathcal{D}}[\text{cost}(\mathcal{A}, x)]$. The left side is at most $\max_x \mathbb{E}[\text{cost}(\mathcal{R}, x)]$; minimize over $\mathcal{R}$, then maximize over $\mathcal{D}$. ■

So against a randomized algorithm one exhibits a *hard distribution* and analyzes the best deterministic procedure against it — which is why so many lower bounds in the corpus are phrased as “draw the input from Σ ; any estimator with T queries fails” (the pattern of Section 5.4).

The next two patterns need no planting at all: the adversary *is* an object.

Proposition 5.4 — Witness counting. If some instance I of size n has an output set $\text{Out}(I)$ with $|\text{Out}(I)| \geq g(n)$, and a correct algorithm must report every element of $\text{Out}(I)$, then every correct algorithm takes $\Omega(g(n))$ time. ■

Proof. The algorithm must write $g(n)$ distinct answers; writing is a lower bound on running time. ■

The classical witness is Moon–Moser’s [10]: a graph on n vertices with $3^{n/3}$ maximal cliques, each of which an enumerator must output. The pattern reappears with a defect budget in Section 5.1, where the witness is generalized to k -defective cliques and certifies an enumeration algorithm as worst-case optimal.

When the object under attack is a *language* rather than a cost, the shared invariant is a growth ceiling: everything expressible in the model is small in some measure, and the target operation is not.

Proposition 5.5 — Growth ceiling. Let $\mu \geq 0$ be a measure on instances such that every expression of the model with description length ℓ , applied to inputs of size n , produces outputs with $\mu \leq n^{O(\ell)} \cdot c^\ell$ for a constant c of the model. If the target operation T has, for every fixed ℓ , an input I with $\mu(T(I)) > n^{O(\ell)} \cdot c^\ell$ (for instance $\mu(T(I)) = 2^{|I|}$), then no expression of the model defines T . ■

Proof. Any candidate defining expression has some fixed length ℓ ; pick n large enough that the demanded μ -value exceeds its ceiling

¹ The tolerance in Proposition 5.2 is where approximation parameters enter: planting instances whose answers differ by just over 2θ is exactly how ε ’s in lower bounds are beaten — the gap must exceed the promised accuracy, not merely be nonzero.

$n^{O(\ell)}c^\ell$, which exists because 2^n outgrows every polynomial in n times a constant. The expression's output is then too small in μ to equal $T(I)$. ■

² The corpus instance: on $\mathbb{N}$ -databases, every expression of basic relational algebra produces multiplicities at most polynomial in the input size times a factor exponential only in the expression's own length, while division demands multiplicities 2^n — so division is inexpressible [2]. Expressivity counterexamples of this kind go back to Codd [3]. No pair of instances is needed; growth rates alone separate.

2

Finally, when no direct planting suffices, the adversary is *imported*: a reduction converts instances of a believed-hard problem into instances of yours.

Proposition 5.6 — Reduction transfer. Suppose instances of a problem H , conjectured to require $T_H(n)$ time, map in $o(T_H(n))$ time to instances of P of size at most $s(n)$. Then any algorithm for P running in $T(n)$ time with $T(s(n)) = o(T_H(n))$ refutes the conjecture; equivalently, P requires $\Omega(T_H(s^{-1}(n)))$ time under the conjecture. ■

Proof. Compose the reduction with the algorithm: an instance of H of size n becomes an instance of P of size $\leq s(n)$, solved in $T(s(n)) + o(T_H(n)) = o(T_H(n))$ time, contradicting the conjecture. ■

The conjecture is usually a fine-grained one — combinatorial k -clique, OuMv [4], SETH — and the reduction is itself an adaptive adversary: it manufactures the update stream, and indistinguishability is inherited from the source problem rather than planted directly. When the source is a communication problem instead, the two-instance pattern arrives prefabricated: index, set-disjointness, and gap-Hamming transcripts are information-blind by construction [8], the route by which sketching lower bounds entered databases with the frequency-moment bound [1].

5.0.2 How to choose the pattern

The selection is mechanical once two questions are answered. *First, what is the resource under attack?* If it is running time or query complexity, ask whether the model restricts observation (a sketch, a query interface, one pass over a stream): it does $\rightarrow$ plant two instances or a hard distribution; it does not (arbitrary computation allowed) $\rightarrow$ import hardness via a reduction, since nothing else stops an unrestricted algorithm. If the resource is the *output*, use a witness object. If it is *expressivity*, use a growth ceiling or a symmetry. If it is *update work* in a dynamic setting, the shared observation is the data structure's state and the instances are futures of the update stream. *Second, is the algorithm randomized?* Then Yao (Theorem 5.3) converts your task to exhibiting one hard distribution and beating the best deterministic procedure on it.

When to Build an Adversary

Reach whenever the deliverable is a no-better-is-possible statement — certifying an algorithm worst-case optimal, showing an index’s edge budget cannot shrink, separating a language fragment, proving an update operation cannot stay local — or when planning an upper bound and needing to know what the adversary will force. Then run the recipe:

1. Fix the model precisely: what procedures may observe, what they must output, the tolerance θ , the cost measure. Sloppy models make sloppy impossibilities.
2. Identify what *all* procedures must share: a transcript, a state, a growth ceiling, an output set. This shared object is where the adversary lives.
3. Plant the two instances (Proposition 5.2), a hard distribution (Theorem 5.3), a witness object (Proposition 5.4), or a ceiling-breaking instance (Proposition 5.5); if nothing plants, import via a reduction (Proposition 5.6).
4. Tune the parameters so the shared observation is exactly identical (not merely similar) while the demanded answers differ by more than 2θ ; verify no adaptive strategy can break the symmetry.
5. If every two-instance attempt fails, take the hint: the matching upper bound may simply exist. Try to prove it before fighting the lower bound further.

5.1 *Defective clique enumeration: the output as adversary*

The exemplar is a SIGMOD’26 paper on relaxed clique computation [5]. Exact cliques are famously brittle: one absent edge destroys one, so real communities — friendship groups, protein complexes, fraud rings — are better modeled by *defective* cliques, vertex sets that miss at most k edges for a small budget k . The paper studies enumerating all *maximal* k -defective cliques and searching for large ones, the way a graph engine would when k -hop tolerance matters. Its design question is classical branch-and-bound craft — how to pivot, how to prune — but its headline is a worst-case optimality guarantee: the search tree has at most $O(3^{n/3} \cdot n^k)$ nodes for constant k , and this is optimal because no algorithm can beat the size of the output it must produce. The adversary here is not a pair of instances at all: it is a single witness graph, generalized from Moon–Moser’s, whose maximal defective

cliques are so numerous that enumerating them dominates any analysis. The paper complements the witness with degeneracy-aware bounds that make the algorithm practical on sparse graphs.

Problem. Given a graph $G = (V, E)$ with n vertices and an integer $k \geq 0$, a vertex subset $S \subseteq V$ is a *k-defective clique* if $G[S]$ misses at most k edges, i.e. has at least $\binom{|S|}{2} - k$ edges. S is *maximal* if no superset is *k-defective*. The task is to enumerate all maximal *k-defective* cliques; the cost measure is the number of search-tree nodes (and total time) of any branching algorithm, judged against $\max_G |\text{Out}(G)|$, the worst-case output size.

3

³ Every *k-defective* clique of size q contains a clique of size $q - k$ — delete one endpoint of each missing edge. So the defective budget interpolates between clique enumeration ($k = 0$, Moon–Moser territory) and densest-subgraph-style relaxation.

Guarantee.

Theorem 5.7 — Worst-case optimal enumeration, combining the search-space and output-size theorems of the paper. For constant k , the worst-case output size of maximal *k-defective* clique enumeration is $\Omega(3^{n/3} \cdot n^k)$: there is a graph on n vertices with that many maximal *k-defective* cliques. The paper’s branching algorithm enumerates them all with a search tree of $O(3^{n/3} \cdot n^k)$ nodes in $O(m \cdot 3^{n/3} \cdot n^k)$ time, and its degeneracy-aware variant runs in $O(n \cdot 3^{\delta_G/3} \cdot \delta_G^{O(k)})$ time. ■

How the adversary is built. The witness is the Moon–Moser graph M_n : the complete $(n/3)$ -partite graph whose parts have size 3 (equivalently, the complement of $n/3$ disjoint triangles) [10]. The $k = 0$ case is the classical count — one vertex per part gives $3^{n/3}$ maximal cliques — and the defective budget k multiplies the harvest by a polynomial factor:

Lemma 5.8 — Witness count for defective cliques; output lemma of the paper, notation cleaned. For n a multiple of 3 with $n/3 \geq k$, the Moon–Moser graph M_n contains at least $3^{n/3} \cdot \binom{n/3}{k} = \Omega(3^{n/3} \cdot n^k)$ maximal *k-defective* cliques. ■

Proof sketch. Choose k of the $n/3$ parts; inside each chosen part keep *both* endpoints of one of its 3 vertex pairs (one missing edge), and in every unchosen part keep a single vertex. The resulting set misses exactly k edges — one per chosen part — and is maximal because the only addable vertices are the third vertices of chosen parts, each of which would create a second missing edge in its part. Counting the choices gives $\binom{n/3}{k} \cdot 3^k \cdot 3^{n/3-k}$ distinct maximal *k-defective* cliques. (Full proof in Section 5.6.) ■

Every maximal *k-defective* clique must appear in the enumeration, so Proposition 5.4 converts the witness count into the time lower bound

of Theorem 5.7 — and the algorithm’s $O(3^{n/3}n^k)$ search tree matches it, with no gap. The upper-bound side then does the delicate work: a pivoting rule for defective cliques (every maximal solution through a partial set must touch the non-neighborhood of a pivot) and a branching-vector induction showing the tree size never exceeds the output bound, which is where the n^k factor is earned rather than assumed.

What it buys. The engine gets a certificate that its enumeration cost is a property of the problem, not of the algorithm: no competitor can be asymptotically faster at worst case. The degeneracy-aware bound turns the same analysis into a practical promise — on sparse real graphs the exponent collapses from $n/3$ to $\delta_G/3$ — and the witness graph itself becomes the natural stress test for any implementation claiming output sensitivity.

5.2 Proximity graphs: point sets that force edges

The exemplar is a SIGMOD’26 paper on proximity graphs for similarity search [9]. Vector-database indexes increasingly build a *proximity graph* over the point set and answer nearest-neighbor queries by greedy navigation: from any start point, repeatedly hop to the out-neighbor closest to the query, and stop when no neighbor is closer. The paper asks the quantitative question behind every such index: how many *edges* must a graph carry so that greedy navigation always lands on a $(1 + \varepsilon)$ -approximate nearest neighbor? It answers both directions. Positively, in d -dimensional Euclidean space a $(1 + \varepsilon)$ -proximity graph with $O((1/\varepsilon)^\lambda \cdot n)$ edges — linear in n — exists and can be built in near-linear time with high probability, where λ is the doubling dimension. Negatively, in general doubling metrics no such sparsity is possible: there are point sets on which *any* $(1 + \varepsilon)$ -proximity graph, no matter how much query time it is allowed, must carry nearly $n \log \Phi$ edges, where Φ is the aspect ratio of the point set. The lower bound is the paper’s separation theorem: it shows the Euclidean assumption is not cosmetic — drop it and the edge budget necessarily blows up by a factor $\log \Phi$.

Problem. Let P be n points in a metric space with aspect ratio Φ (largest over smallest nonzero pairwise distance) and doubling dimension λ . A directed graph G on P is a $(1 + \varepsilon)$ -proximity graph $((1 + \varepsilon)$ -PG) if, for every query point q and every start vertex, greedy navigation — hop to the out-neighbor nearest to q — terminates at a $(1 + \varepsilon)$ -approximate nearest neighbor of q in P . The task is to bound the minimum number of edges of a $(1 + \varepsilon)$ -PG over P ; a “2-PG” is the case $(1 + \varepsilon) = 2$.

⁴ Doubling dimension λ : every ball of radius $2r$ is coverable by 2^λ balls of radius r ; it is the standard notion of intrinsic dimension for metric indexes. Navigability and $(1 + \varepsilon)$ -PG-hood are equivalent in the paper’s formulation, so the bound is about graphs, not about any particular search procedure.

Guarantee.

Theorem 5.9 — Size lower bound, Statement 1 of the paper’s lower-bound theorem, notation cleaned. For any powers of two $n \geq 2$ and Φ with $n^2 \leq \Phi \leq 2^n$, there is a set P of $\Theta(n)$ points with aspect ratio Φ , in a metric space of doubling dimension 1, such that any 2-PG for P must have $\Omega(n \log \Phi)$ edges, regardless of the query time allowed. ■

A companion statement extends the bound to a range of ε : with $n = s^d \cdot t$ points, doubling dimension at most $\log(1 + 2^d)$ and $\varepsilon = 1/(2s)$, any $(1 + \varepsilon)$ -PG needs $\Omega(s^d \cdot n)$ edges — again independent of query time.

How the adversary is built. The adversary is a planted point set in a tree-like metric — doubling dimension 1 is the regime where every ball splits into two half-radius balls, and the construction exploits exactly that binary hierarchy across the $\log \Phi$ distance scales spanned by the aspect ratio. The mechanism is *scale separation*: the two-instance pattern of Proposition 5.2 applied once per distance scale. The witness spans $\log \Phi$ scales, and a navigable graph must support greedy descent through every one of them: at each scale the construction places groups of points whose relevant distances sit at that scale, while an edge that could serve one scale cannot serve another because consecutive scales differ by more than the stretch factor. The necessity counting runs per scale and yields a linear edge count for each; summing over the $\log \Phi$ scales gives $\Omega(n \log \Phi)$ (the paper’s proof develops the counting in detail; we sketch the architecture). Two features make the bound robust. First, it is *non-geometric*: the witness metric is a hierarchy, not a point set in any $\mathbb{R}^d$, which is precisely why it does not contradict the Euclidean upper bound — the separation between general doubling metrics and Euclidean space is the theorem’s point. Second, the counted quantity is a property of the *object* (the edge set of any graph that is navigable at all), not of a procedure’s cost, which is what the “regardless of the query time allowed” clause buys: no amount of walk-length tolerance dodges an edge count. The generalization is honest work, as the paper notes: for ε -dependence (Statement 2), the hard input extends a classical query-time lower-bound witness for approximate nearest-neighbor data structures, but lifting a query-time witness to a *size* witness demands new ideas, since the adversary must now defeat graphs that may be arbitrarily dense in the wrong places.

What it buys. For index builders the theorem is a floor: on skewed-distance data (large aspect ratio), a proximity-graph index with near-linear edges simply does not exist in general metrics, so practitioners either exploit Euclidean structure or pay $\log \Phi$. The upper bound’s

linear-in- n Euclidean graph with near-linear construction closes the design space from above, and the pair of results together tells system designers exactly which assumption — doubling dimension plus geometry versus doubling dimension alone — their edge budget depends on.

5.3 *Bi-core maintenance: an update pair that lies in wait*

The exemplar is a SIGMOD’26 paper on maintaining bipartite cores [12]. Bipartite graphs model user–item, author–paper, and customer–product relations, and their *bi-cores* are the standard cohesion notion: the (α, β) -core is the largest subgraph in which every vertex on the left has degree at least β and every vertex on the right at least α . When edges arrive and depart, a system wants to maintain all bi-core memberships without recomputing from scratch. The paper’s first contribution is an asymmetry theorem explaining why this is hard in one direction only: under the model of *locally persistent* algorithms — the model prior maintenance algorithms implicitly operate in, where an update is processed by examining local structure around the affected edge — handling *deletions* is bounded (work proportional to what actually changes), while handling *insertions* is unbounded. Its second contribution is a family of order-based algorithms that exploit the asymmetry: a maintained vertex order with “lack values” gives necessary conditions pruning which vertices can possibly change, taming insertion in practice.

Problem. Let $G = (V_1, V_2, E)$ be a bipartite graph; for a pair (α, β) , the (α, β) -core is the maximal subgraph where every $v \in V_1$ has degree $\geq \beta$ and every $v \in V_2$ has degree $\geq \alpha$; each vertex belongs to a set of *c-pairs*. Under single-edge insertions and deletions, maintain all *c-pairs*. An update algorithm is *locally persistent* (LP) if its processing of an update is confined to the persistent local information around the affected vertices; it is *bounded* if its cost per update is a function of the true change set ΔR alone — in particular $O(1)$ on updates that change no core membership.

5

Guarantee.

Theorem 5.10 — Insertion–deletion asymmetry, Theorems 3.1–3.3 of the paper.

(i) Given a bipartite graph G and all its bi-cores, updating them after inserting an edge (u, v) is *unbounded* under the model of locally persistent algorithms: some updates force $\omega(1)$ work while the true change set is empty. (ii) Updating the bi-cores after deleting an edge (u, v)

⁵ The LP model abstracts what deployed maintenance algorithms actually do: identify candidate *c-pairs* near the edge, expand a candidate set, and update — never recomputing globally. “Unbounded” then means: there are updates where the algorithm must do $\omega(|\Delta R|)$ work, i.e. where local examination cannot certify that nothing changed.

is *bounded*, in time $O(|\text{CHANGED}| + |N_1(\text{CHANGED})|)$ for the natural deletion procedure. ■

How the adversary is built. The adversary is the two-instance pattern wearing dynamic clothes (Figure 5.1): the shared observation is the fully processed state of G , and the two instances are its two futures. The construction is distilled in the paper’s workhorse lemma.

Lemma 5.11 — The two-update trap, proof of Theorem 3.1 of the paper, notation cleaned. There is a bipartite graph G with n vertices and two single-edge insertions $\Delta G_1, \Delta G_2$ such that: (i) for each of ΔG_1 and ΔG_2 alone the true change set ΔR is empty; (ii) applied in sequence, $|\Delta R| = \Omega(n)$. Hence any LP algorithm that is bounded spends $O(1)$ on either update alone, yet any algorithm that maintains correctness must spend $\Omega(\ell)$ work across the two updates for a non-constant ℓ — a contradiction. ■

Proof sketch. Build G so that each insertion is individually harmless — neither moves any vertex across a core threshold by itself — but the two jointly push a cascade of $\Omega(n)$ vertices over their thresholds. A bounded LP algorithm must return in $O(1)$ time after the first update (since $\Delta R = \emptyset$ certifies there is nothing to do), leaving its local state unprepared; when the second update arrives, correctness forces the full cascade to be processed, which cannot be charged to either update’s change set. (The paper’s complete construction appears in its extended version.) ■

The trap is exactly Proposition 5.2 with τ = the algorithm’s internal state after processing G : the two futures ΔG_1 and ΔG_2 are locally indistinguishable (each looks like a no-op), so the algorithm must behave identically on both — but the demanded maintenance work differs, and one of the two futures punishes it.

What it buys. The theorem tells core-maintenance systems that insertion cannot be both local and worst-case bounded — explaining the observed bottleneck (long BFS expansions) in prior algorithms, which the paper measures at dominating proportions of update time. The fix it motivates is the order-based design: since boundedness is impossible in general, prune with necessary conditions (an update can cascade only along monotone paths in a maintained order, and only through vertices whose “lack values” hit zero), reducing practical insertion cost to $O(|N_1(\text{AFF})| \cdot |N_2(\text{AFF})|)$ over the actually affected sets.

5.4 Personalized PageRank: hard distributions with hidden splits

The exemplar is a PODS'26 paper on single-source personalized PageRank [6]^{PODS}. Personalized PageRank (PPR) asks where a random surfer who teleports back to a source s with probability α at every step will land: the score $\pi(s, t)$ powers recommendation, ranking, and graph mining over large graphs, and systems from web search to graph databases estimate it millions of times a day. The single-source task estimates the whole vector $\pi(s, \cdot)$ at once, with two standard accuracies — relative error for the coordinates above a threshold δ , or additive error ε — and cost measured in *queries*: the paper works in the arc-centric model, where the algorithm may ask for the k -th in- or out-edge of any node but cannot see the adjacency list wholesale. The paper is the corpus's pairing exhibit in its purest form: it gives an algorithm, DistWalks, answering relative-error queries in $O(\min\{\log n/\delta, m + n \log n \log(m\delta/n)\})$ expected queries — the first bound linear in m for dense graphs — and then proves matching lower bounds showing that, up to logarithmic factors, nothing better exists in this model. Both directions rest on the same adversary: a family of graphs whose useful information hides in a random classification that a sublinear number of queries cannot recover.

Problem. Fix a directed graph G (multigraphs allowed), source s , decay α . The PPR value $\pi(s, t)$ is the probability that an α -decay random walk from s (it stops at each step with probability α) ends at t . The SSPPR-R task: given error $c \in (0, \frac{1}{2})$, failure probability p_f , and threshold δ , output estimates $\hat{\pi}(s, \cdot)$ such that with probability at least $1 - p_f$, $|\hat{\pi}(s, t) - \pi(s, t)| \leq c \cdot \pi(s, t)$ for every t with $\pi(s, t) \geq \delta$. The algorithm accesses G through arc-centric queries (the k -th in- or out-edge of a named node); the cost is the expected number of queries. The lower bound may choose any node count n_0 , edge count $m_0 \in [n_0, O(n_0^2)]$, and threshold $\delta_0 \in (0, 1)$.

6

Guarantee.

Theorem 5.12 — Query lower bound for SSPPR-R, Theorem 1.6 of the paper, notation cleaned; upper bound Theorem 1.4 of the paper. For every n_0 , every $\delta_0 \in (0, 1)$ and every $m_0 \in \Omega(n_0) \cap O(n_0^2)$, any algorithm solving SSPPR-R with constant error c and constant failure probability p_f requires $\Omega(\min\{m_0, \log(1/\delta_0)/\delta_0\})$ expected queries on graphs with n_0 nodes and m_0 edges and threshold δ_0 . DistWalks answers in $O(\min\{\log n/\delta, m + n \log n \log(m\delta/n)\})$ expected queries — match-

⁶ DistWalks itself is a concentration showcase — Monte-Carlo discovery with Chernoff bounds and martingale error control — but its *optimality* is an adversarial construction; the two tool families meet in this paper, and Chapter 2 develops the former. Here we dissect the latter.

ing up to logarithmic factors. ■

Two sibling bounds complete the picture: additive-error estimation requires $\Omega(\min\{1/\varepsilon_0^2, m_0\})$ queries, and the single-target version $\Omega(\min\{m_0, n_0 \log n_0/\delta_0\})$ — the latter by reversing every edge of the hard family, since an arc-centric algorithm cannot tell a graph from its reverse.

How the adversary is applied. The construction runs Yao’s principle (Theorem 5.3) through a *hidden split*. The template graph $U(n, D, d)$ has a source s feeding a middle layer Y that points into a target layer X split into two equal classes X_1, X_2 : by the template, every node of X_1 receives D of its in-edges from the Y -layer, every node of X_2 only $D - d$. The PPR values separate exactly at the resolution the task demands — a calculation gives $\pi(s, x_1) = \frac{1-\alpha}{2} \cdot \frac{D}{(2D-d)n}$ and $\pi(s, x_2) = \frac{1-\alpha}{2} \cdot \frac{D-d}{(2D-d)n}$, a ratio $\frac{D-d}{D}$ — so an estimate within relative error c classifies every node of X : whoever can estimate PPR can recover the hidden split. Everything else about the graph is drawn uniformly at random: the distribution $\Sigma(n, D, d)$ puts a uniform prior on the split and uniform bijections on all half-edge connections, and an *r-padding* (extra edges into dummy node sets) lets the family be made dense without revealing edges, so the bound covers sparse and dense regimes alike. The argument then has two moves.

Move 1 (reduction): an SSPPR-R algorithm using $o(\min\{m_0, \log(1/\delta_0)/\delta_0\})$ queries yields, after post-processing its PPR estimates, a split-recovery procedure using at most γnD arc-centric queries with average success at least $1 - 2p_f$ (Lemma 4.4 of the paper; the parameters satisfy $n(r + D) = \Theta(\min\{m_0, \log(1/\delta_0)/\delta_0\})$, so sublinear in the budget means $o(nD)$).

Move 2 (indistinguishability): the split cannot be recovered from $o(nD)$ queries — the workhorse lemma.

Lemma 5.13 — Switched-split indistinguishability; Lemma 4.6 of the paper, notation cleaned. For a sufficiently small constant γ , after any $T \leq \gamma nD$ arc-centric query answers about $U \sim \Sigma(n, D, d)$ there are $\Omega(n^2)$ *switched splits* — splits obtained from the true one by trading the classes of one node of X_1 and one node of X_2 — whose likelihood given the answers is within a factor $n^{1-\varepsilon}$ (of a constant $\varepsilon > 0$) of the true split’s likelihood. Consequently every split has posterior probability less than $1/n$: no procedure outputs the true split with useful probability. ■

Proof sketch. Each revealed edge adjacent to a switched node shifts the conditional density by at most $1 + O(d/D)$ (the X_1 versus X_2 in-degree discrepancy); all other edges contribute an exact factor 1 by the symmetry of the uniform bijections. Few nodes have many revealed edges

(“traversal is hard”), so the product over a switched pair’s revealed edges is at most $(1 + O(d/D))^{O(\gamma D)} \leq e^{O(\gamma d)} \leq n^{1-\varepsilon}$ using $d \leq \log n$. With $\Omega(n^2)$ switched splits each nearly as likely as the truth and a uniform prior, Bayes crushes the posterior of any single split below $1/n$. (Full proof in Section 5.6.) ■

Moves 1 and 2 contradict for large n ($1 - 2p_f$ versus $1/n$), so the assumed fast SSPPR-R algorithm cannot exist. The two “hard” intuitions the paper names — *traversal is hard* (sublinear queries leave most nodes’ in-edges almost entirely unseen) and *random walk is hard* (what is unseen is uniformly random, so it carries no class information) — are exactly the two-instance pattern sharpened into a likelihood ratio.

What it buys. The pairing certifies DistWalks as the end of the line for PPR query complexity in the arc-centric model, up to logarithmic factors — a systems-relevant answer, since it tells engines that the observed cost of PPR service is intrinsic, not an artifact of any implementation. The construction pattern transfers broadly: any system that must estimate many values from partially revealed graph structure (adjacency-list services, crawl-based analysis, API-rate-limited settings) inherits the same hidden-classification adversary.

5.5 Advanced usage and further reading

Everything so far plants instances that are *exactly* indistinguishable, counts objects, or imports conjectures. Two advanced regimes relax the pattern in different directions: the indistinguishability becomes *computational* (assumed, not exact), and the adversary attacks a *property* of a design rather than a cost. Each is sketched through one corpus exemplar.

5.5.1 Computational adversaries: indistinguishability by assumption

Proposition 5.2 needs transcript distributions that coincide exactly. The cryptographic weakening asks only that no *efficient* procedure tell the instances apart — and then the two-instance pattern survives with efficient adversaries.

Proposition 5.14 — Computational two-instance lower bound. If instances I_0, I_1 are indistinguishable to all polynomial-time procedures (no efficient test separates them with non-negligible advantage) but $f(I_0)$ and $f(I_1)$ differ beyond the tolerance, then no efficient algorithm computes f : an efficient correct algorithm would itself be a distinguisher. ■

Proof. Run the supposed algorithm for f on both instances; its answers, which must be accurate, separate I_0 from I_1 — contradicting indistinguishability. ■

Undetectable DP backdoors [11]. The exemplar is a SIGMOD’26 security paper asking whether a malicious vendor can backdoor a differentially private system without anyone noticing. The *Gaussian pancake mechanism* (GPM) replaces the Gaussian mechanism’s isotropic noise by noise that is Gaussian except for periodic density spikes — “pancakes” — concentrated along a secret direction w : noise drawn from the hCLWE distribution. To any polynomial-time auditor without the key w , GPM’s outputs are indistinguishable from an honest Gaussian mechanism, because distinguishing hCLWE from Gaussian is hard under lattice assumptions (SIVP or GapSVP versus quantum computation); that is Proposition 5.14 with I_0, I_1 the two noise distributions, and the reduction to hCLWE is the imported adversary. Yet with the key, the mechanism is grossly non-private: privacy loss $\epsilon \in \Theta(1/\beta^2)$ — exceeding 10^4 in practical regimes — because neighboring databases shift the pancake alignment along w , and the density ratio between aligned and misaligned peaks is enormous. The paper is the two-instance pattern *weaponized*: the adversary constructs the indistinguishable pair and profits from the gap only it can see. Its defensive value is symmetric — knowing the construction tells auditors what assumptions they must break or refuse to trust.

5.5.2 Adversaries against properties, not costs

A lower bound certifies optimality; a *counterexample wreck* certifies that a design principle is unachievable. The smallest adversaries in the corpus are single small graphs that kill a hoped-for structural property of an objective — monotonicity, submodularity, the absence of free riders — and thereby redirect algorithm design. The exemplar is the SIGMOD’26 community-search paper LMSC [7]: its objective, local sketch modularity $\text{LSM}(G, C, \tau) = l_C / (o_C \cdot |C|^\tau)$ (internal over external weight with a size penalty), is shown by explicit clique constructions to be *non-monotone* and *non-submodular* — so no greedy algorithm carries approximation guarantees, and the paper’s chain-based relaxation is the response. The same paper’s NP-hardness (a Set-Cover reduction) and its free-rider threshold — LSM provably resists query-independent free riders once $\tau > \log_{1+1/h}(h(h-1)/2)$ for size bound h — show the two regimes coexisting: an imported adversary for the complexity statement, a witness graph of a handful of vertices for the property statements. When your target is a property rather than a cost, the recipe of Section 5.0.2 shrinks to: write the property as an inequality between

two small instances, and search for the smallest graph that violates it.

5.5.3 Further reading

- Yao [13] — the minimax principle turning randomized lower bounds into hard distributions; one page, endlessly reused.
- Kushilevitz and Nisan [8] — the two-player adversary catalog (index, set-disjointness, gap-Hamming) from which communication-complexity lower bounds are imported.
- Alon, Matias, and Szegedy [1] — where sketching lower bounds began; the two-instance pattern in stream form.
- Moon and Moser [10] — the $3^{n/3}$ witness and its descendants in enumeration optimality.
- Henzinger, Krinninger, Nanongkai, and Saranurak [4] — the OuMv conjecture and the reduction-transfer pattern for dynamic problems.
- Codd [3] — the original expressivity question; counterexamples measuring relational languages ever since.

Chapter Recap

- **What the tool is.** Adversarial constructions prove that no better promise exists: a counter-world (instance pair, hard distribution, witness object, ceiling-breaking input, or imported reduction) in which every procedure permitted by the model fails or pays.
- **Which pattern when.** Observation-restricted model → plant two instances with identical transcripts (Proposition 5.2); randomized algorithms → harden into a distribution via Yao (Theorem 5.3); cost is the output itself → witness counting (Proposition 5.4); target is expressivity → growth ceiling (Proposition 5.5); unrestricted computation → import a conjecture (Proposition 5.6); dynamic setting → two futures of an update stream sharing a data-structure state.
- **Why it works.** Everything the algorithm does is a function of what it observes; two worlds with identical observations but different demands break every algorithm at once. The craft is making “identical” exact — or computational (Proposition 5.14) — while the demanded answers differ beyond tolerance.
- **What the corpus showed.** Defective-clique enumeration (Section 5.1): a Moon–Moser witness generalized by a defect budget, certifying worst-case optimality. Proximity graphs (Section 5.2): a tree-like point set forcing $\Omega(n \log \Phi)$ edges regardless of query

time, separating Euclidean from general doubling metrics. Bi-core maintenance (Section 5.3): a two-update trap making insertion unbounded for locally persistent algorithms while deletion stays bounded. Personalized PageRank (Section 5.4): a hidden split under a uniform distribution, with likelihood ratios over switched splits matching the upper bound up to logarithms — the corpus’s signature upper/lower pairing.

- **Advanced usage.** Computational indistinguishability turns the two-instance pattern into an assumption-backed weapon (the DP backdoor of Section 5.5.1); counterexample wrecks attack properties of designs rather than costs (Section 5.5.2). Deferred proofs live in the Appendix (Section 5.6).

5.6 Appendix

Proofs omitted from the main text are collected here.

Proof of Lemma 5.8 (Moon–Moser witness count)

Statement (recalled). For n a multiple of 3 with $n/3 \geq k$, the Moon–Moser graph M_n (complete $(n/3)$ -partite graph, parts of size 3) contains at least $3^{n/3} \binom{n/3}{k} = \Omega(3^{n/3} n^k)$ maximal k -defective cliques. ■

Proof. Write $T_1, \dots, T_{n/3}$ for the parts of M_n : vertices in different parts are adjacent; the 3 pairs inside a part are non-edges. A set $S \subseteq V$ is k -defective exactly when at most k of the non-edges of M_n lie inside S . Construct S as follows: choose a set K of k parts ($\binom{n/3}{k}$ choices); in each part of K keep both endpoints of one of its 3 vertex pairs (3 choices each, the kept pair being one missing edge); in each of the remaining $n/3 - k$ parts keep one vertex (3 choices each). Then S contains exactly the k non-edges contributed by the parts of K and no others, so $G[S]$ misses exactly k edges: S is k -defective. It is maximal: the only vertices outside S are the third vertices of parts in K , and adding any such vertex v creates a second non-edge inside v ’s part (v is non-adjacent to both kept vertices of that part), so $S \cup \{v\}$ misses $k + 1$ edges and is not k -defective. Distinct choices yield distinct sets S (the parts of K are exactly those contributing two vertices), so the number of maximal k -defective cliques is at least

$$\binom{n/3}{k} \cdot 3^k \cdot 3^{n/3-k} = \binom{n/3}{k} \cdot 3^{n/3} = \Omega(3^{n/3} \cdot n^k),$$

where the last step uses $\binom{n/3}{k} = \Omega(n^k)$ for constant k . Since every maximal k -defective clique of M_n must be output, Proposition 5.4 converts the count into the enumeration time lower bound of Theorem 5.7. ■

Proof of Lemma 5.13 (switched-split bound)

Statement (recalled). For small enough constant γ , after any $T \leq \gamma n D$ arc-centric query answers about $U \sim \Sigma(n, D, d)$ there are $\Omega(n^2)$ switched splits with likelihood within a factor $n^{1-\varepsilon}$ of the true split's, and every split has posterior probability below $1/n$. ■

Proof. Work in the template family of Section 5.4: nodes of X_1 receive D in-edges from the Y -layer, nodes of X_2 receive $D - d$; the split and all half-edge bijections are uniform. Let $e_1, \dots, e_T$, $T \leq \gamma n D$, be the revealed edges and SP the true split. *Step 1 (traversal is hard).* Each revealed edge is incident to at most one node of X , so the T edges touch at most $\gamma n D$ in-edge slots of X ; by averaging, at least half the nodes of X are incident to at most $2\gamma D$ revealed edges, and because the split is uniform and independent of what queries reveal, both X_1 and X_2 contribute $\Omega(n)$ such low-incidence nodes: fix $X'_1 \subseteq X_1$, $X'_2 \subseteq X_2$ with $|X'_1|, |X'_2| \geq cn$ and every $x \in X'_1 \cup X'_2$ incident to at most $2\gamma D$ revealed edges. *Step 2 (likelihood ratio per switched pair).* Fix $x_{i_1} \in X'_1$, $x_{i_2} \in X'_2$ and let SP' be the switched split. By Bayes with a uniform prior and the chain rule,

$$\frac{\Pr[e_1, \dots, e_T \mid SP]}{\Pr[e_1, \dots, e_T \mid SP']} = \prod_{t=1}^T \frac{\Pr[e_t \mid SP, e_1, \dots, e_{t-1}]}{\Pr[e_t \mid SP', e_1, \dots, e_{t-1}]}.$$

For a revealed edge e_t not adjacent to x_{i_1} or x_{i_2} , the conditional laws under the two splits coincide — the uniform bijections are symmetric in the classes — and the factor is exactly 1. For an edge adjacent to a switched node, the discrepancy between the X_1 in-degree D and the X_2 in-degree $D - d$ changes the conditional density of seeing that particular edge by at most $\frac{D}{D-d} = 1 + O(d/D)$. At most $2 \cdot 2\gamma D$ factors differ from 1, so

$$\frac{\Pr[e_1, \dots, e_T \mid SP]}{\Pr[e_1, \dots, e_T \mid SP']} \leq \left(1 + O\left(\frac{d}{D}\right)\right)^{O(\gamma D)} \leq e^{O(\gamma d)} \leq n^{1-\varepsilon},$$

using $d \leq \log n$ and choosing γ a sufficiently small constant (with $\varepsilon \in (0, 1)$ a constant not depending on n). *Step 3 (posterior crash).* The $|X'_1||X'_2| = \Omega(n^2)$ switched splits are distinct and uniform under the prior, so Bayes gives

$$\Pr[SP \mid e_1, \dots, e_T] = \frac{\Pr[e_1, \dots, e_T \mid SP]}{\sum_{SP''} \Pr[e_1, \dots, e_T \mid SP'']} \leq \frac{\Pr[e_1, \dots, e_T \mid SP]}{\Omega(n^2) \cdot \Pr[e_1, \dots, e_T \mid SP] / n^{1-\varepsilon}} < \frac{1}{n}.$$

The same bound applies to every candidate split — each is the “true” split of some draw, or dominates an analogous family of switched alternatives — so any decoder’s success probability is below $1/n$. (Constants follow the paper’s appendix; the multigraph templates are lifted to simple graphs by the paper’s 2-lift, which the arc-centric interface cannot distinguish.) ■

References

- [1] Noga Alon, Yossi Matias, and Mario Szegedy. The space complexity of approximating the frequency moments. *Journal of Computer and System Sciences*, 58(1):137–147, 1999.
- [2] Guillermo Badía, Phokion G. Kolaitis, and Carles Noguera. Codd’s theorem for databases over semirings. *Proceedings of the ACM on Management of Data*, 3, 2025. DOI: 10.1145/3767713. SIGMOD 2026 (rounds 1-2).
- [3] Edgar F. Codd. Relational completeness of data base sublanguages. In *Database Systems (Courant Computer Science Symposia 6)*, pages 65–98. Prentice-Hall, Englewood Cliffs, NJ, 1972.
- [4] Monika Henzinger, Sebastian Krinninger, Danupon Nanongkai, and Thatchaphol Saranurak. Unifying and strengthening hardness for dynamic problems via the online matrix-vector multiplication conjecture. In *Proc. 47th ACM Symposium on the Theory of Computing (STOC)*, pages 21–30, 2015.
- [5] Jihoon Jang, Yehyun Nam, Kunsoo Park, and Hyunjoon Kim. Efficient defective clique enumeration and search with worst-case optimal search space. *Proceedings of the ACM on Management of Data*, 3, 2025. DOI: 10.1145/3769787. SIGMOD 2026 (rounds 1-2).
- [6] Xinpeng Jiang, Li Liu, Siqiang Luo, and Xiaokui Xiao. Near-optimality for single-source personalized pagerank. *Proceedings of the ACM on Management of Data*, 4, 2026. DOI: 10.1145/3801906. SIGMOD 2026.
- [7] Dahee Kim, Taejoon Han, Kaiyu Feng, Junghoon Kim, and Susik Yoon. Lmsc: Local sketch modularity optimisation for size-constrained community search in networks. *Proceedings of the ACM on Management of Data*, 4, 2026. DOI: 10.1145/3786666. SIGMOD 2026.
- [8] Eyal Kushilevitz and Noam Nisan. *Communication Complexity*. Cambridge University Press, 1997.
- [9] Shangqi Lu and Yufei Tao. Proximity graphs for similarity search: Fast construction, lower bounds, and euclidean separation. *Proceedings of the ACM on Management of Data*, 3, 2025. DOI: 10.1145/3767716. SIGMOD 2026 (rounds 1-2).
- [10] J. W. Moon and L. Moser. On cliques in graphs. *Israel Journal of Mathematics*, 3:23–28, 1965.

- [11] Haochen Sun and Xi He. Gpm: The gaussian pancake mechanism for planting undetectable backdoors in differential privacy. *Proceedings of the ACM on Management of Data*, 4, 2026. DOI: 10.1145/3802068. SIGMOD 2026.
- [12] Qiaoyuan Yang, Wensheng Luo, Yixiang Fang, and Yuanyuan Zeng. Order-based algorithms for efficient core maintenance in large bipartite graphs. *Proceedings of the ACM on Management of Data*, 4, 2026. DOI: 10.1145/3786674. SIGMOD 2026.
- [13] Andrew Chi-Chih Yao. Probabilistic computations: Toward a unified measure of complexity. In *Proc. 18th IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 222–227, 1977.

6

Differential Privacy Composition

Among the 26 SIGMOD'26 papers tagged `privacy-dp`, composition accounting is close to a monoculture: 69% of them assemble their end-to-end privacy guarantee by composing mechanisms, and the tool `dp-composition` is used by 18 of the 228 tagged papers overall, its strongest pairing being `privacy-guarantee` $\times$ `dp-composition` (16 papers). The arithmetic itself is shifting: the exemplars below still mostly add plain (ϵ, δ) budgets, but the frontier results state composition for Rényi DP and f -DP, where the composed object is a whole divergence curve rather than a pair of scalars.

6.1 The tool

Definition 6.1 (Differential privacy). A randomized mechanism $\mathcal{M} : \mathcal{D} \rightarrow \mathcal{R}$ satisfies (ϵ, δ) -differential privacy if for all neighboring datasets $D \sim D'$ (differing in one record, edge, or node) and all measurable $S \subseteq \mathcal{R}$,

$$\Pr[\mathcal{M}(D) \in S] \leq e^\epsilon \cdot \Pr[\mathcal{M}(D') \in S] + \delta.$$

With $\delta = 0$ the guarantee is *pure* ϵ -DP [?].

Five primitives, one line each:

- **Sequential composition.** An adaptive sequence of releases with budgets (ϵ_i, δ_i) satisfies $(\sum_i \epsilon_i, \sum_i \delta_i)$ -DP: budgets add [? ?].
- **Post-processing immunity.** Any function of an (ϵ, δ) -DP output that does not touch the raw data is itself (ϵ, δ) -DP: analysis after the release is free [?].
- **Group privacy.** Datasets at distance λ inherit $(\lambda\epsilon, \lambda\delta)$ -DP: the guarantee degrades linearly in group size [?].
- **The Rényi ladder.** The Gaussian mechanism (noise $\mathcal{N}(0, \sigma^2 I)$, ℓ_2 -sensitivity Δ_2) is $(\alpha, \alpha\Delta_2^2/(2\sigma^2))$ -RDP for every $\alpha > 1$; RDP composes by adding the $\rho(\alpha)$; and (α, ρ) -RDP converts to $(\rho(\alpha) +$

$\log(1/\delta)/(\alpha - 1)$, δ)-DP, optimized over α — the engine behind $O(\epsilon\sqrt{k})$ rather than $k\epsilon$ for k releases [? ?]. f -DP makes the divergence curve itself the composed object [?].

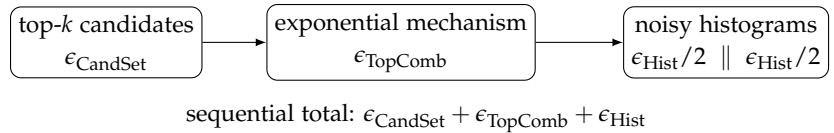
- **Amplification by subsampling.** An ϵ -DP mechanism run on a p -rate Poisson subsample satisfies $\log(1 + p(e^\epsilon - 1))$ -DP: smaller samples buy smaller budgets [?].

6.2 Exemplars from SIGMOD'26

Budget ledgers for private cluster explanations. The paper privately explains a given clustering by selecting the attribute combination that best accounts for it: top- k candidate selection, then an exponential mechanism over combinations, then per-cluster histograms [?].

Proposition 6.2 — Sequential composition. Let $\mathcal{M}_1 : \mathcal{D} \rightarrow \mathcal{R}_1$ and $\mathcal{M}_2 : \mathcal{D} \times \mathcal{R}_1 \rightarrow \mathcal{R}_2$. Suppose $\mathcal{M}_1$ satisfies ϵ_1 -DP and, for every $y \in \mathcal{R}_1$, $\mathcal{M}_2(\cdot, y)$ satisfies ϵ_2 -DP (as a function of its first input). Define $\mathcal{M}_3 : \mathcal{D} \rightarrow \mathcal{R}_2$ by $\mathcal{M}_3(D) = \mathcal{M}_2(D, \mathcal{M}_1(D))$. Then $\mathcal{M}_3$ satisfies $(\epsilon_1 + \epsilon_2)$ -DP.

The end-to-end guarantee is the budget sum $(\epsilon_{\text{CandSet}} + \epsilon_{\text{TopComb}} + \epsilon_{\text{Hist}})$ -DP, while inside the histogram stage the clusters are disjoint, so per-attribute and per-cluster releases compose *in parallel*, each contributing $\epsilon_{\text{Hist}}/2$ — pure multiplicative e^ϵ arithmetic, no divergence needed.



Node-DP from edge-DP via group privacy. N2E converts any edge-DP graph mechanism into a node-DP one by clipping the input graph to a degree bound Δ^* that is itself privately approximated, then running the edge mechanism on the clipped graph [?].

Lemma 6.3 — Group privacy. If a mechanism $\mathcal{M}$ satisfies (ϵ, δ) -differential privacy, then for any two graphs G, G' with $d(G, G') = \lambda$ (i.e., differing by λ edges or nodes) and any subset of outputs $S \subseteq \mathcal{R}$,

$$\Pr[\mathcal{M}(G) \in S] \leq e^{\lambda\epsilon} \cdot \Pr[\mathcal{M}(G') \in S] + \lambda\delta.$$

The mechanism satisfies $(\lambda\epsilon, \lambda\delta)$ -DP for the group of λ edges or nodes.

The composition primitive doing the lifting is group privacy: a one-node change is a group of λ edge changes, so the edge-DP calculus transfers to node-DP neighborhoods. The degree search spends sparse-vector-technique queries at a logarithmic scale, so only logarithmically

many sequential compositions are consumed before post-processing turns the noisy degree into the clipping Δ^* ; the whole framework satisfies (ϵ, δ) -node-DP [?].

Per-record budgets: composition of functions, not scalars. Per-record DP replaces the global ϵ by a budget function $\epsilon_{\mathcal{E}}(r)$ (a bank may assign wealthier clients smaller budgets), and estimates sums/counts by partitioning records into geometrically doubling budget slices, one Laplace mechanism per slice [?].

Lemma 6.4 — PrDP parallel composition. Let $R_1, \dots, R_k$ be disjoint subdomains and $\mathcal{M}_1, \dots, \mathcal{M}_k$ a series of $\mathcal{E}_i$ -PrDP mechanisms. Their parallel composition $\mathcal{M}(D) := (\mathcal{M}_1(D_{R_1}), \dots, \mathcal{M}_k(D_{R_k}))$ satisfies $\mathcal{E}'$ -PrDP, where $\mathcal{E}'(r) = \max\{\mathcal{E}_i(r)\}_{i=1}^k$ for all $r \in [U]^d$.

Here the composed object is a budget *function*: releases on disjoint subdomains take a pointwise maximum instead of a sum, so a record is only ever charged the largest budget of any slice touching it. Near-optimality of the resulting error is certified against down-neighborhood lower bounds [?].

Concurrent composition for continual mechanisms. Continual mechanisms answer streams of queries; the hard question is what composition survives an adversary who adaptively and *concurrently* interleaves access to many such mechanisms [?].

Theorem 6.5 — Concurrent composition of continual mechanisms, informal. The concurrent composition of continual mechanisms $\mathcal{M}_i$ that are (ϵ_i, δ_i) -DP with respect to a verification function f_i is (ϵ, δ) -DP for the same (ϵ, δ) that holds for composing noninteractive mechanisms that are (ϵ_i, δ_i) -DP. This allows the adversary to adaptively and concurrently choose the neighboring sequences fed to each $\mathcal{M}_i$ based on the responses received from the other mechanisms.

What carries the guarantee is a simulation: every (ϵ, δ) -DP interactive mechanism can be realized by interactive post-processing of randomized-response outputs, so the divergence arithmetic of ordinary composition transfers verbatim to concurrent adversaries. The same paper pushes the frontier from (ϵ, δ) to f -DP and Rényi DP, complete with a counterexample showing where intuition breaks — the direction in which the currency is moving. [?]^{PODS}

Composing obliviousness in a multi-way join. The differentially oblivious join evaluates queries in untrusted memory while leaking only the input size $|I|$ plus a DP-noisy upper bound $\widehat{OUT}$ on the join size [?].

Theorem 6.6 — Meta-join. Algorithm 1 is $(\epsilon_1 + \epsilon_2, \delta_1 + \delta_2)$ -DO.

Two primitives carry it: an (ϵ_1, δ_1) -DO subroutine releases the (ϵ_2, δ_2) -DP bound $\widehat{OUT}$, and the advised join's access pattern is a deterministic function of $|I|$ and $\widehat{OUT}$ only. Sequential composition adds both coordinates, and post-processing immunity is precisely what makes an *access pattern* a legitimate object to compose [?].

When to reach for DP composition

Reach for it whenever a system releases *several* interacting statistics and the privacy proof is a ledger: candidate selections, noisy summaries, clipped releases. Then budgets add (exemplars 1 and 5 above, with δ 's summed too in the approximate case), disjoint data slices switch the arithmetic from sums to maxima (exemplars 1 and 3), and group privacy reduces an expensive neighborhood (node-DP) to a cheap one (edge-DP) at linear cost in group size (exemplar 2). Reach for divergence arithmetic — RDP or f -DP curves — when many releases would make summed ϵ 's vacuous, buying $O(\epsilon\sqrt{k})$ over $k\epsilon$ (the frontier of exemplar 4); reach for subsampling amplification when releases are computed on samples anyway. Post-processing immunity ships every downstream artifact — clipped graphs, access patterns, indices — for free. Skip the machinery when a single calibrated release suffices, or when every output is a post-processing of one DP output: composing there only overcounts.

7

Exchange and Greedy Arguments

Few tools rose as sharply through our SIGMOD’26 corpus as this one: exchange/greedy arguments tagged 4 papers in the arXiv-visible cohort and jumped to 17 once the ACM-only mass arrived—one of the largest risers in the survey. The reason is a uniform paper shape: prove your selection problem NP-hard by reduction, then buy a polynomial-time algorithm back with a provable approximation factor. In our tag co-occurrences the tool pairs with hardness 10 times and with an approximation-ratio property 9 times—the signature of “NP-hard, but structured.” When the structure is submodularity, a covering constraint, or a metric, the greedy (or local-search) algorithm comes with a ready-made certificate, and the paper’s theory section shrinks to checking the hypothesis.

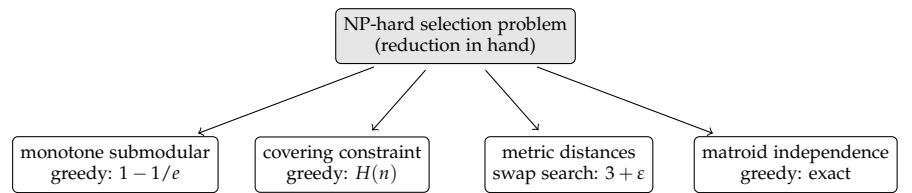
7.1 The Tool: Exchange Arguments and Their Guarantees

An exchange argument certifies a greedy choice by showing that any solution avoiding it can be *transformed* into one that makes it, at no loss. In a matroid, independent sets admit pairwise exchanges (if S is independent and smaller than independent S' , some $x \in S' \setminus S$ keeps $S \cup \{x\}$ independent), so greedy is *exact* for any linear objective [?]. Outside matroids exactness dies, but three structures keep a quantitative exchange certificate alive.

Definition 7.1 (submodularity). $f : 2^U \rightarrow \mathbb{R}_{\geq 0}$ is *submodular* if $f(S \cup \{x\}) - f(S) \geq f(T \cup \{x\}) - f(T)$ whenever $S \subseteq T$ and $x \in U \setminus T$; it is *monotone* if $S \subseteq T \Rightarrow f(S) \leq f(T)$.

- **Monotone submodular, cardinality k .** Diminishing returns let greedy’s next pick match the average marginal value of the optimum’s remaining elements; k geometric decay steps compound to $f(S_{\text{gre}}) \geq (1 - 1/e) \cdot \max_{|B|=k} f(B)$, and $1 - 1/e$ is tight unless $P = NP$ [?].

- **Submodular without monotonicity.** Randomized greedy variants retain a $1/2$ guarantee (exactly, via double greedy in the unconstrained case) and degrade gracefully under cardinality constraints [?].
- **Covering constraints (set cover).** To cover a universe at minimum weight, charge each newly covered element the amortized cost of the set covering it; the averaging step against an optimal cover yields the harmonic factor $H(n) = \sum_{i=1}^n 1/i$ [?].
- **Metric structure.** With triangle-inequality distances, exchanging one center for another (local search) reassigns points through two triangle inequalities: single swaps give $(3 + \epsilon)$ for k -median, multi-swaps $(2 + \epsilon)$ [?].



Which structure earns which guarantee; non-monotone objectives fall to randomized variants at $1/2$.

7.2 Five Views from the Corpus

7.2.1 Monotone Submodular Diversity: Greedy at $1 - 1/e$

Selecting the k most *diverse* answers to a fixed conjunctive query, where diversity is defined through a volume assignment Ω over a universe U of possible solutions.

Theorem 7.2 — volume-based diversity. Let Ω be any volume assignment over a universe U of possible solutions. Then δ_Ω is always monotone and submodular. Since Q is fixed and $Q(D)$ is computable in polynomial time over D , the greedy strategy that starts from $S = \emptyset$ and in each iteration adds the tuple $t \in Q(D)$ maximizing the marginal diversity $\delta_\Omega(S \cup \{t\})$ returns a k -subset S with

$$\delta_\Omega(S) \geq \left(1 - \frac{1}{e}\right) \cdot \max_{B \subseteq Q(D), |B|=k} \delta_\Omega(B).$$

The factor is earned by pure Nemhauser–Wolsey–Fisher structure: a measure-algebra argument shows every volume assignment makes δ_Ω monotone and submodular, and greedy inherits $1 - 1/e$ —which is tight (maximum coverage is hard beyond $1 - 1/e$ unless $P = NP$). The contrast is instructive: for the paper’s distance-based analogues under Hamming or Euclidean metrics the best known ratios are strictly worse and degrade with ϵ or k , while every polynomial-time-computable volume assignment gets the clean factor [?].

7.2.2 NP-Hard Representative Sampling: Lazy Greedy at $1 - 1/e$

Choosing a representative subset of large-scale vector data under a budget: the paper first proves its Problem 1 (Representative Sampling) NP-hard, then turns the budgeted objective into submodular form.

Lemma 7.3 — submodularity of the sampling objective. Let F be the sampling objective, built from per-point gains g_s that are each nondecreasing and concave. Then $F : 2^U \rightarrow \mathbb{R}_{\geq 0}$ is nondecreasing and submodular.

Theorem 7.4 — greedy guarantee. Let B be the budget and let A_{gre} be the size- B output of (lazy-)greedy. Then

$$F(A_{\text{gre}}) \geq \left(1 - \frac{1}{e}\right) \max_{|A| \leq B} F(A).$$

Concavity of every per-point gain g_s is the whole proof: sums of concave lifts of coverage are submodular, and the classical bound then prices the greedy output at $1 - 1/e$ —a one-page certificate for a problem the reduction just declared hopeless exactly [?].

7.2.3 Greedy Risk Coverage: A Set-Cover-Sized Coreset

Robust query optimization: given a workload distribution W and plan space Π , select a small set of plans whose risk *covers* the workload, forming a coreset that supports local plan inference.

Lemma 7.5 — greedy termination. Assuming the concentration conditions of the sampling lemmas hold in every round, the greedy cover terminates with $k_r \leq 2\kappa^*$.

Theorem 7.6 — risk coreset via greedy cover. Let W be a workload distribution. Given plans Π with $|\Pi| = N$ and parameters $\varepsilon \in (0, 1]$, $\tau \geq 1$, $\delta \in (0, 1)$, let κ^* denote the size of the smallest $(\varepsilon, \tau, \delta|W|)$ -coreset for risk. Fix any constants $c_1, c_2 > 1$. There exists an algorithm that samples $O((\kappa^*/\delta^2) \ln N \ln(\delta^{-1}))$ queries from W and computes, in time $O((\kappa^*/(\varepsilon^2 \delta^2)) \cdot N \ln^3 N \ln(\delta^{-1}))$, a set $C \subseteq \Pi$ of size $O(\kappa^* \ln \delta^{-1})$ such that, with high probability, C is a $(c_1 \varepsilon, \tau, c_2 \delta|W|)$ -coreset for risk.

This is the set-cover pattern in the wild: risk coverage is submodular, each greedy round shrinks the residual failure probability geometrically (the termination lemma caps rounds at twice the optimum), and the price of coverage is the familiar logarithmic slack $\ln(\delta^{-1})$ in the coreset size. Hoeffding and union bounds merely ensure the greedy cover runs on faithful proxy risks rather than true ones [?]^{PODS}.

7.2.4 Kruskal Edge-Exchange: Exact Indexing for Temporal Graphs

Answering cohesive-subgraph queries (connected components, k -core) over *all* time windows of a temporal graph, from one spanning-forest index built Kruskal-style with amortized union-find.

Lemma 7.7 — affiliated-forest invariant. Given a spanning edge $e = (u, v, t_1, t_2)$, let $\tau(e)$ be the timestamp such that $e \in F_{t_1, \tau(e)}$ and $e \notin F_{t_1, \tau(e)+1}$. An affiliated forest $(V, F_{x,y})$ built by BuildAF contains e if and only if $0 \leq x \leq t_1$ and $t_2 \leq y \leq \tau(e)$.

Here the exchange is exact, not approximate: graphic-matroid cycles let the construction swap an arriving edge for an expiring one while keeping a forest valid for every window simultaneously—Lemma above is the invariant those exchanges preserve. The certificate is completeness, not a factor: an $O(m \log m)$ construction and $O(m)$ -space index answer any window exactly, with queries in $O(n + \log^2 m)$ [?].

7.2.5 Swap Exchanges for Symmetry Breaking in Exact Search

Exact branch-and-bound for maximum common induced subgraph (MCIS), NP-hard, with branches pruned using vertex and value symmetries.

Theorem 7.8 — soundness of symmetry breaking. Variable symmetry breaking does not violate the correctness or completeness of the MCIS solution.

The proof is a pure swap: exchanging the conflicting pairs $(v_1, u_2), (v_2, u_1)$ for $(v_1, u_1), (v_2, u_2)$ maps any pruned branch to a symmetric one considered *earlier* in the fixed branching order, so some optimal mapping always survives the pruning. The exchange partner is supplied by automorphisms—two vertices are modular-symmetric exactly when their transposition is an automorphism of the graph—turning group symmetry into a completeness certificate for exact search [?].

When to reach for exchange-greedy

- Your selection, covering, or clustering problem is proved NP-hard by reduction, and the objective has *checkable* structure: diminishing returns (submodularity), a covering constraint, metric distances, or matroid independence.
- You want the algorithm's quality to be a theorem, not a benchmark: the exchange proof is usually a page, and the factor $(1 - 1/e, H(n), 3 + \epsilon, 1/2, \text{ or exact})$ is inherited, not re-derived.
- Audit the hypotheses, not the algorithm: pairwise marginal

tests or a concave-gain decomposition for submodularity; the triangle inequality for metrics; the exchange axiom for matroids. Near-modular or noisy objectives void the certificate.

- $1 - 1/e$ needs *monotone* submodular; without monotonicity fall back to randomized variants ($1/2$ unconstrained).
- The same idea certifies *exactness*: forest exchanges and symmetry swaps prove that pruning or one-pass construction loses nothing—completeness with no approximation factor at all.

8

Spectral and Matrix Tools

Why do eigenvalues keep appearing in database papers? Because the field's core objects are linear-algebraic in disguise: similarity and embedding quality are spectral properties of affinity and covariance matrices; the structured inverses arising from random walks, kernels, and fixed-point equations exist—and can be approximated—precisely when their spectra stay clear of zero; iterative processes converge at rates set by spectral radii and eigengaps; and low-rank summaries are optimal because of singular values. In this corpus, 13 papers reach for this machinery; after a compact toolbox we see three exemplars using it three different ways.

8.1 The tool: workhorse results

Throughout, A is symmetric with eigenvalues $\lambda_1 \geq \dots \geq \lambda_n$ and singular values $\sigma_1 \geq \sigma_2 \geq \dots$; all statements are classical (see [? ?]; for graphs specifically [?]).

Proposition 8.1 — Courant–Fischer.

$$\lambda_k(A) = \min_{\dim V = n-k+1} \max_{0 \neq x \in V} \frac{x^\top A x}{x^\top x} = \max_{\dim V = k} \min_{0 \neq x \in V} \frac{x^\top A x}{x^\top x}.$$

Hence if $B \in \mathbb{R}^{n \times s}$ has orthonormal columns, the eigenvalues μ_i of the compression $B^\top A B$ interlace: $\lambda_i(A) \geq \mu_i \geq \lambda_{n-s+i}(A)$ (Poincaré separation).

Lemma 8.2 — Weyl; Davis–Kahan $\sin \Theta$. $|\lambda_k(A + E) - \lambda_k(A)| \leq \|E\|_2$ for symmetric E ; and an invariant subspace U of A separated from the rest of the spectrum by a gap Δ moves to $\tilde{U}$ of $A + E$ by at most $\|\sin \Theta(U, \tilde{U})\|_2 \leq \|E\|_2 / \Delta$: subspace stability is perturbation norm over eigengap.

Theorem 8.3 — matrix Bernstein [?]. For independent, centered, symmetric

$d \times d$ matrices Y_i with $\|Y_i\|_2 \leq R$ surely and $\sigma^2 = \|\sum_i \mathbb{E} Y_i^2\|_2$,

$$\Pr\left[\left\|\sum_i Y_i\right\|_2 \geq t\right] \leq 2d \exp(-t^2/(2(\sigma^2 + Rt/3))).$$

This is the bridge to the concentration chapter: sums of independent matrices concentrate like sums of scalars, paying only a factor d .

Also worth pocketing: by Eckart–Young–Mirsky, truncated SVD is the optimal rank- k summary ($\|A - B\|_2 = \sigma_{k+1}$, $\|A - B\|_F = (\sum_{i>k} \sigma_i^2)^{1/2}$); power iteration contracts the angle to the dominant direction by σ_2/σ_1 per step (squared on $A^\top A$; $(\sigma_{k+1}/\sigma_k)^q$ over q subspace-iteration steps), so the gap is the convergence rate [? ?]. Eigenvalues of a Kronecker sum are pairwise sums, $\lambda(A \oplus B) = \{\lambda_i(A) + \lambda_j(B)\}$, and $\rho(A) < 1$ gives the Neumann expansion $(I - A)^{-1} = \sum_{t \geq 0} A^t$ with geometrically decaying truncation error [?]; orthogonal projections are non-expansive. For fixed x , the first d rows of a random orthogonal matrix satisfy $\mathbb{E}\|Ax\|_2^2 = (d/D)\|x\|_2^2$ with chi-square tails, so $d = \Theta(\varepsilon^{-2} \log(1/\beta))$ dimensions give $(1 \pm \varepsilon)$ distortion (distributional Johnson–Lindenstrauss) [?].

8.2 Three views from the corpus

Effective clustering for large multi-relational graphs

An NP-hard normalized-cut-type objective over a dense multi-relational affinity is relaxed spectrally, with Sinkhorn–Knopp scaling and a Neumann series replacing a full eigendecomposition.

Theorem 8.4 — doubly stochastic affinity gives k -means. If S_ω is doubly stochastic and the relaxed embedding is $Z = Y^\top$, then optimizing the relaxed objective is equivalent to optimizing

$$\min_{C_1, \dots, C_K} \sum_{k=1}^K \sum_{v_i \in C_k} \|z_i - c^{(k)}\|_2^2, \quad c^{(k)} = \sum_{v_j \in C_k} \frac{y_j}{|C_k|},$$

where $c^{(k)}$ is the center of cluster C_k .

The spectral work sits in producing Y : a truncated Neumann series $(I - Z)^{-1} = \sum_{t \geq 0} Z^t$, valid since the dominant eigenvalue obeys $|\lambda| < 1$, with truncation error after L terms decaying geometrically (on the order of $(\alpha/(1 + \alpha))^{L+1}$) at a rate governed by an eigengap; the supporting lemmas are that expansion, the Weyl/Lidskii-type inequality $\lambda_{\max}(A + B) \leq \lambda_{\max}(A) + \lambda_{\max}(B)$ for Hermitian matrices, and a CountSketch estimate of residual Frobenius norms. [?]

Subspace collision for high-dimensional approximate nearest neighbor search

k -ANNS by counting “collisions” of points with the query across many s -dimensional subspaces needs projections that preserve neighbor orderings.

Theorem 8.5 — ordering preservation. Let $B = [B_1 \dots B_{N_s}] \in \mathbb{R}^{d \times (N_s \cdot s)}$ with $N_s \cdot s \leq d$ have orthonormal columns, i.e. eigenvectors of the covariance, so that $BB^\top$ is an orthogonal projection. Suppose the local difference vectors satisfy $\|(I_d - BB^\top)(o_i - o_j)\|^2 \leq \varepsilon \|o_i - o_j\|^2$ for some $\varepsilon \in (0, 1)$. Then $\|B^\top(o_i - o_j)\|^2 \geq (1 - \varepsilon) \|o_i - o_j\|^2$. Consequently, whenever $\|o_i - o_j\|^2 < (1 - \varepsilon) \|o_i - o_z\|^2$, the ordering survives the transformation: $\|B^\top(o_i - o_j)\| < \|B^\top(o_i - o_z)\|$.

The load-bearing facts are Pythagoras and interlacing: the lower bound is the decomposition $\|x\|^2 = \|B^\top x\|^2 + \|(I - BB^\top)x\|^2$ plus the hypothesis, the upper bound the non-expansiveness of orthogonal projections. Design is settled by Poincaré separation: since the eigenvalues of each compressed covariance $B_j^\top \Sigma B_j$ interlace those of Σ , monotonicity of the logarithm turns log-determinant subspace allocation into an eigenvalue assignment. [?]]

Weighted Fourier factorizations for private marginal queries (PODS)

Under μ -GDP Gaussian noise, all marginal queries induced by a collection $\mathcal{S}$ of subsets of $[d]$ must be answered, with weights and a factorization chosen to minimize weighted error.

Theorem 8.6 — weighted factorization attains $\gamma_{2,F}$. With workload matrix W for the marginal workload $Q_{\mathcal{S}}$ over $U = U_1 \times \dots \times U_d$, $U_i = \{0, \dots, m_i - 1\}$, weights $p : \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}$, and P the diagonal matrix having entries $P_{(S,t),(S,t)} = p(S)/|U_S|$ for $t \in U_S$, the factorization $LR = W$ of the paper’s Algorithm 1 satisfies

$$\gamma_{2,F}(PW) = \|PL\|_F \|R\|_{1 \rightarrow 2} = \left(\sum_{R \in \mathcal{S}^\downarrow} \sum_{\substack{T \in \mathcal{S} \\ T \supseteq R}} \sqrt{\sum_{j \in R} \frac{p(T)}{|U_T|} (m_j - 1)^2} \right)^{1/2},$$

where $\mathcal{S}^\downarrow$ is the downward closure of $\mathcal{S}$. Moreover, for the weights p^* maximizing the right-hand side over the simplex $\{p \geq 0 : \sum_S p(S) = 1\}$, the resulting mechanism is μ -GDP for all of $Q_{\mathcal{S}}$, its worst-case error equals $1/\mu$ times the same expression at p^* , and this bound is optimal among all factorization mechanisms.

Nothing here is spectral in the eigenvalue sense—the machinery is *factorization norms*: the error of a Gaussian-noise factorized release is governed by $\|PL\|_F \|R\|_{1 \rightarrow 2}$, identified with the factorization norm $\gamma_{2,F}$;

first-order (KKT) conditions on the weight LP pin p^* , and Gaussian concentration lifts average error to worst-case error. $[?]^{PODS}$

When to reach for spectral tools

- **Relaxing a combinatorial objective.** Eigenvectors of the (Sinkhorn-balanced, doubly stochastic) affinity turn NP-hard partitioning into k -means on an embedding; Neumann series and eigengaps control the approximation.
- **Certifying invertibility of a structured matrix.** For Kronecker sums and $I - Z$, localize the spectrum (pairwise sums; spectral radius below one); positivity or contraction yields existence, geometric approximation, convergence rates.
- **Choosing or defending a subspace.** Interlacing says compressions inherit sandwiched eigenvalues—use it to optimize what to project onto; Davis–Kahan bounds the drift of the chosen subspace by perturbation over eigengap.
- **Making distances cheap.** Random orthogonal projections are approximate isometries with chi-square tails (distributional JL), so projected thresholds separate near from far with controlled error probabilities.
- **Bounding error of a noisy linear mechanism.** When the strategy is a factorization $LR = W$, error is a product of operator norms ($\|PL\|_F \|R\|_{1 \rightarrow 2} = \gamma_{2,F}$); when noise is a sum of independent random matrices, matrix Bernstein takes over.
- **Caveats.** Weyl, Davis–Kahan, and interlacing need symmetric structure—otherwise argue about singular values or a Schur form; eigengap hypotheses are load-bearing; JL dimensions $d = \Theta(\epsilon^{-2} \log(1/\beta))$ decide whether projecting is actually cheaper.

9

Amortized Analysis and Potential Functions

Amortized analysis is the systems community's favorite certificate that rebuilds, compression, merges, and splits are *free*: a single operation may detonate an $O(n)$ reorganization, but if many cheap operations must accumulate before the next detonation becomes possible, the cost per operation stays $O(1)$. Eleven papers in this corpus reach for the amortized/potential toolbox, and they own one of the most lopsided tool–property pairings in the survey: the pair *upper-bound* $\times$ *amortized-potential* occurs in ten of the eleven. The tool almost always arrives to certify a per-operation time or space bound that an honest worst-case statement could not quote.

9.1 *The tool*

Three equivalent framings [?]:

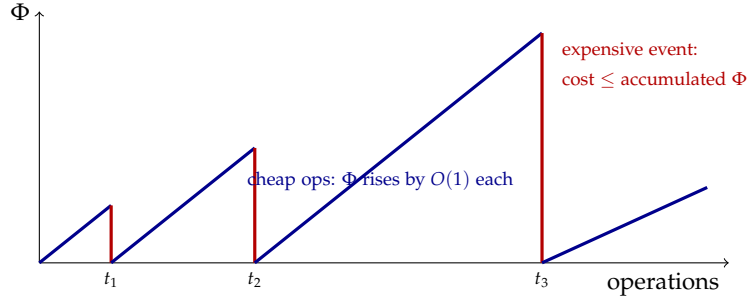
- **Aggregate method:** bound the total cost $T(n)$ of the worst sequence of n operations and report $T(n)/n$.
- **Accounting (banker's) method:** charge each operation a fictitious amortized price; cheap operations bank the surplus, expensive operations spend it, and the balance must never go negative.
- **Potential (physicist's) method:** fix a function Φ from data-structure states to $\mathbb{R}_{\geq 0}$ and define the amortized cost $\hat{a}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$. Telescoping gives $\sum_i c_i \leq \sum_i \hat{a}_i$ whenever $\Phi(D_n) \geq \Phi(D_0)$.

The design recipe for Φ has three canonical rules:

1. **Nonnegativity.** $\Phi(D) \geq 0$ for every reachable state (typically $\Phi(D_0) = 0$) — this is what makes the telescoping inequality safe for every prefix of the sequence.

2. **Deposits.** Every cheap operation raises Φ by at most its amortized cost: $+O(1)$ per insertion into a counter, a slack variable, or a size-excess budget.
3. **Withdrawals.** Every expensive event consumes a *large* accumulated Φ : it can fire only once some measurable quantity (counter, excess, rounding slack) has crossed a threshold, and Φ then drops by at least the event's cost.

Figure 9.1: The sawtooth: cheap operations deposit into Φ ; an expensive event fires only when enough has accumulated, and its cost is paid entirely by the drop in Φ . Doubling the event cost doubles the inter-event gap.



Amortized bounds beat worst-case bounds exactly when expensive events are rare *because they require accumulation*: a worst-case bound must quote the burst cost C for every operation, while the amortized bound divides C by the $\Theta(\text{accrual})$ operations that had to precede it. The classic showcases are dynamic tables, Fibonacci heaps, and splay trees, where Φ is a sum of imbalance terms [? ?]; periodic global rebuilding is the degenerate special case “rebuild cost over ℓ ” for period ℓ [?]. The signature recipe in this corpus is *rare-event bounding by a potential function*: a GK-style quantile sketch holds at most $2s \ln(n/s + 2)$ tuples because every compaction spends accumulated error slack [?], and a B-tree root split needs amortized $\Omega(b^{h+2})$ insertions before it can recur. The price of the method: the bound is a promise about *sequences*, not single operations — latency- or adversary-sensitive settings require deamortized variants.

9.2 Five views from the corpus

The five exemplars below exhibit progressively looser readings of the method: an explicit sawtooth potential, an implicit batching argument, potential as rounding slack, amortization imported as a black box, and potential as a progress measure.

9.2.1 SplineSketch: the sawtooth, literally

A streaming quantile sketch maintains k buckets over the keys seen so far, each with a counter b_i of absorbed items; arrivals increment coun-

ters, a bucket whose counter violates its budget ($b_i > C_b \cdot n/k$) must *split*, and adjacent buckets are *joined* to reclaim space. The maintenance question is how many splits a stream of n items can force.

Lemma 9.1 — Split budget. Combining the lower bound on the decrease due to a split with the upper bound on the total potential, the number of splits during $(t', t]$ is at most

$$\frac{t}{\beta \cdot C_b \cdot \frac{t}{2.5k}} = \frac{2.5k}{\beta \cdot C_b} \leq \frac{k}{18},$$

where C_b is chosen based on β .

The potential is a function of the bucket counters — their excess over the budget $C_b \cdot n/k$: every arrival deposits $O(1)$, and the expensive event, a split, consumes a constant fraction β of what has accumulated, which is literally the sawtooth of Figure 9.1. The split budget then lets the rank error be charged along the split history, giving $\varepsilon^n(\sigma_i) \leq O(\log \alpha \cdot n/k)$ for any threshold σ_i of the final sketch [?].

9.2.2 AeroSketch: batching the re-decomposition

Tracking a covariance sketch of a row stream requires repeatedly extracting the top eigenvectors of a $d \times d$ matrix — far too expensive to redo on every arrival.

Theorem 9.2 The algorithm solves the ATTP matrix-sketch problem with space complexity $O(d \log \|A\|_F^2)$ and amortized time complexity $O(d \log d)$ per update; with probability amplification it achieves amortized time complexity $O(d \log d \log^2 \frac{1}{\delta})$ per update while guaranteeing success probability at least $1 - \delta$, where δ is a tunable parameter.

There is no explicit Φ here: the expensive event is the eigen-decomposition of the accumulated covariance, executed by simultaneous iteration in $O(d \log d)$ using $\log_2 d + 1$ power-iteration steps, and it fires only when a dump threshold θ is crossed; each arriving row pays its share of the burst, i.e. the amortized cost is burst/ℓ for batch length ℓ . A decomposability lemma — per-site sketches compose additively in covariance error — then carries the same accounting to sliding windows and distributed sites [?].

9.2.3 Approximate query processing under updates: slack as potential

For free-connex conjunctive queries over a semiring, exact incremental view maintenance is blocked by dynamic lower bounds outside the tiny q -hierarchical class. The escape is to keep auxiliary views only approximately: each cached annotation is rounded up within a multiplicative factor b and refreshed lazily when the true value outgrows the cache.

Theorem 9.3 For any $b > 1$ and any free-connex query $Q(H, F)$, all auxiliary views can be updated in amortized $O(\log_b M)$ time under an insertion-only update sequence, where $M = \max_{i,e} \max_{t \in R_e^i} A_{R_e}^i(t)$. Applied to maintaining the join size of a 3-way join, which has $M = O(N^2)$, the amortized update cost is thus $O(\log_b N)$.

The potential is the total rounding slack $\sum_t \log_b A_R(t)$: an insertion deposits $O(1)$, and the expensive event — a cascading recomputation up the view hierarchy, triggered when a tuple’s annotation exceeds its cached value by the factor- b slack — consumes a unit, so each tuple can fire at most $\log_b M$ times. The insertion-only hypothesis is essential: fully-dynamic sequences face an amortized $\Omega(\sqrt{N})$ lower bound for non-q-hierarchical queries, and it holds even under approximation [?].

9.2.4 Subset sampling over joins: amortization imported

One-shot and dynamic subset sampling over an acyclic join Q must draw each result tuple $u \in \text{Join}(Q)$ independently with probability $p(u)$, where p factorizes over the relations of Q ; the index shards tuples into weight buckets — each sub-instance is either light or near-uniform — with a meta-index steering rejection sampling across buckets.

Theorem 9.4 Given an acyclic join instance Q and a set of associated weight functions $\{p_j^R\}_{j \in Q}$, we can maintain a dynamic index under tuple insertions using $O(N \log N)$ space, with amortized update time $O(\log^3 N \log \log N)$, while supporting the subset sampling query in $O(1 + \mu_\Psi \log N)$ expected time, where N is the total number of insertions.

The expensive events are index rebalancing and the periodic re-derivation of per-bucket statistics as insertions shift tuples across weight classes; the amortized $O(\log N)$ maintenance primitive is imported rather than re-proved (the paper’s Lemma 5.1, from prior work: the direct-access join index over real and dummy tuples updates in amortized $O(\log N)$ per inserted tuple). The composability lesson: a data structure with an amortized guarantee can serve as a black-box component inside a larger amortized analysis [?]^{PODS}

9.2.5 Jaguar: potential as progress measure

Jaguar evaluates conjunctive queries in submodular-width time by recursively decomposing the instance; at each node, CALIBRATE re-derives size bounds $g(Z)$ for subsets Z of the query variables — in fact a shortest-path computation on a weighted graph whose vertices are the subsets of V .

Theorem 9.5 Let $\varepsilon > 0$ be an arbitrary constant. Given a CQ Q , a set of statistics (Σ, n) , and a database instance D_0 that matches the schema of Q and satisfies the statistics (Σ, n) , Jaguar correctly computes $Q(D_0)$ in time $O(N^{\text{subw}(Q, \Sigma, n) + \varepsilon} + \text{OUT})$ in data complexity, where $N = |D_0|$ and OUT is the output size. In particular, after a preprocessing time of $O(N^{\text{subw}(Q, \Sigma, n) + \varepsilon})$, Jaguar supports constant-delay enumeration of the output tuples.

Here the potential is not a bank but a *progress measure*: the log-relation-size bound g . Light recursion edges leave the statistics intact (a light path has length at most $2^{|V|} = O(1)$), while each heavy edge decreases the potential by at least ε — so any root-to-leaf path carries at most $O(1)$ heavy edges and the recursion tree has $O(1)$ depth in data complexity. The expensive event is the heavy, statistics-tightening recursive call: it can fire only while potential remains to burn, and with linear-time CALIBRATE plus the invariant $c \leq \text{subw}(Q, \Sigma, n)$ on every calibrated bound, no level of the constant-depth tree exceeds the advertised $N^{\text{subw}(Q, \Sigma, n) + \varepsilon}$ [?] ^{PODS}

When to reach for amortized analysis

- You need a *per-operation* time or space bound, and the algorithm has an occasional expensive structural event — split, merge, rebuild, compression, re-decomposition, full scan — that only many cheap operations can set up.
- The trigger is measurable: some counter, excess, or slack that rises by $O(1)$ per cheap operation and drops sharply when the event fires. If you can write it as a Φ obeying the three rules, the bound follows by telescoping — and you get to tune the regime (batch length ℓ , slack ratio b , bucket budget C_b) to trade burst size against frequency.
- Sequences, not single operations: if the deployment cares about tail latency or faces an adversary who inspects timings, budget for deamortized variants; and remember that insert-only analyses can shatter under deletions (the $\Omega(\sqrt{N})$ fully-dynamic lower bound above).

10

Coresets and Randomized Numerical Linear Algebra

This chapter is being revised; see Chapter [o](#) for the survey context in the meantime.

11

Information-Theoretic Bounds

This chapter is being revised; see Chapter [o](#) for the survey context in the meantime.

12

Communication Complexity

This chapter is being revised; see Chapter [o](#) for the survey context in the meantime.

13

Online decision-making: competitive analysis and regret

Online decision theory enters this corpus through systems doors: when to adapt a filter to a recurring false positive, whether to trust an ML oracle’s prediction, how to clean a time series that has not yet arrived, how to tune a discretization knob. Four papers carry it—two via *competitive analysis* [? ?], two via *regret* [? ?], none an online-algorithms paper per se. They share one shape: the system commits to decisions while the input’s future is unknown, and the committed sequence is judged against a hindsight optimum that sees everything. Two classical yardsticks quantify that gap, and choosing between them is this chapter’s lesson.

13.1 The tools

Definition 13.1 (Competitive ratio). An online algorithm A for a cost-minimization problem is c -competitive if there is a constant b such that

$$\text{cost}_A(\sigma) \leq c \cdot \text{cost}_{\text{OPT}}(\sigma) + b \quad \text{for every input sequence } \sigma,$$

where OPT is the offline optimum that observes all of σ —including where it ends—before acting.

The canonical micro-example is *ski rental*: skis rent for 1 per day and cost B to buy; each morning the skier learns only whether the season continues. Renting for B days and then buying is 2-competitive, and no deterministic algorithm does better; randomization lowers the barrier to $e/(e-1) \approx 1.58$ against an oblivious adversary [?]. The framework descends from the amortized analysis of list update and paging [?]. Figure 13.1 shows the geometry; the memoryless “when do I commit?” structure is why this toy model reappears below in database clothing.

Definition 13.2 (Regret). For decisions $a_1, \dots, a_T$, where round t charges a loss $\ell_t(a_t)$ revealed only after the decision, the *regret* against the best

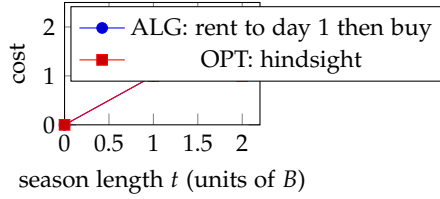


Figure 13.1: Ski rental with rent 1 and buy price $B = 1$: rent-to- B -then-buy matches OPT while $t < B$ and never exceeds $2 \cdot \text{OPT}$.

fixed comparator is

$$R_T = \sum_{t=1}^T \ell_t(a_t) - \min_{a \in \mathcal{A}} \sum_{t=1}^T \ell_t(a).$$

An algorithm is *no-regret* if $R_T = o(T)$: its average loss converges to that of the best action chosen once, in hindsight.

In the stochastic multi-armed bandit (K actions, rewards in $[0, 1]$ with fixed unknown means, only the chosen arm's reward observed), UCB plays the arm with the highest optimism index $\bar{r}_k + \sqrt{2 \ln t / n_k}$ and achieves $R_T = O(\sqrt{KT \log T})$ [?]; adversarial and unified views are surveyed in [?]. In short: the *competitive ratio* prices the unknown future—unrestricted adversary, switching comparator, guarantee on every input—for one-shot commitments under adversarial workloads; *regret* prices ignorance of a distribution—repeated rounds, per-round feedback, fixed comparator, $O(\sqrt{T})$ bound—for stochastic tuning loops.

13.2 Four exemplars from the corpus

13.2.1 Ski rental inside a sketch filter

A streaming distinct-count filter occasionally yields a false positive; each repeat costs a database lookup C_{DB} , adapting the entry costs an extra reverse-map lookup $C_{RM} = \alpha C_{DB}$, and after adapting both costs stop. Whether and when to adapt is ski rental in disguise—rental price C_{DB} , purchase price $C_{DB} + C_{RM}$, adversarial arrival stream.

Theorem 13.3 — Adapt after $\lfloor \alpha \rfloor$ repeats; 2-competitive. If $\alpha \geq 1$, i.e., the cost of querying the reverse map is greater than a database lookup, the optimal deterministic strategy is to adapt immediately after the false positive repeats $\lfloor \alpha \rfloor$ times, and is 2-competitive.

Remark 13.4 (The golden-ratio threshold). For $\alpha < 1$ the two candidate policies have ratios $1 + \alpha$ (adapt immediately) and $(2 + \alpha)/(1 + \alpha)$ (adapt on the second false positive); these cross where $\alpha^2 + \alpha - 1 = 0$, i.e. at $\alpha = (\sqrt{5} - 1)/2 \approx 0.618$. The optimal rule: adapt immediately when $\alpha < 0.618$, adapt on the second occurrence otherwise.

The competitive ratio is the right yardstick: the arrival stream is adversarial and the offline comparator knows when the repeats end. The

skeleton also composes—augmenting a strongly adaptive filter with this routine bounds false positives by $(1 + \lceil \alpha \rceil)(n\epsilon + O(\sqrt{n\epsilon \log n + \log n}))$ with high probability, the 2-competitive core living inside a high-probability bound. [?]

13.2.2 Paying for oracle predictions at enumeration time

Subgraph-isomorphism (SUBISO) enumeration is accelerated by an ML oracle that prunes the search at prediction cost $f(Q, M)$, but a wrong prediction misses matches. The paper’s β -robustness—extra output cost relative to the optimal exact enumerator—is a competitive ratio; the general case is barred, then per-algorithm ratios are proved.

Theorem 13.5 — No constant robustness in general. No algorithm exists for SUBISO that is both output polynomial and β -robust for any positive constant β unless $\text{fewP} = P$, where fewP denotes the class of problems recognizable by a nondeterministic Turing machine in polynomial time with only polynomially many solutions.

For the paper’s oracle-guided hybrids the matching upper bounds are constant competitive ratios, e.g. $5N_c(f(Q, M) + 2|G||Q|)$ for VF3_M with probability at least $1 - \epsilon$, proved via a generalized Bonferroni inequality over the oracle’s false predictions. The barrier is a parsimonious reduction; the ratios fit competitive analysis since the graph is arbitrary and the comparator is the exact enumerator’s cost—paid in oracle calls and output quality rather than money. [?]

13.2.3 Windowed time-series repair with decay regret

Given a multivariate time series violating soft and hard constraints, the offline optimal repair is computed exactly as the maximum-weight path in a DAG over anchor points. The online algorithm is the same dynamic program restricted to a sliding window of width W , so its regret is precisely what the truncated lookback misses.

Theorem 13.6 — Exponentially decaying window regret. The regret R_n of the online algorithm is bounded:

$$R_n = \sum_{i=1}^n r_i \leq \sum_{i=1}^n e^{-\beta(W+1)} = n e^{-\beta(W+1)},$$

where each one-step regret r_i compares the score of the offline optimal anchor point with that of the best anchor visible within the window, and the strictly decreasing path weights obey $\gamma(\Delta) \leq e^{-\beta\Delta}$.

The comparator is a best *sequence* (the exact offline optimum), a competitive-analysis comparator, yet the guarantee is a sum of per-step gaps in the regret style; the exponential decay makes a bounded

window affordable—each extra unit of window multiplies the neglected tail by $e^{-\beta}$. [?]

13.2.4 Attribute discretization as a bandit

Choosing how to bin numerical attributes is posed as maximizing $J(B, D) = \alpha M_{\text{sem}}(B) + (1 - \alpha) M_{\text{util}}(B, D)$ over partition sets, with the search organized as an MDP over clusters of similar partitions. The only provable guarantee comes from the case where the search collapses to a repeated choice among K cluster-arms.

Theorem 13.7 — The single-attribute case is exactly a K -armed bandit. If (i) $d = 1$ (single attribute, one action per episode) and (ii) MCC uses the UCB index for action selection and updates, then Algorithm 1 becomes a UCB-guided MCC variant and reduces exactly to a stochastic multi-armed bandit with K arms (one per cluster) and reward in $[0, 1]$ given by J ; this equivalence does not extend to the multi-attribute case ($d > 1$). Running Algorithm 1 for T episodes yields the regret guarantees inherited from UCB: gap-free regret $O(\sqrt{KT \log T})$.

This is the reduction school of regret usage: engineer the system so an existing bandit theorem applies verbatim and inherit $O(\sqrt{KT \log T})$ for free. Regret fits because episodes repeat with stochastic rewards in $[0, 1]$ and the comparator is the best fixed cluster; for $d > 1$ the equivalence breaks and no guarantee is claimed. [?]

When to reach for competitive analysis vs regret

- **Competitive ratio:** the decision is a commitment (adapt, buy, rebuild, trust an oracle) under a possibly adversarial input, and you need a guarantee on *every* run—ski-rental-shaped trade-offs between a repeated small cost and a one-time large one. Deterministic bounds are typically 2; randomization usually improves them once an adversary model is fixed.
- **Regret:** the same decision repeats over many rounds with per-round feedback and tacit stochasticity—tuning, configuration search, repair over stationary error processes. The comparator is a *fixed* action in hindsight; the $O(\sqrt{T})$ bound is weak per run, strong on average.
- **Hybrids are the corpus pattern:** reduce the system to a K -armed bandit and inherit UCB's bound wholesale; wrap a 2-competitive sub-routine inside a high-probability filter bound; measure a windowed online heuristic against the exact offline DP with a decay argument.

- **Check before you cite:** regret needs normalized rewards (here $[0, 1]$) and a defined feedback model; a competitive ratio needs a well-defined offline optimum and, if randomized, an oblivious adversary.